

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统



第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

指令系统总结: RISC vs CISC

CISC (复杂指令集、X86-64)

① 指令系统复杂: 指令多, 寻址方式多, 指令格式多

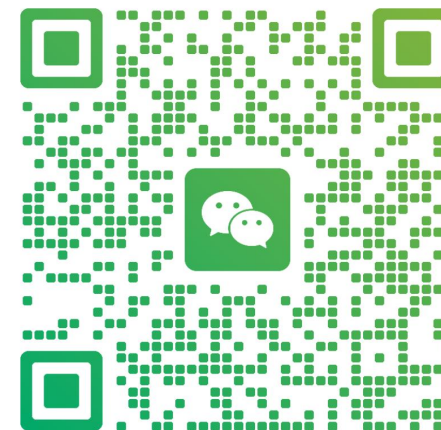
RISC (复杂指令集、MIPS)

**加老汤微信, 帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



扫一扫上面的二维码图案, 加我为朋友。

流水线处理器 (CPU) 的设计：流水线基本原理

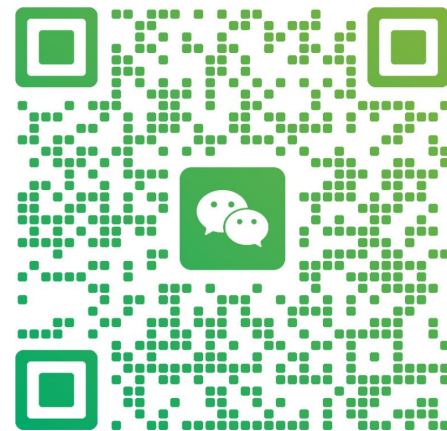
串行方式没有充分利用执行部件的并行性，指令执行效率低

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

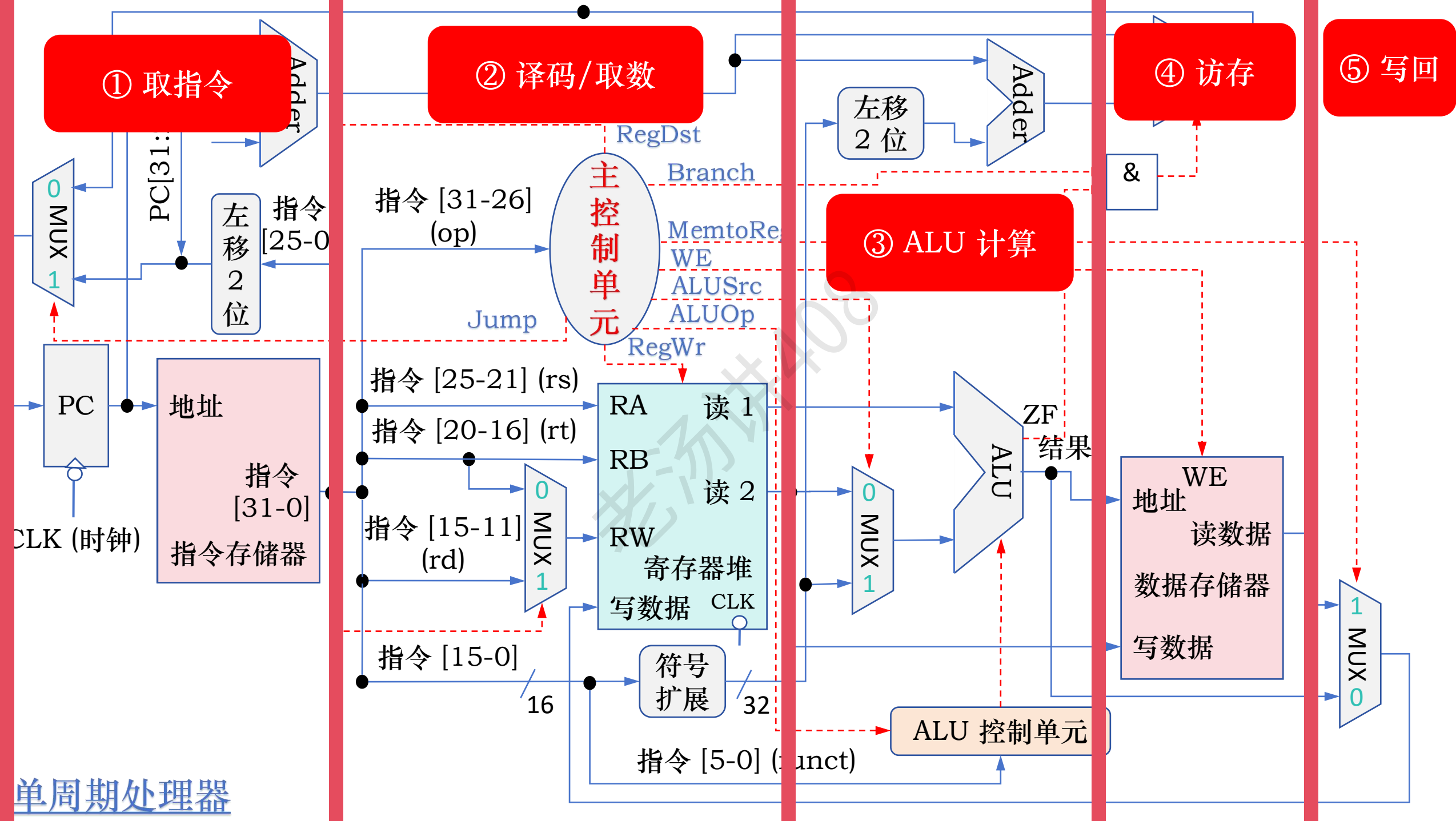


老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。



流水线处理器 (CPU) 的设计：流水线基本原理

串行方式没有充分利用执行部件的并行性，指令执行效率低

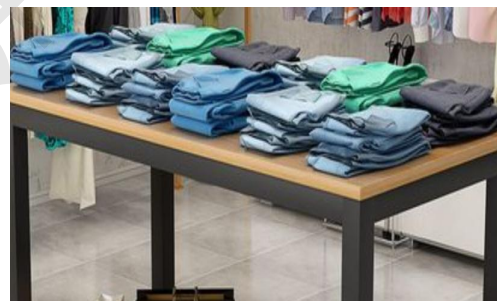
流水线处理器，将多条指令的执行相互重叠起来



洗衣



烘干



叠衣服



放衣柜

流水线处理器 (CPU) 的设计：流水线基本原理

串行方式没有充分利用执行部件的并行性，指令执行效率低

流水线处理器，将多条指令的执行相互重叠起来



流水线处理器 (CPU) 的设计：流水线基本原理

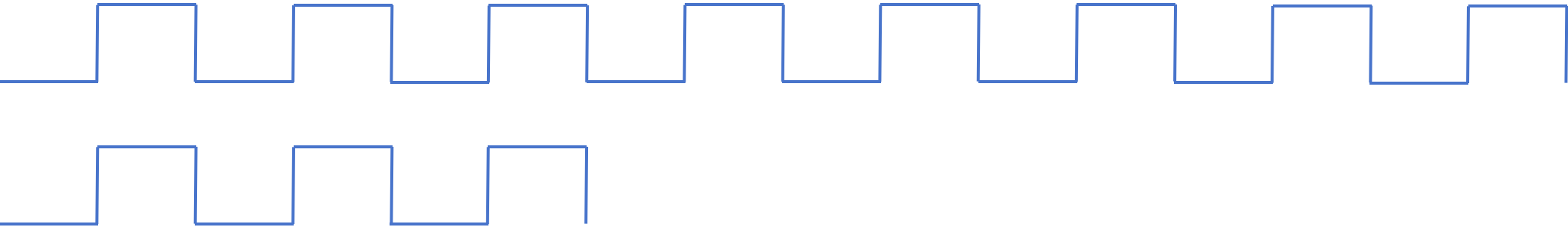
串行方式没有充分利用执行部件的并行性，指令执行效率低

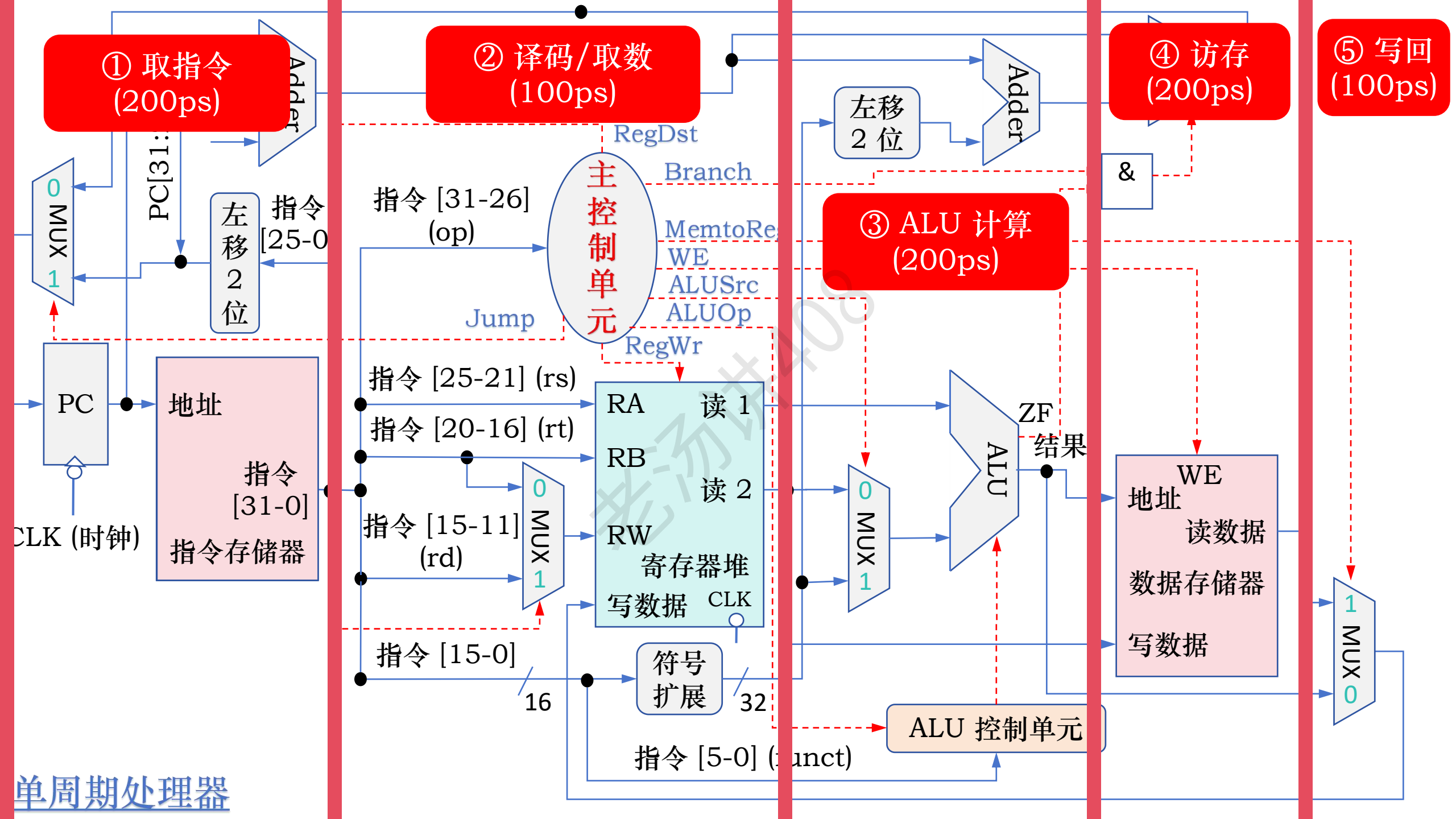
流水线处理器，将多条指令的执行相互重叠起来

```
beq  $4,$5,200
sub  $2,$6,$7
slt  $4,$2,$4
add  $2,$6,$7
lw   $2,200($6)
j    200000
sw   $2,100($6)
```



时钟信号





单周期处理器

流水线处理器 (CPU) 的设计：流水线基本原理

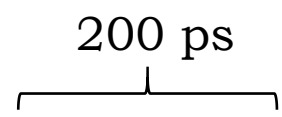
串行方式没有充分利用执行部件的并行性，指令执行效率低

流水线处理器，将多条指令的执行相互重叠起来

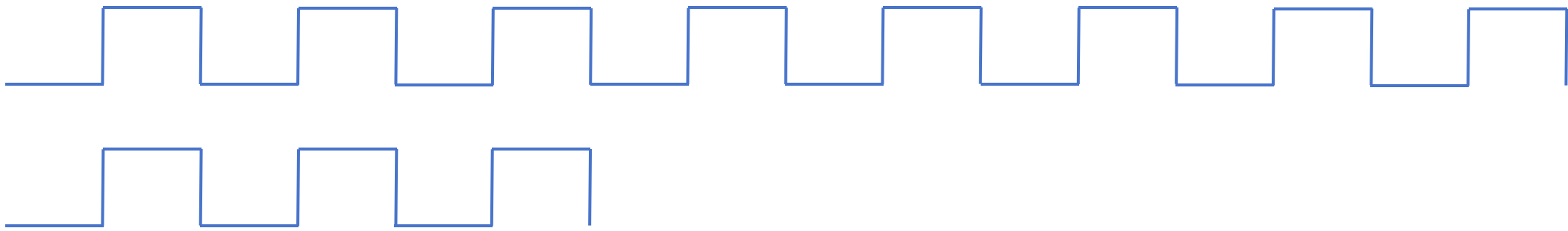
```
beq $4,$5,200
sub $2,$6,$7
slt $4,$2,$4
add $2,$6,$7
lw $2,200($6)
j 200000
sw $2,100($6)
```

$200\text{ps} * 11 = 2200\text{ ps}$

流水段



时钟信号



流水线处理器 (CPU) 的设计：流水线基本原理

指令类型	取指令	译码/取数	ALU 计算	访存	写回	总时间
R 型指令	200ps	100ps	200ps		100ps	600ps
lw 指令	200ps	100ps	200ps	200ps	100ps	800ps
sw 指令	200ps	100ps	200ps	200ps		700ps
beq 指令	200ps	100ps	200ps			500ps
jump 指令	200ps					200ps

单周期处理器时钟周期：800ps $7 \times 800\text{ps} = 5600\text{ps}$

多周期处理器时钟周期：200ps

beq \$4,\$5,200

sub \$2,\$6,\$7

slt \$4,\$2,\$4

add \$2,\$6,\$7

lw \$2,200(\$6)

j 200000

sw \$2,100(\$6)

$3 \times 200\text{ps} = 600\text{ps}$

$4 \times 200\text{ps} = 800\text{ps}$

$4 \times 200\text{ps} = 800\text{ps}$

$4 \times 200\text{ps} = 800\text{ps}$

$5 \times 200\text{ps} = 1000\text{ps}$

$1 \times 200\text{ps} = 200\text{ps}$

$4 \times 200\text{ps} = 800\text{ps}$

5000ps

流水线处理器 (CPU) 的设计：流水线基本原理

beq \$4,\$5,200

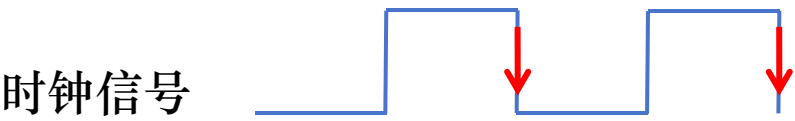
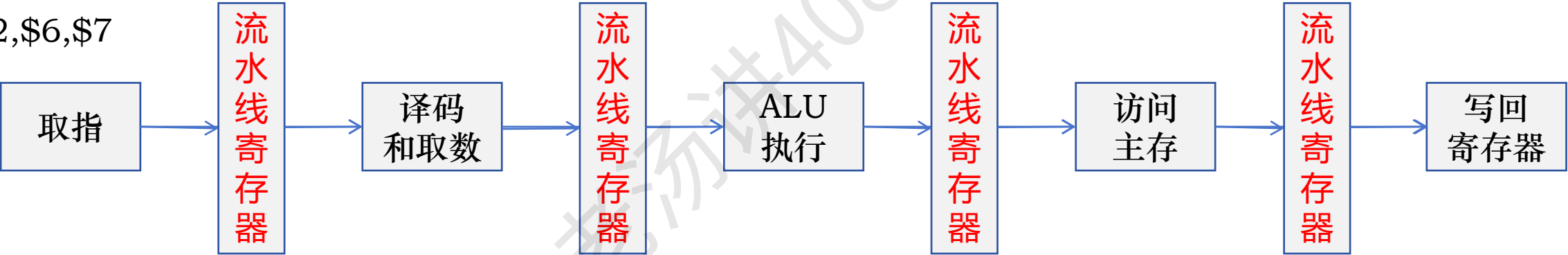


时钟信号



流水线处理器 (CPU) 的设计：流水线基本原理

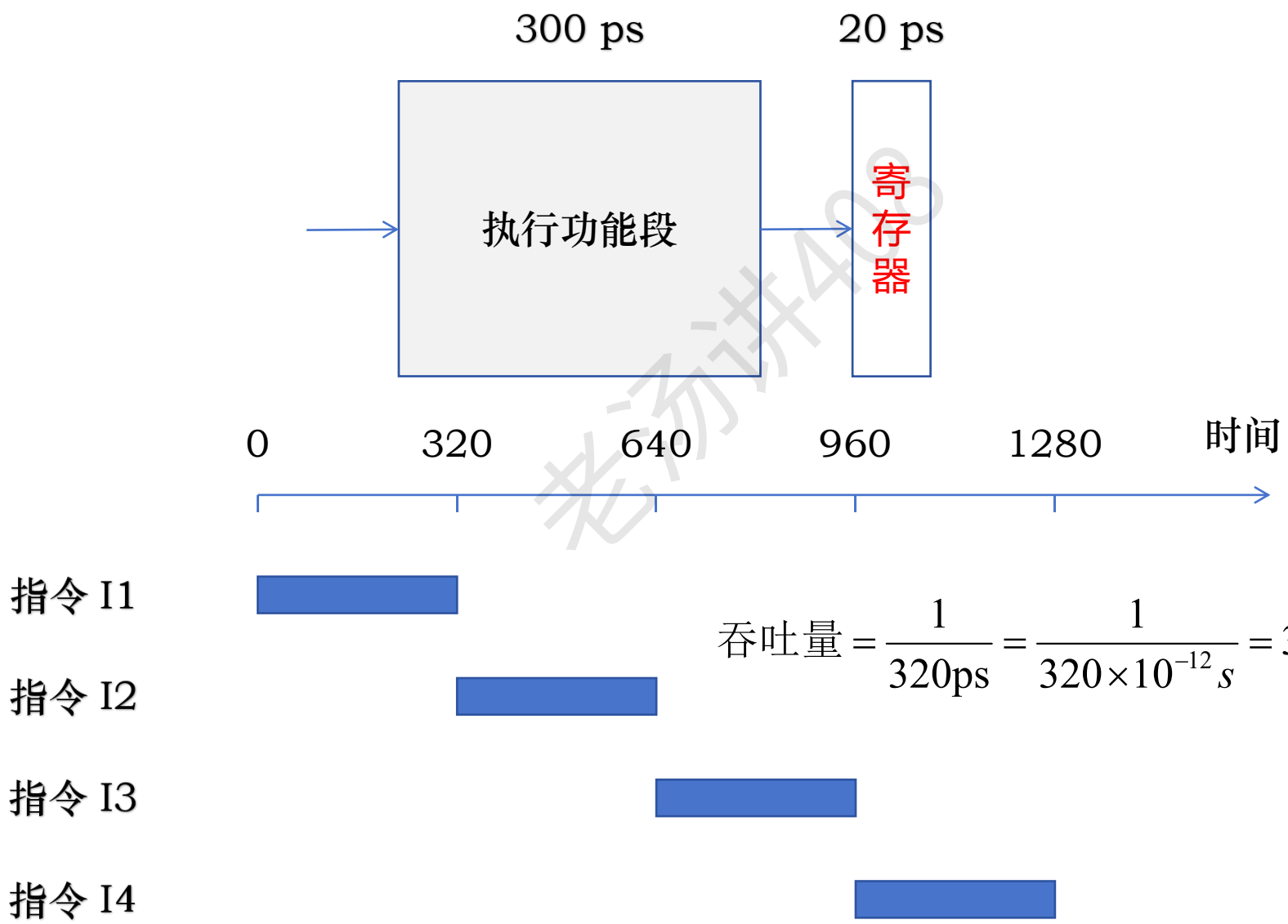
```
beq $4,$5,200
sub $2,$6,$7
```



流水线处理器 (CPU) 的设计：流水线基本原理

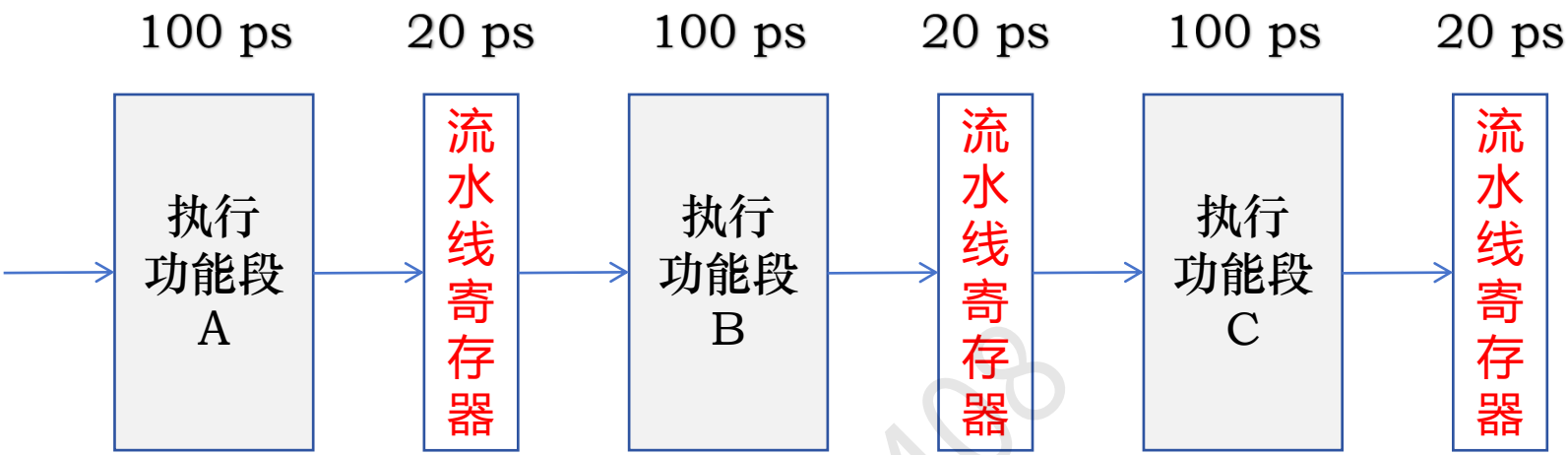
吞吐量，就是 1 秒钟能连续执行的指令的数量，也就是 IPS (Instructions Per Second)

延迟，从头到尾执行一条指令需要的时间

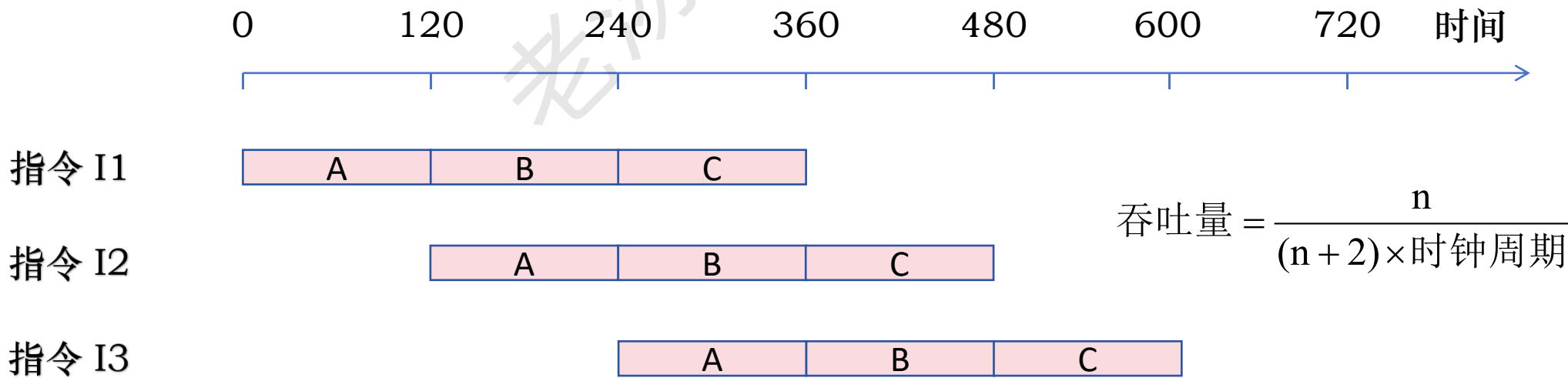


吞吐量 = $\frac{1}{320\text{ps}} = \frac{1}{320 \times 10^{-12} \text{s}} = 3.125 \times 10^9 \text{ IPS} = 3.125 \text{ GIPS}$

流水线处理器 (CPU) 的设计：流水线基本原理



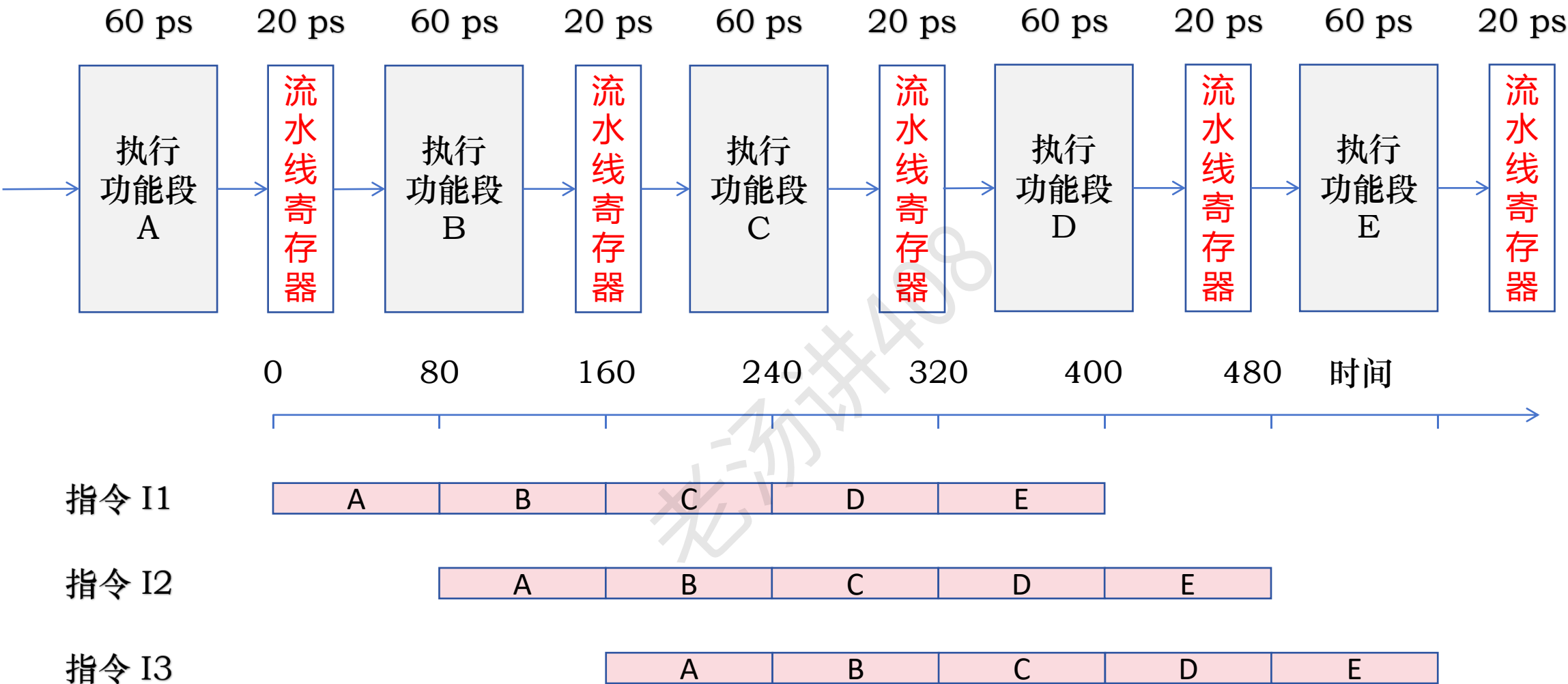
吞吐量 = $\frac{1}{120\text{ps}} = \frac{1}{120 \times 10^{-12}\text{s}} = 8.33 \times 10^9 \text{ IPS} = 8.33 \text{ GIPS}$



吞吐量 = $\frac{n}{(n + 2) \times \text{时钟周期}}$

指令执行时间_{流水线} = 3 × 时钟周期 + (n - 1) × 时钟周期 = (n + 2) × 时钟周期

流水线处理器 (CPU) 的设计：流水线基本原理



指令执行时间_{流水线} = 5 × 时钟周期 + (n - 1) × 时钟周期 = (n + 4) × 时钟周期

流水线处理器 (CPU) 的设计：流水线基本原理

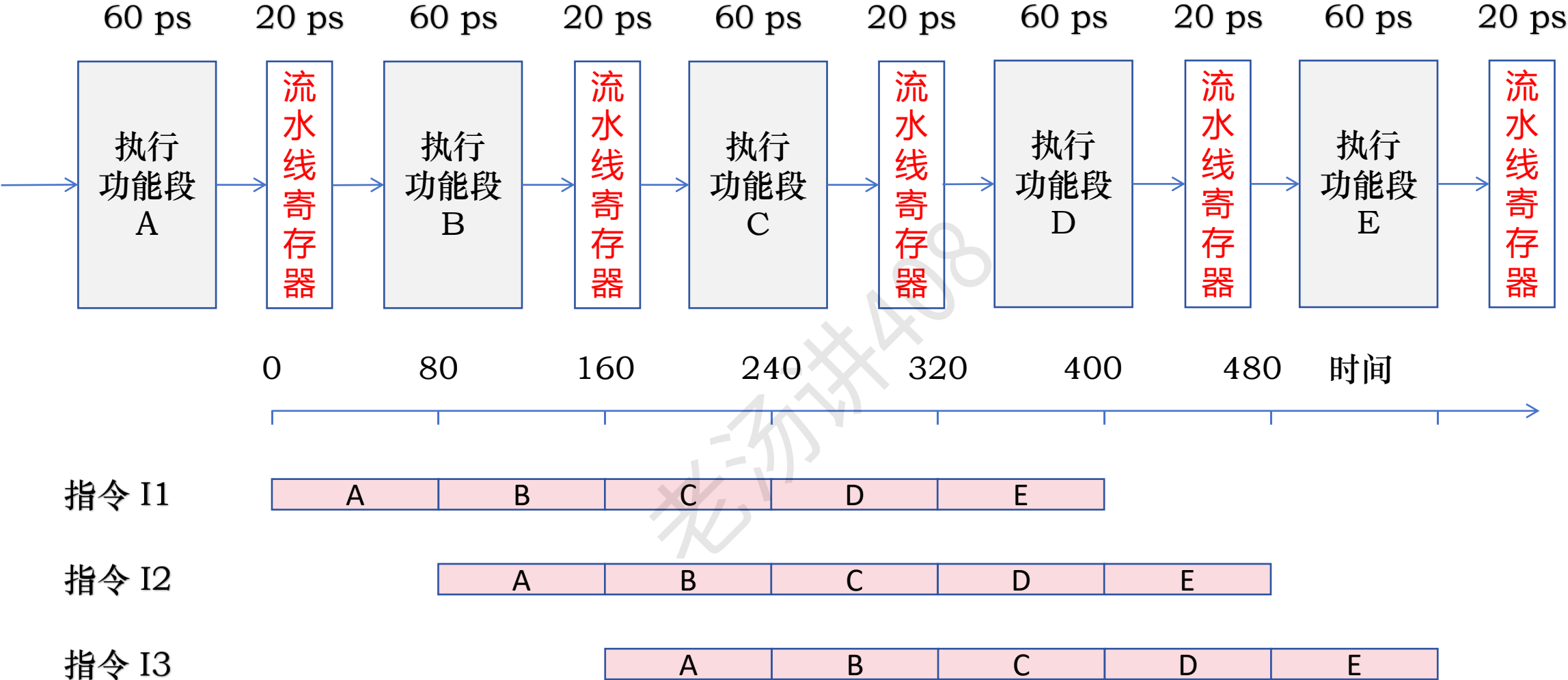
k 级流水线

$$\text{指令执行时间}_{\text{流水线}} = k \times \text{时钟周期} + (n - 1) \times \text{时钟周期} = (n + k - 1) \times \text{时钟周期}$$

$$\text{指令吞吐量} = \frac{n}{(n + k - 1) \times \text{时钟周期}}$$

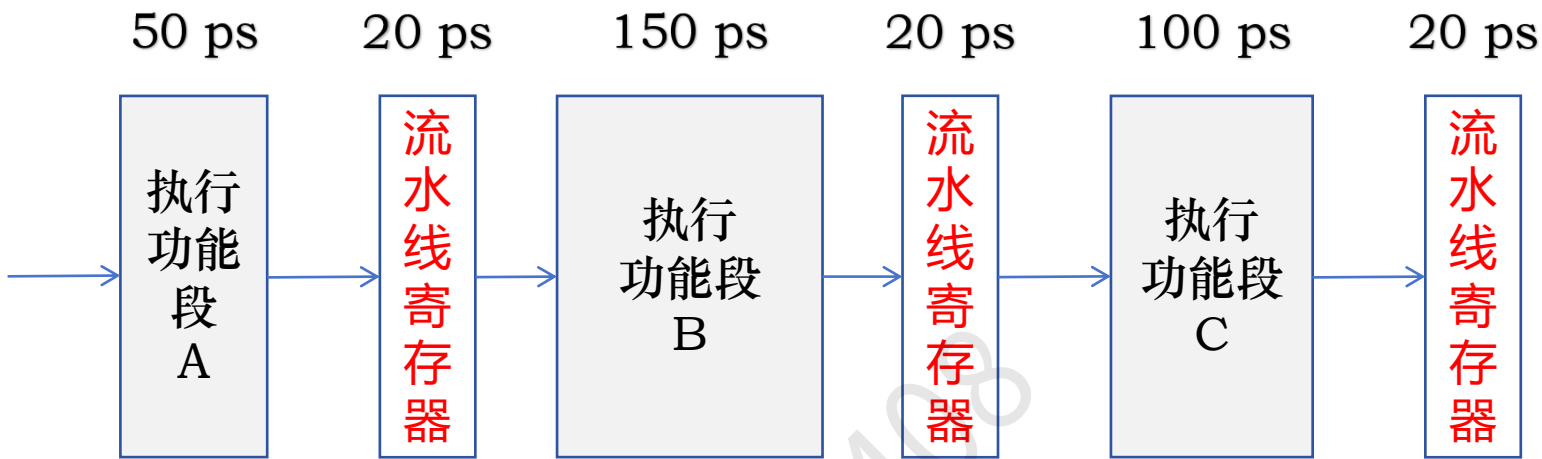
$$\text{指令吞吐量} \approx \frac{1}{\text{时钟周期}}$$

流水线处理器 (CPU) 的设计：流水线基本原理

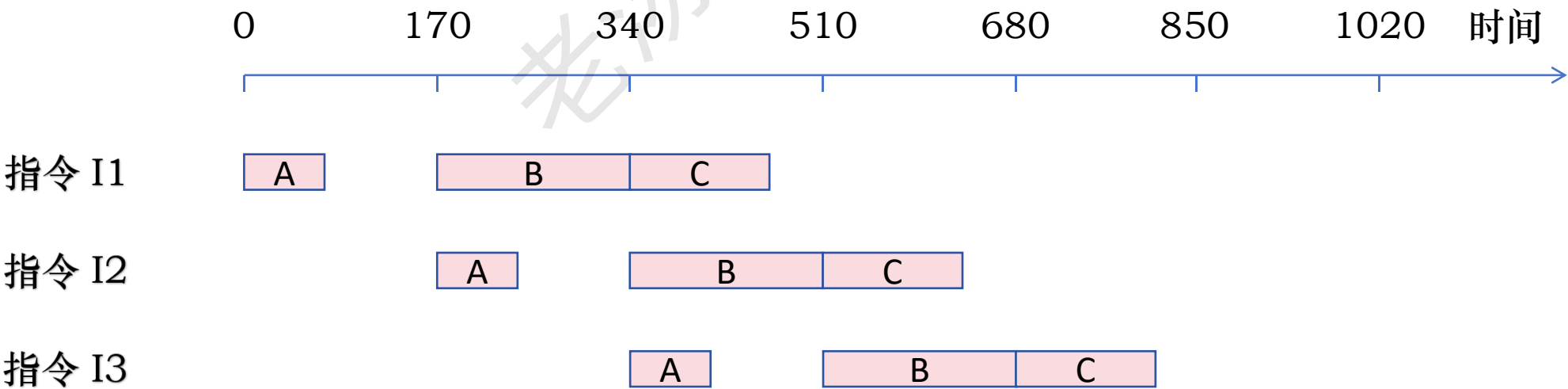


指令吞吐量 = $\frac{1}{80\text{ps}} = \frac{1}{80 \times 10^{-12} \text{s}} = 12.5 \times 10^9 \text{ IPS} = 12.5 \text{ GIPS}$

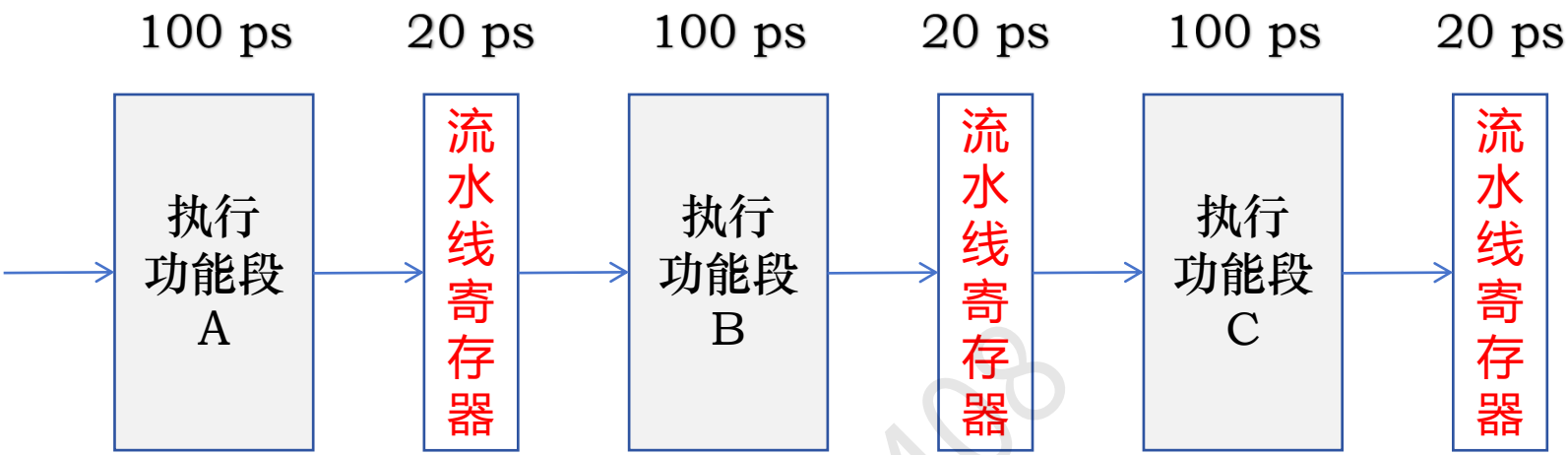
流水线处理器 (CPU) 的设计：流水线基本原理



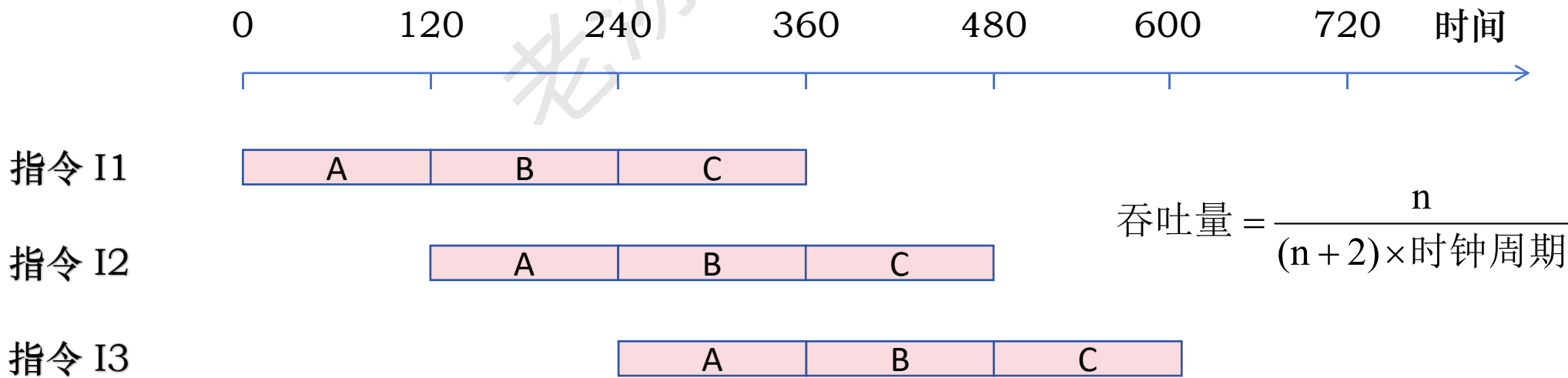
吞吐量 = $\frac{1}{170\text{ps}}$ = $\frac{1}{170 \times 10^{-12}\text{s}}$ = $5.88 \times 10^9 \text{ IPS}$ = 5.88 GIPS < 8.33 GIPS



流水线处理器 (CPU) 的设计：流水线基本原理



吞吐量 = $\frac{1}{120\text{ps}} = \frac{1}{120 \times 10^{-12}\text{s}} = 8.33 \times 10^9 \text{ IPS} = 8.33 \text{ GIPS}$



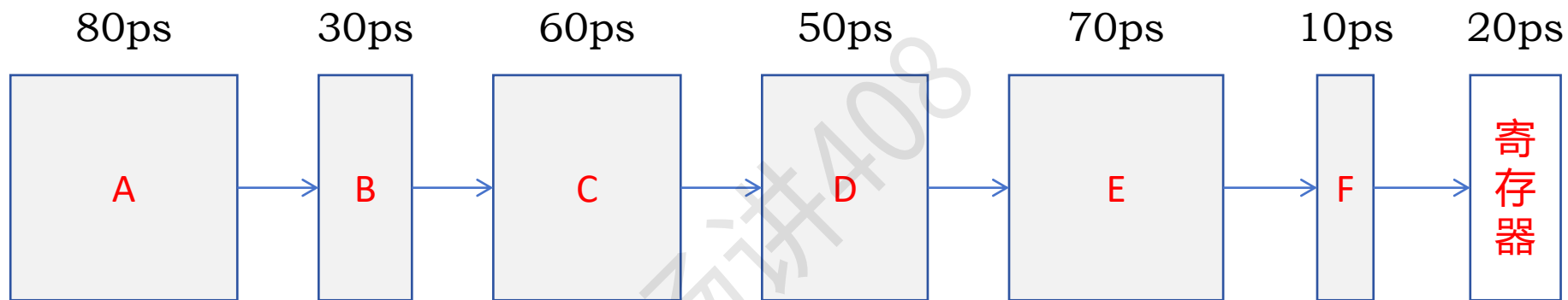
吞吐量 = $\frac{n}{(n + 2) \times \text{时钟周期}}$

指令执行时间_{流水线} = $3 \times \text{时钟周期} + (n - 1) \times \text{时钟周期} = (n + 2) \times \text{时钟周期}$

流水线处理器 (CPU) 的设计：流水线基本原理

根据在哪里插入流水线寄存器，会出现不同的流水线深度 (有多少个阶段) 和最大吞吐量的组合。

假设每个流水线寄存器的延迟为 20ps



只插入一个寄存器，得到一个两阶段的流水线

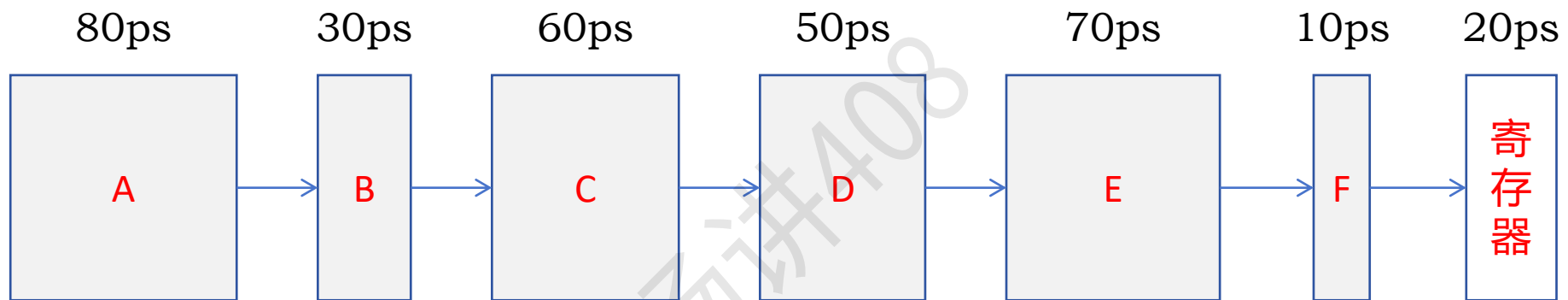
要使**吞吐量最大化**，该在哪里插入寄存器呢？吞吐量和延迟是多少？

$$\text{流水线指令吞吐量} = \frac{1}{190\text{ps}} = \frac{1}{190 \times 10^{-12}\text{s}} = \frac{10^{12}}{190\text{s}} = 5.26\text{GIPS}$$

流水线处理器 (CPU) 的设计：流水线基本原理

根据在哪里插入流水线寄存器，会出现不同的流水线深度 (有多少个阶段) 和最大吞吐量的组合。

假设每个流水线寄存器的延迟为 20ps

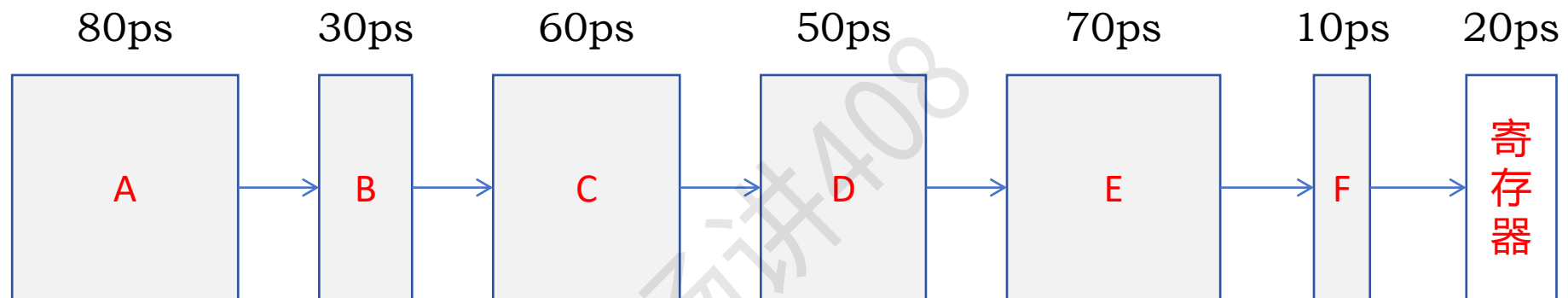


要使一个三阶段的流水线的吞吐量最大化，
该将两个寄存器插在哪里呢？吞吐量和延迟是多少？

流水线处理器 (CPU) 的设计：流水线基本原理

根据在哪里插入流水线寄存器，会出现不同的流水线深度 (有多少个阶段) 和最大吞吐量的组合。

假设每个流水线寄存器的延迟为 20ps



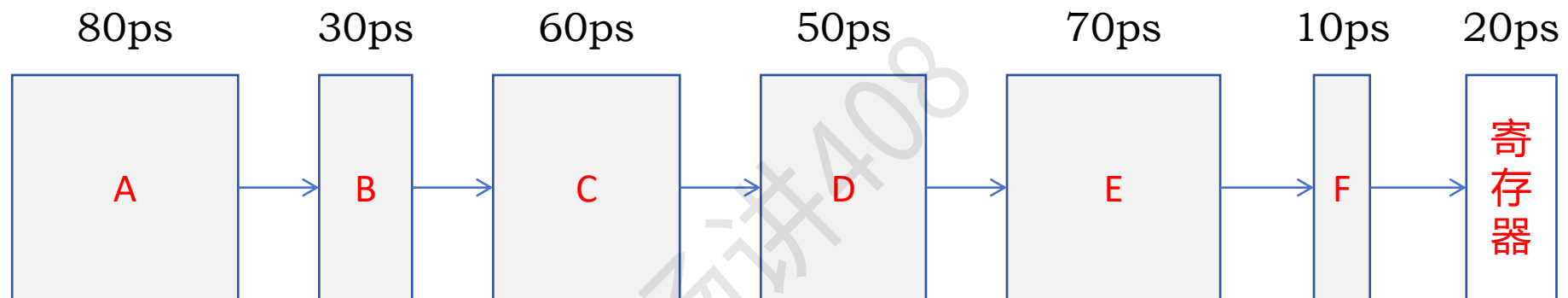
要使一个四阶段的流水线的吞吐量最大化

该将三个寄存器插在哪里呢？吞吐量和延迟是多少？

流水线处理器 (CPU) 的设计：流水线基本原理

根据在哪里插入流水线寄存器，会出现不同的流水线深度 (有多少个阶段) 和最大吞吐量的组合。

假设每个流水线寄存器的延迟为 20ps



要得到一个吞吐量最大的设计，至少要有几个阶段？

描述这个设计及其吞吐量和延迟

流水线处理器 (CPU) 的设计

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

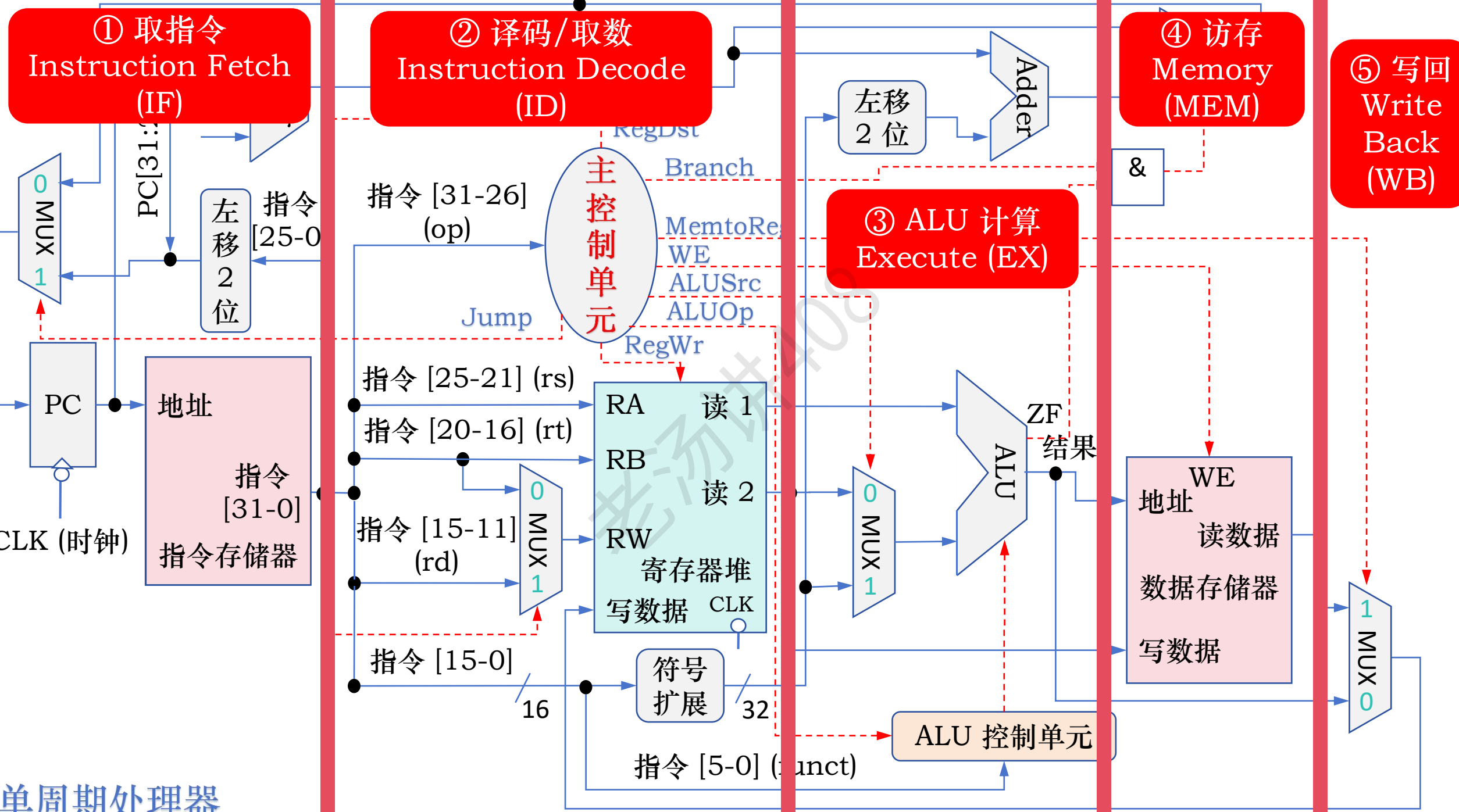


老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。



单周期处理器

流水线处理器 (CPU) 的设计

32 位 MIPS 指令系统

R- 型指令:

add rd, rs, rt

sub rd, rs, rt

or rd, rs, rt

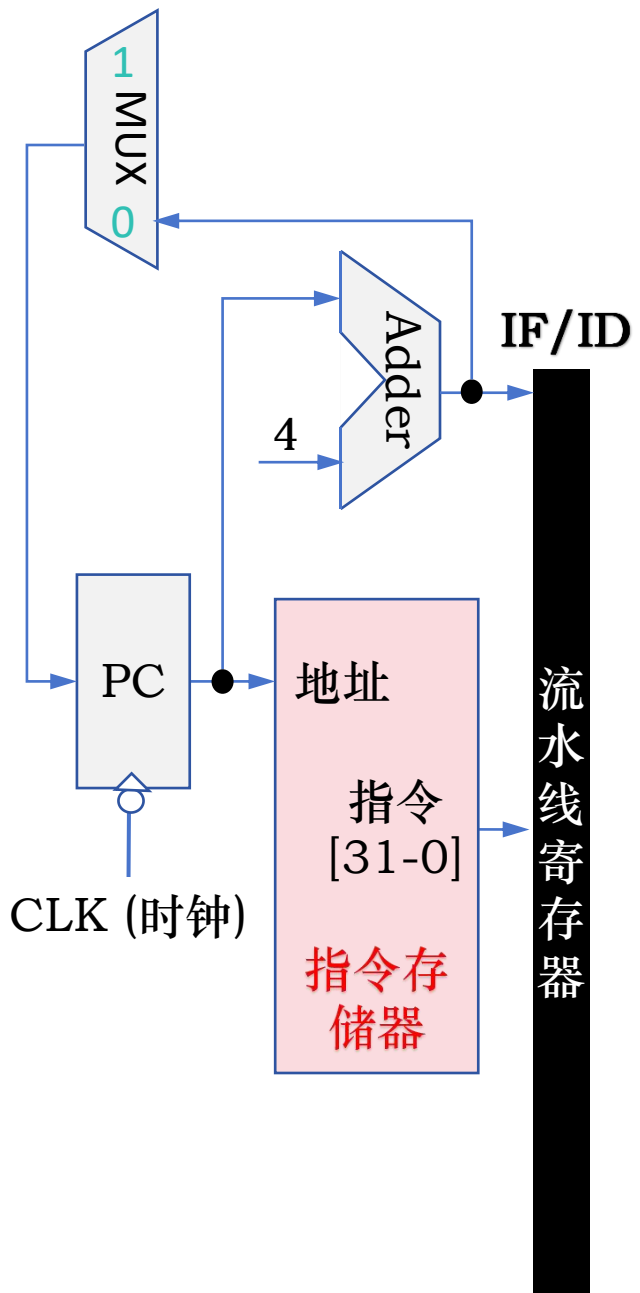
slt rd, rs, rt

I-型指令:

lw rt, imm16(rs)

sw rt, imm16(rs)

beq rs, rt, imm16

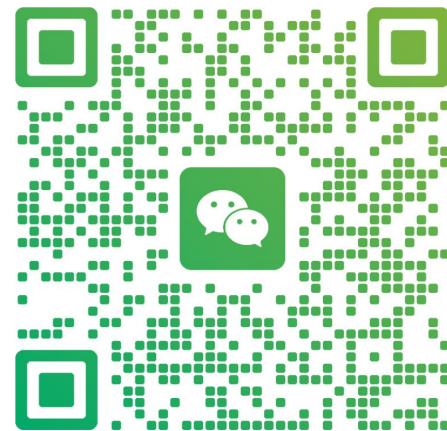


加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



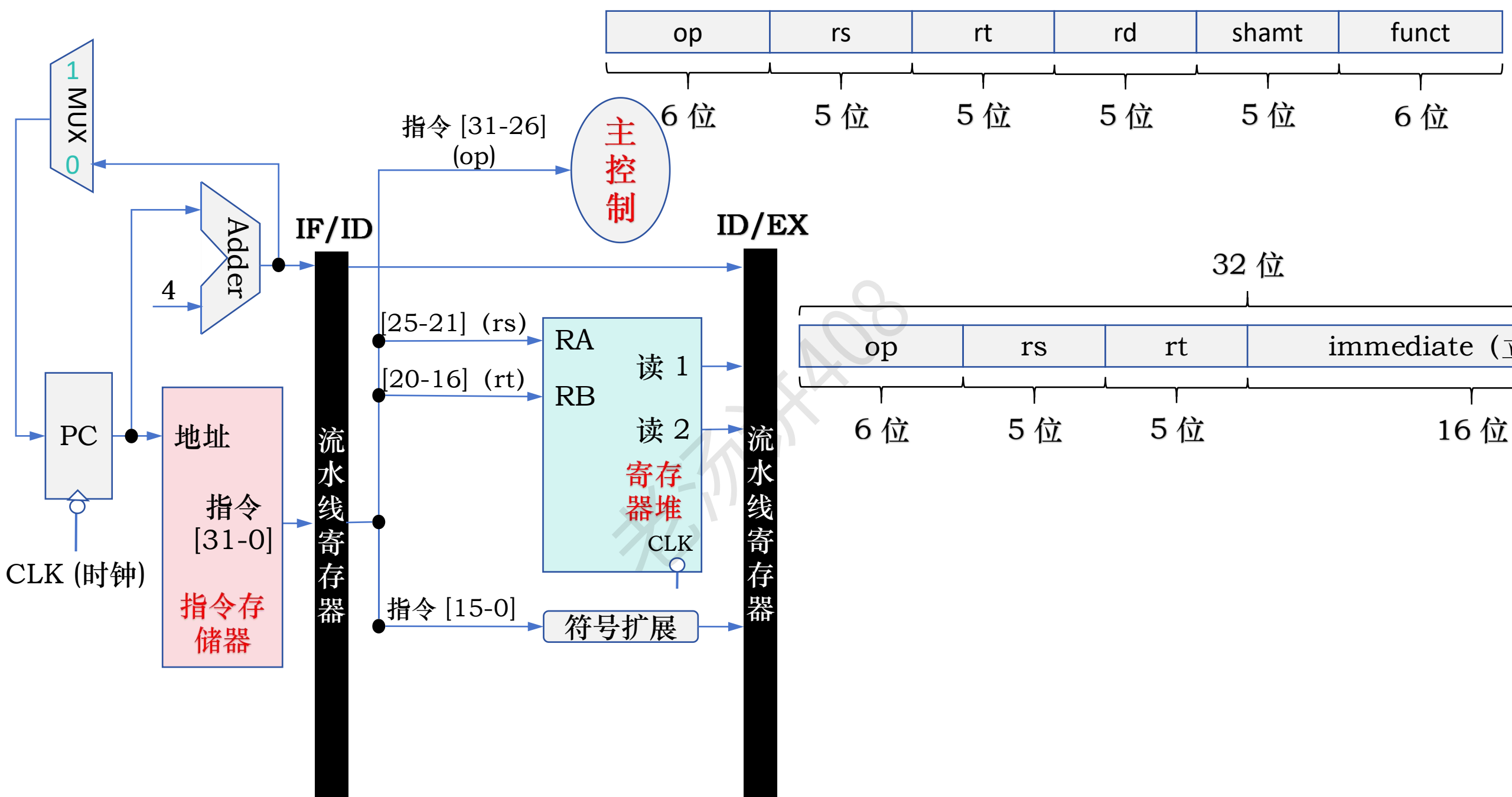
老汤

浙江 杭州

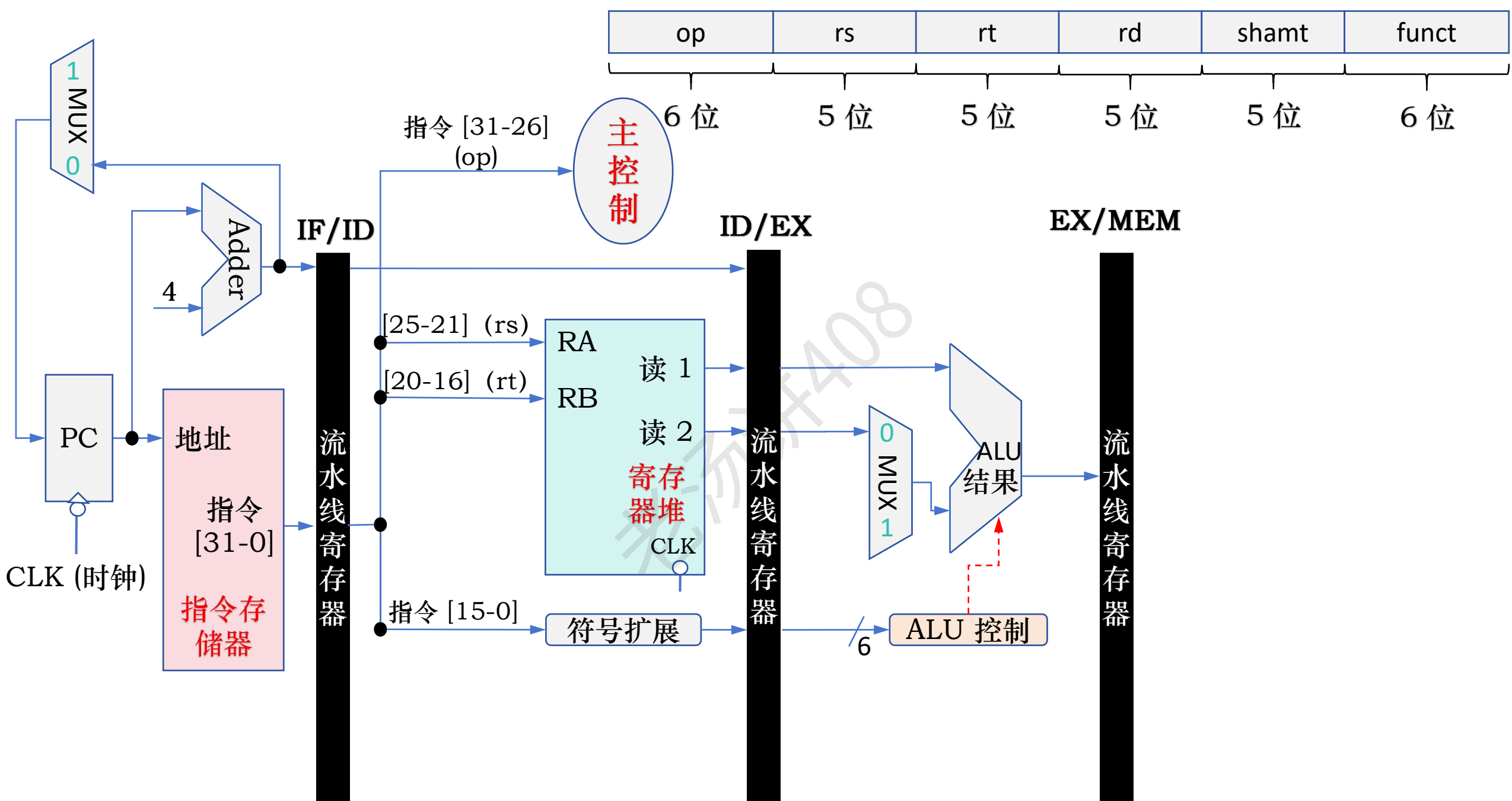


扫一扫上面的二维码图案，加我为朋友。

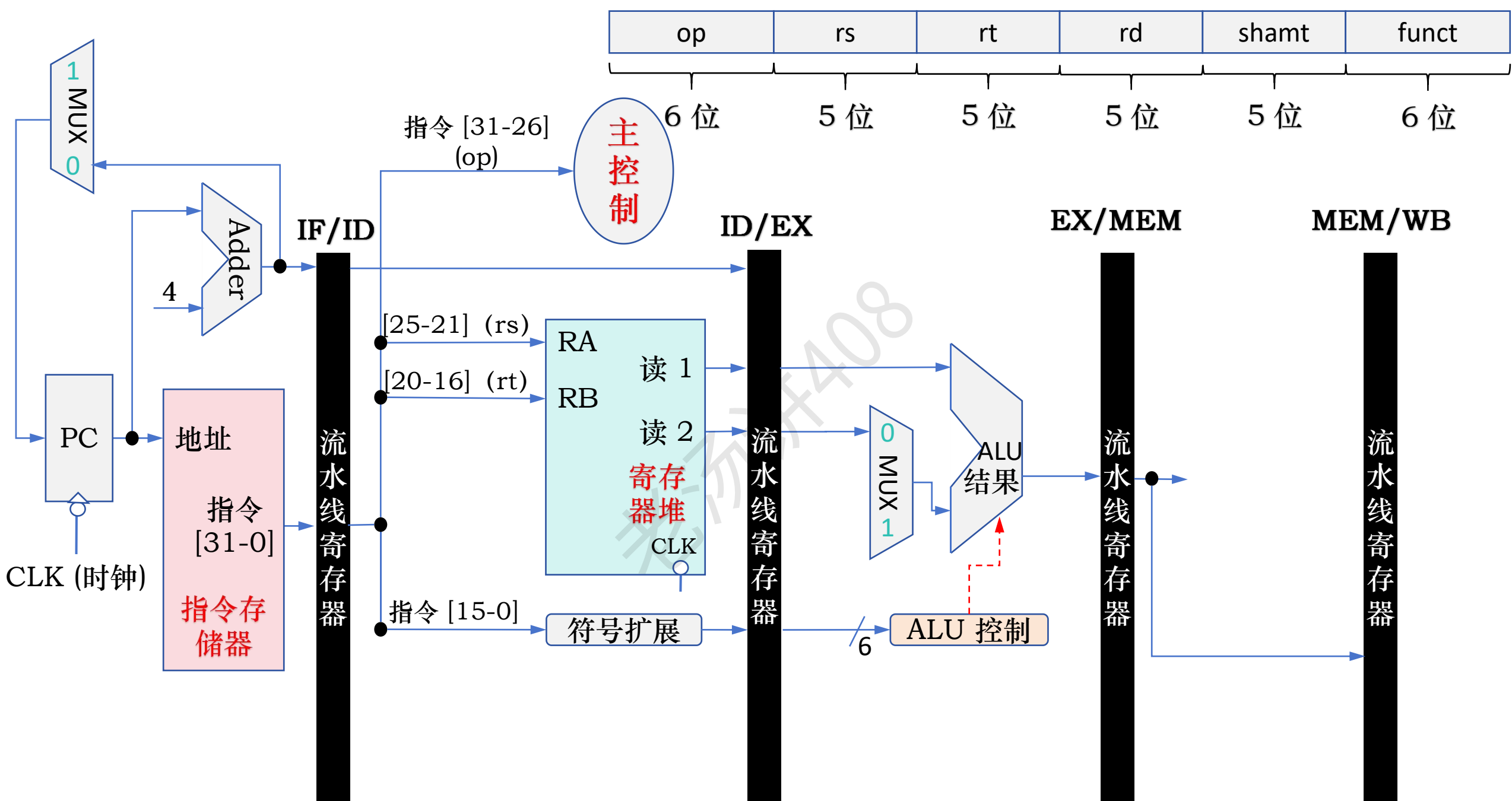
取指令流水段 (IF)



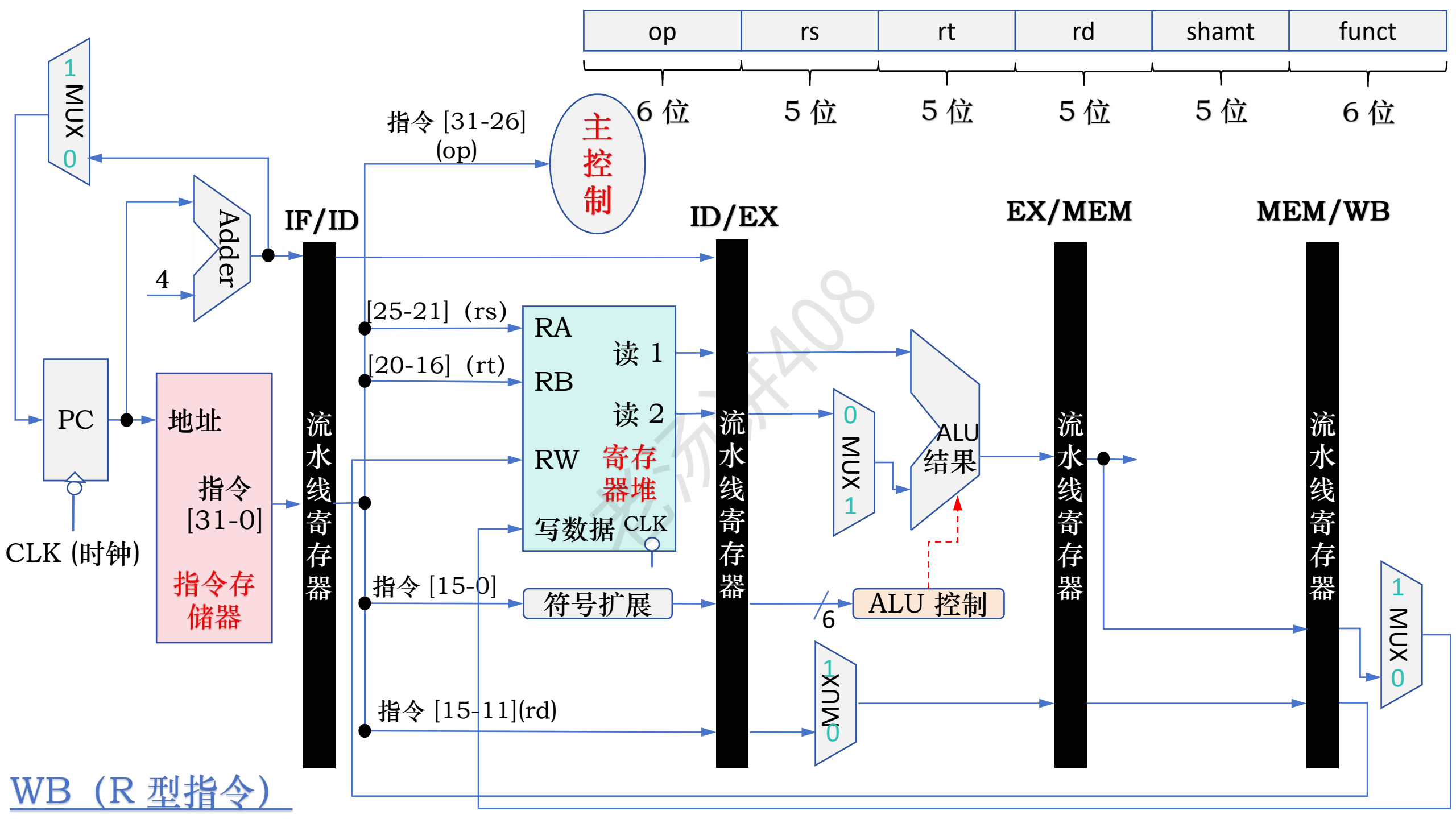
译码/取数流水段 (ID)

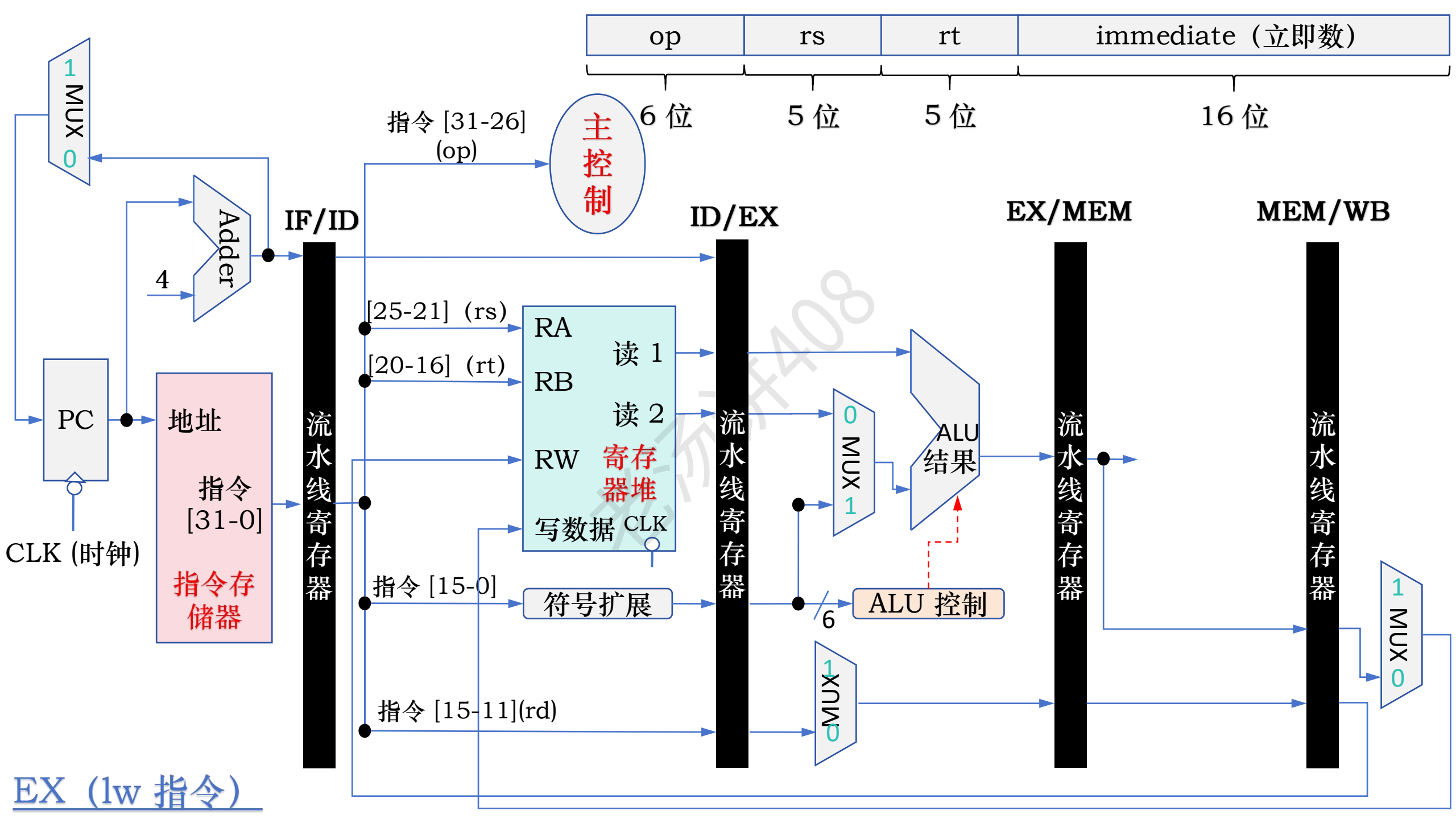


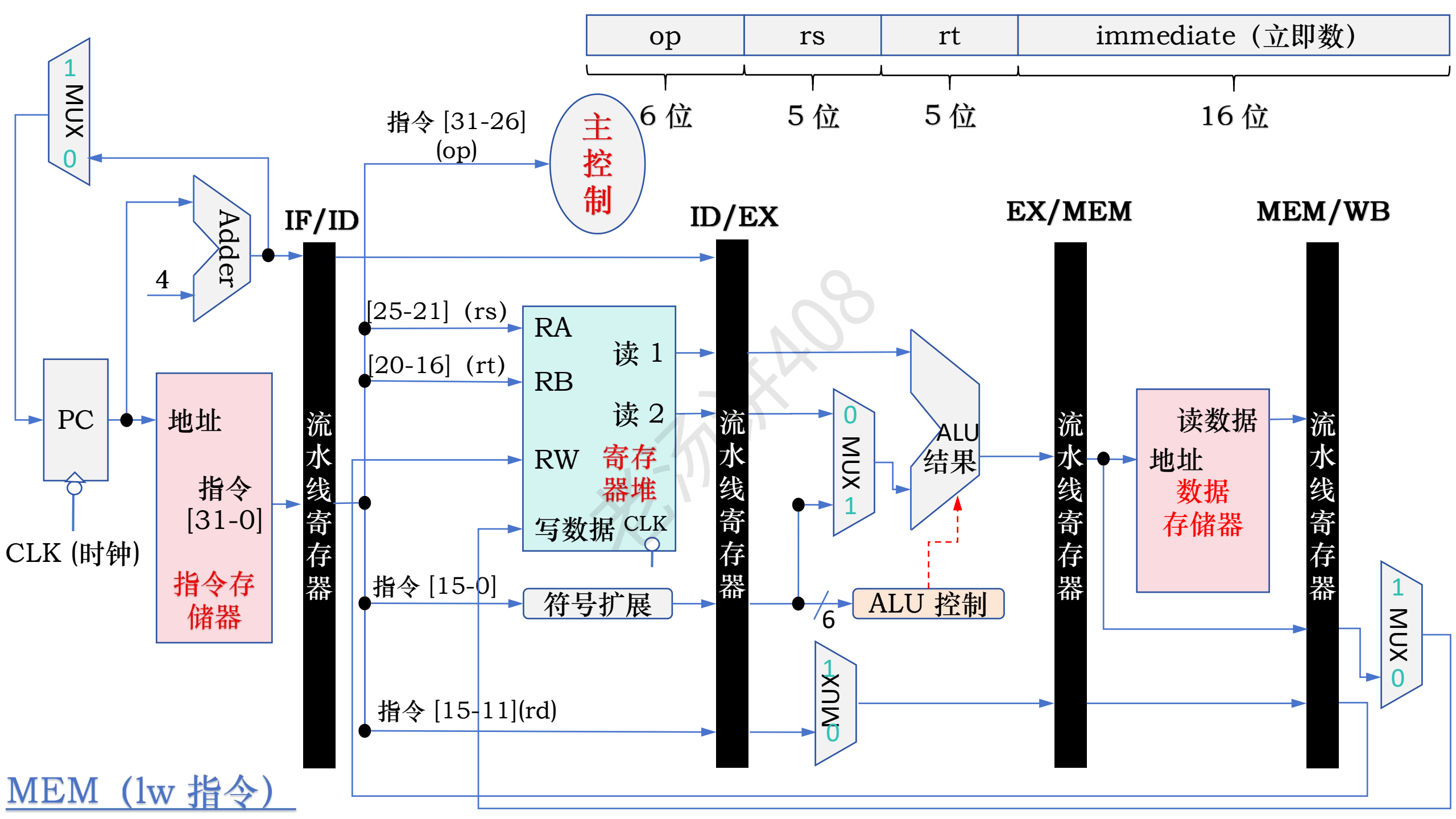
EX 流水段 (R 型指令)

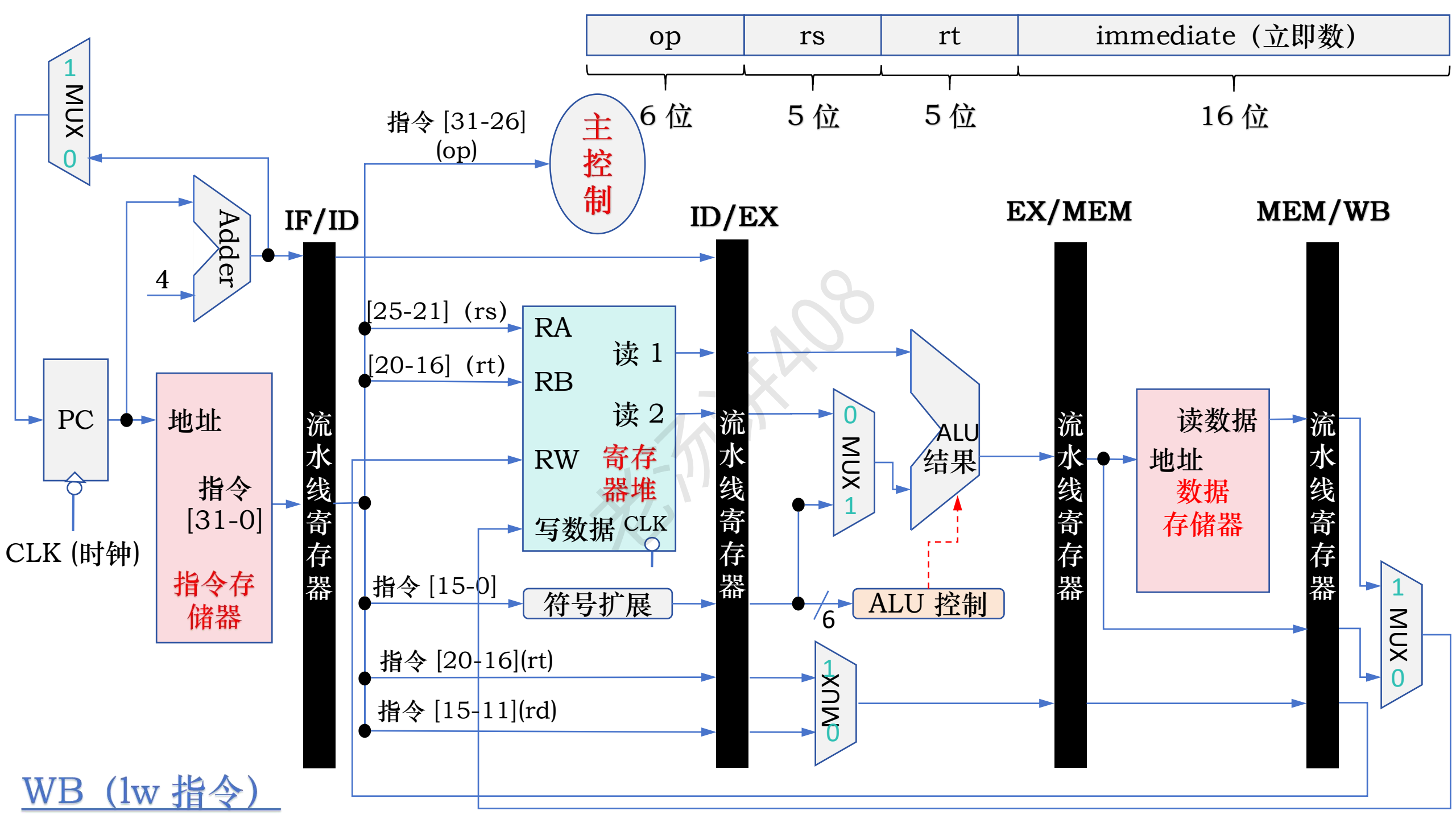


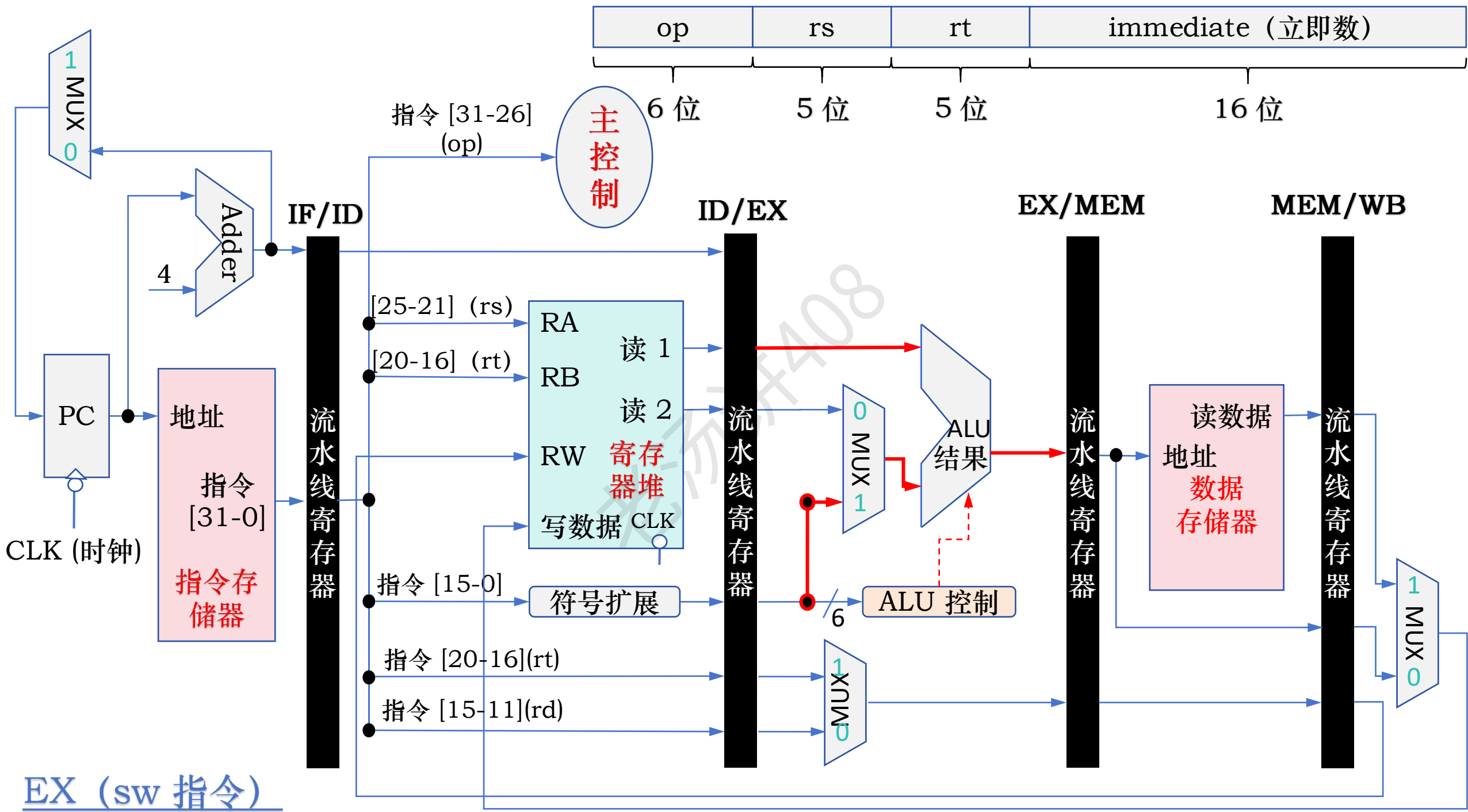
MEM 流水段 (R 型指令)

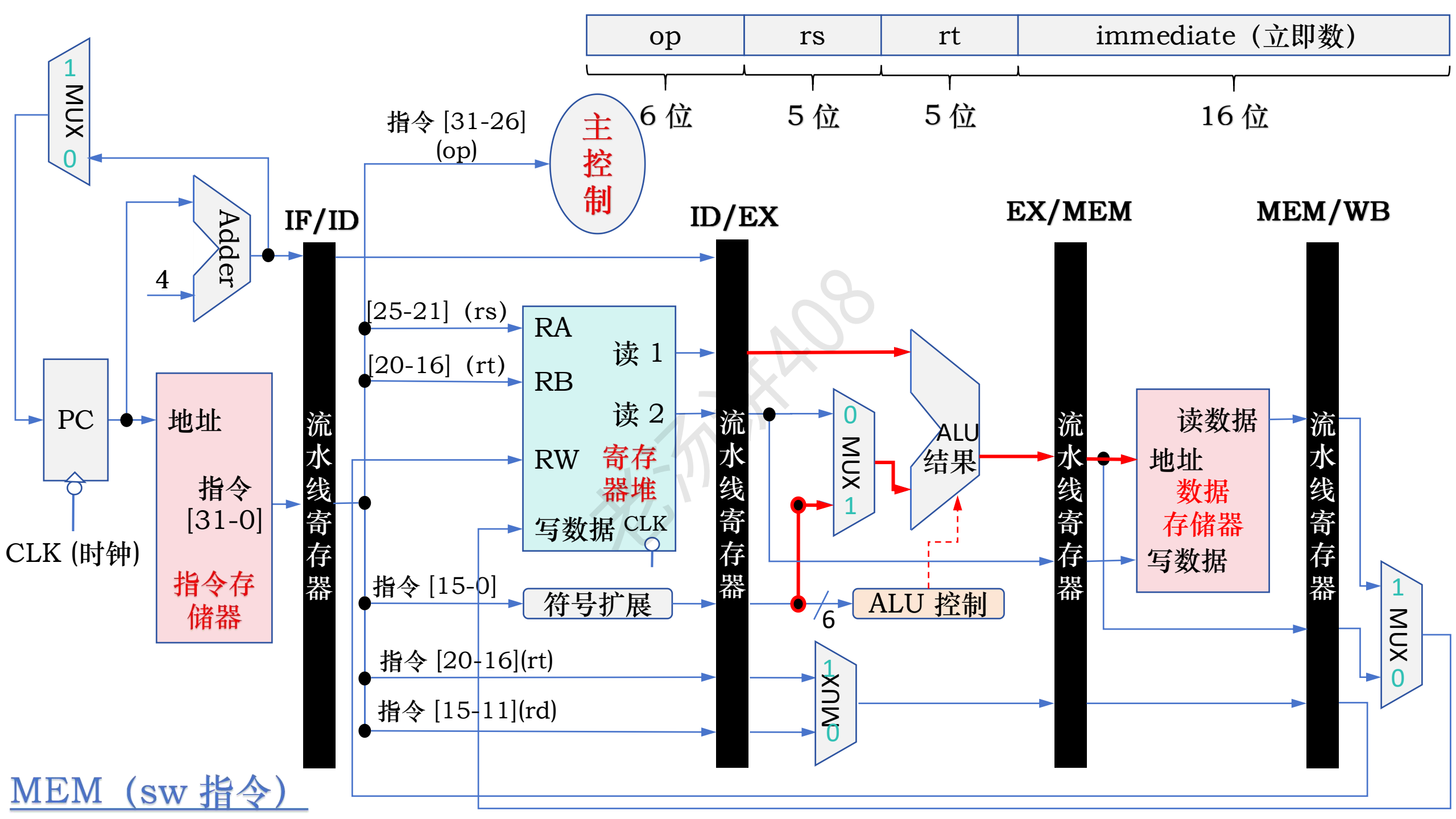


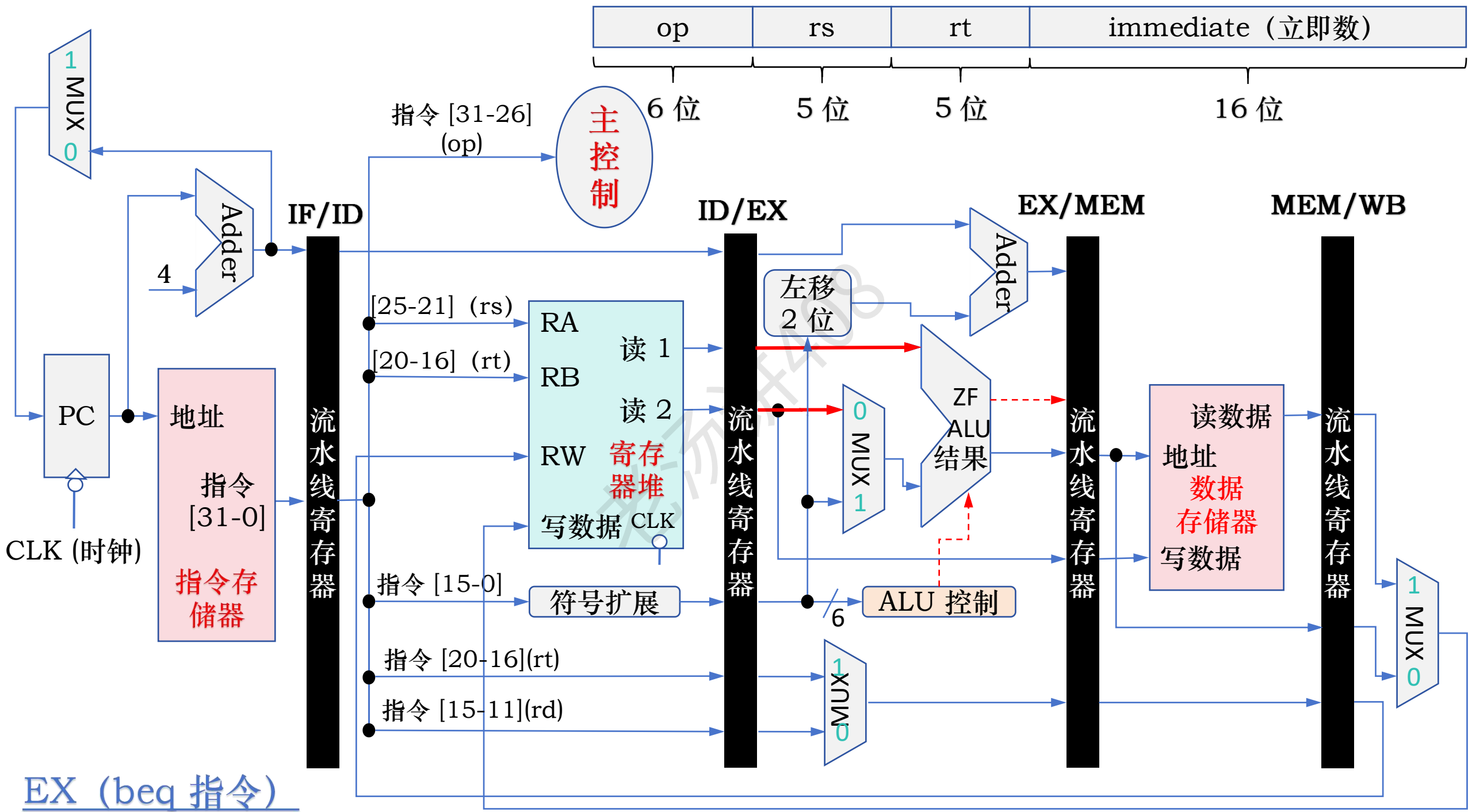


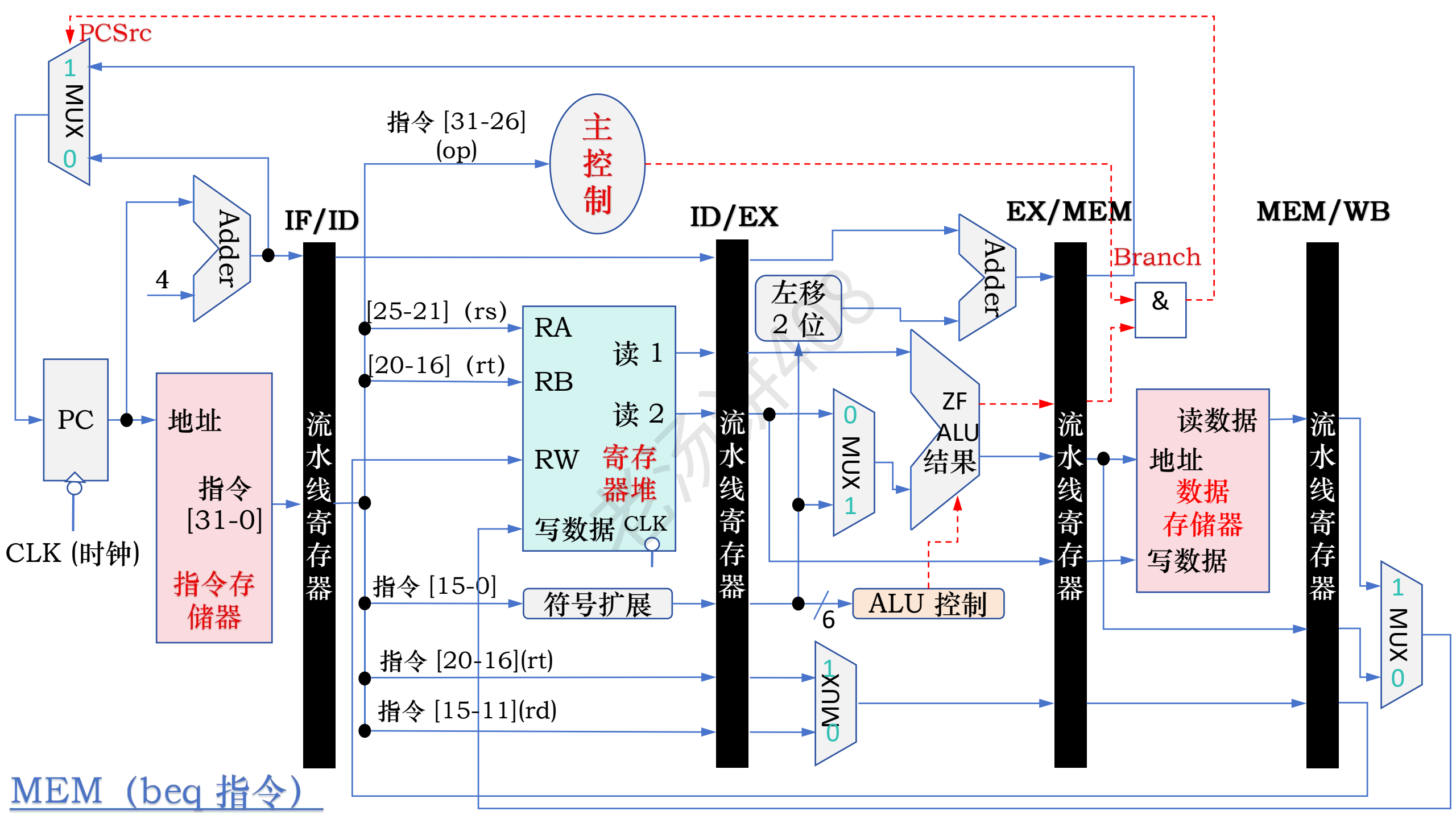


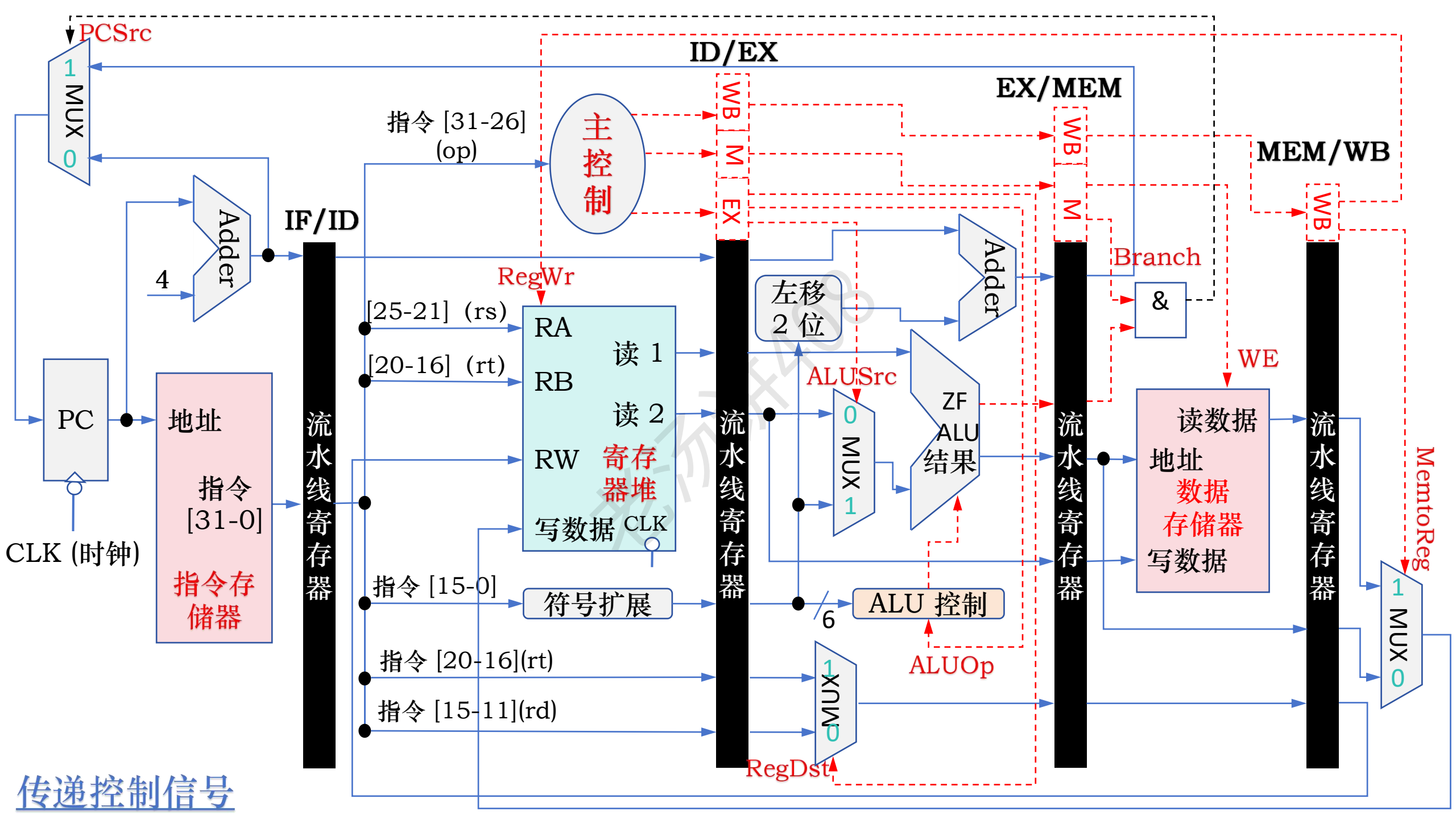












流水线处理器 (CPU) 的设计

在一个时钟周期内同时会有多条指令在一个数据通路中执行

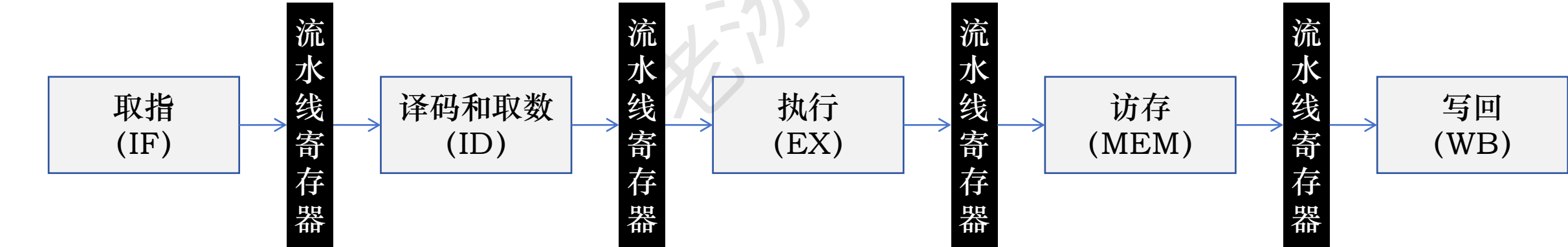
lw \$10, 20(\$1)

sub \$11, \$2, \$3

add \$12, \$3, \$4

lw \$13, 24(\$1)

add \$14, \$5, \$6



时钟信号

流水线处理器 (CPU) 的设计

在一个时钟周期内同时会有多条指令在一个数据通路中执行

① 多时钟周期的流水线图

② 单时钟周期的流水线图

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州

lw \$10, 20(\$1)

sub \$11, \$2, \$3

add \$12, \$3, \$4

lw \$13, 24(\$1)

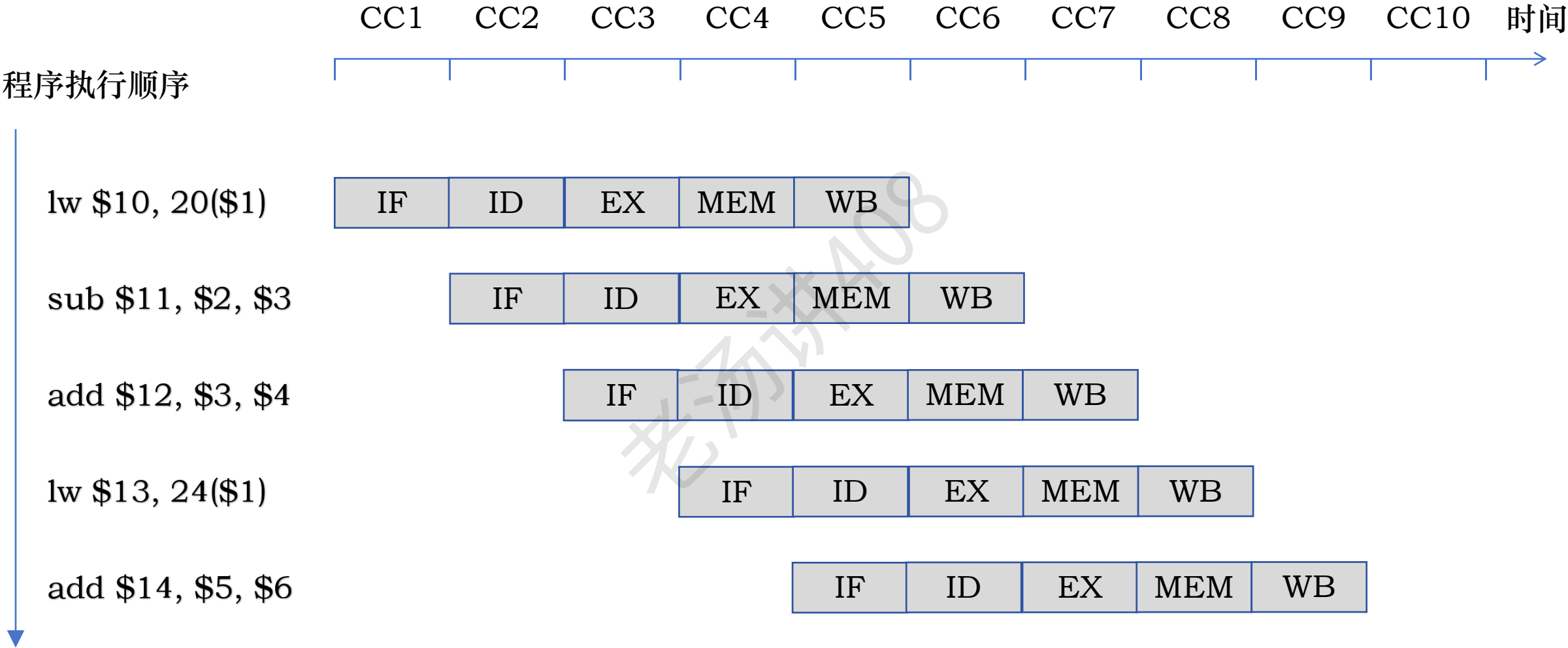
add \$14, \$5, \$6



扫一扫上面的二维码图案，加我为朋友。

流水线处理器 (CPU) 的设计: 多时钟周期的流水线图

时钟周期: Clock Cycle, 简写 CC



流水线处理器（CPU）的设计：单时钟周期的流水线图

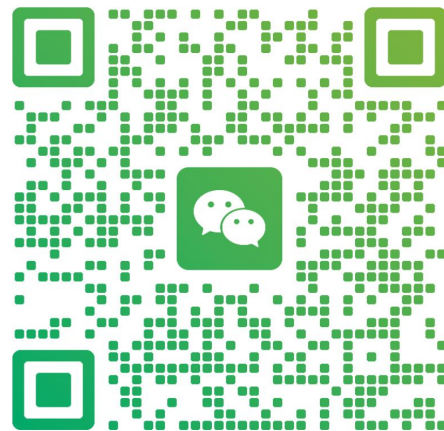
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

在一个时钟周期内整个数据通路的状态



老汤

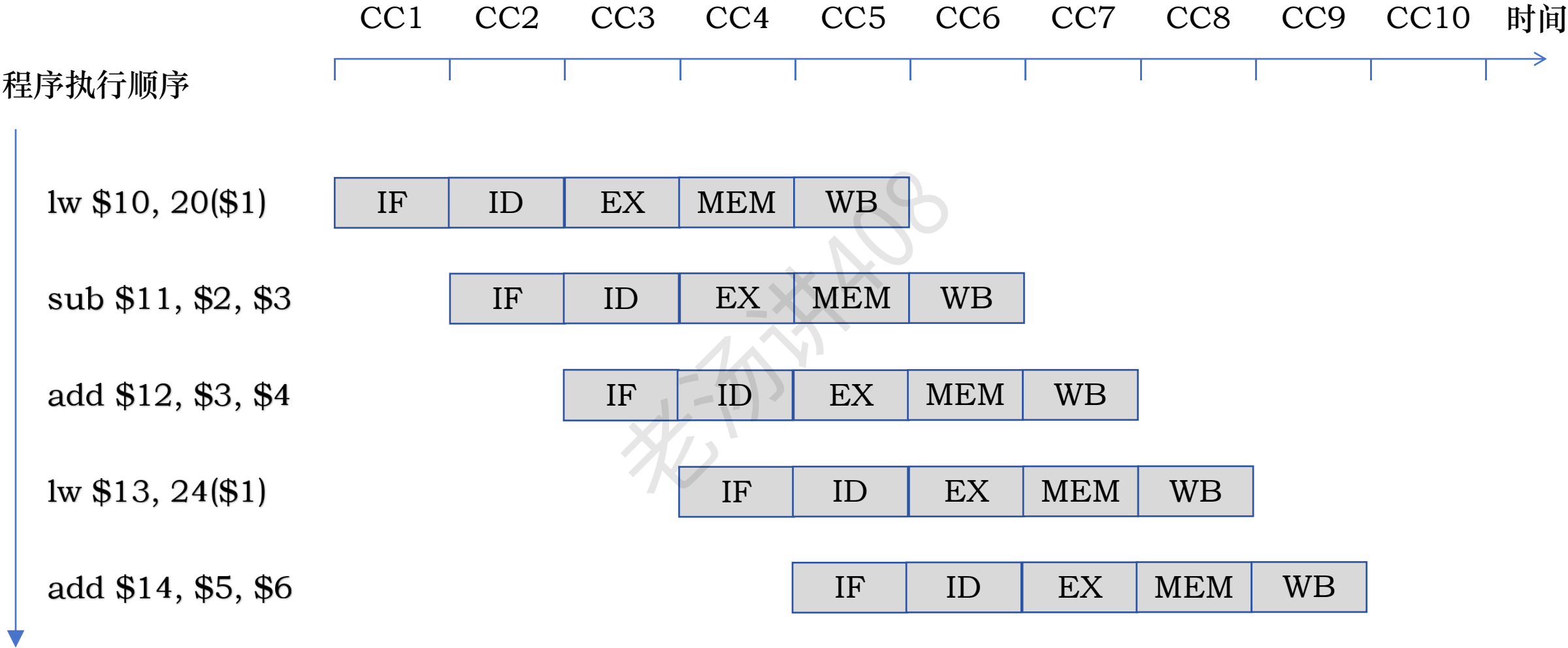
浙江 杭州

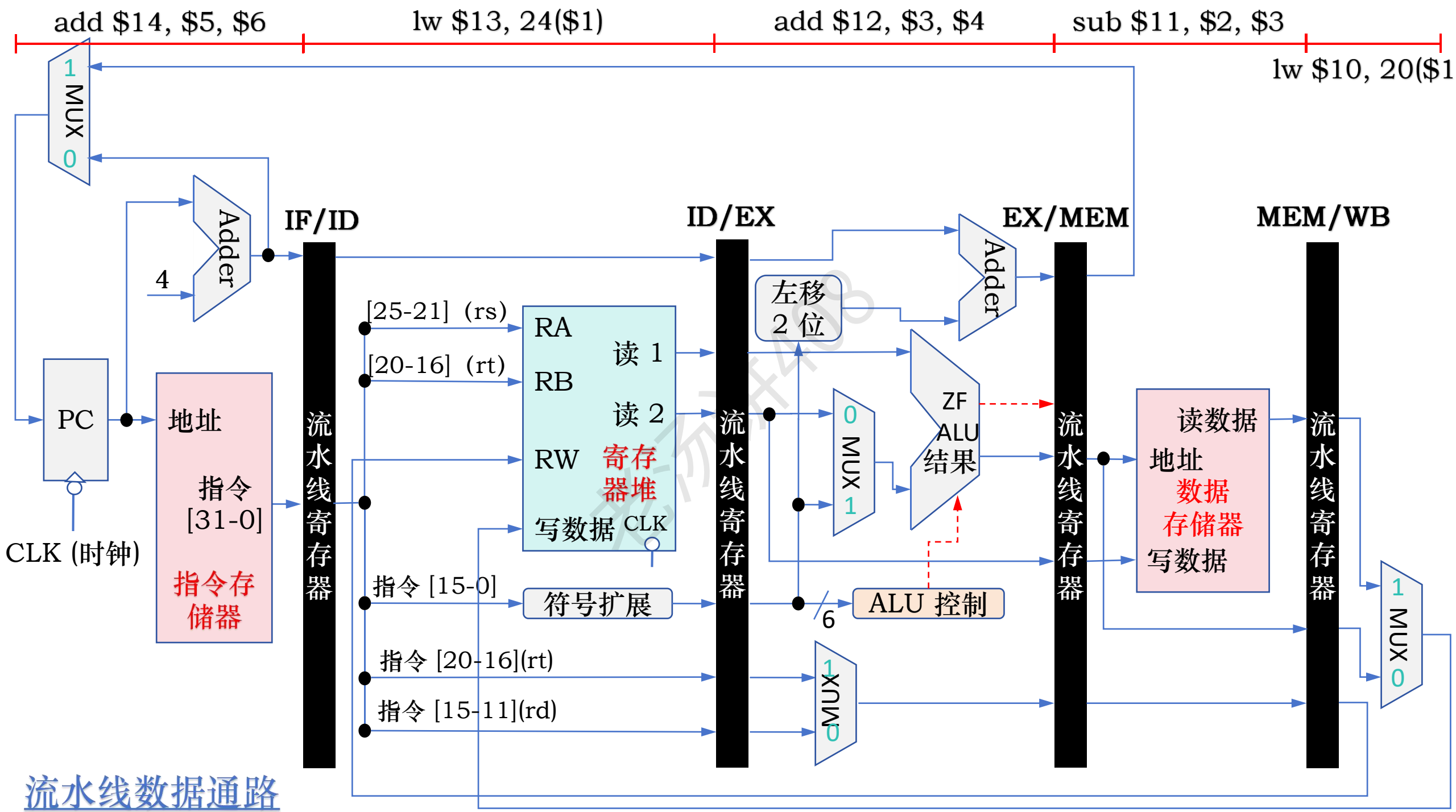


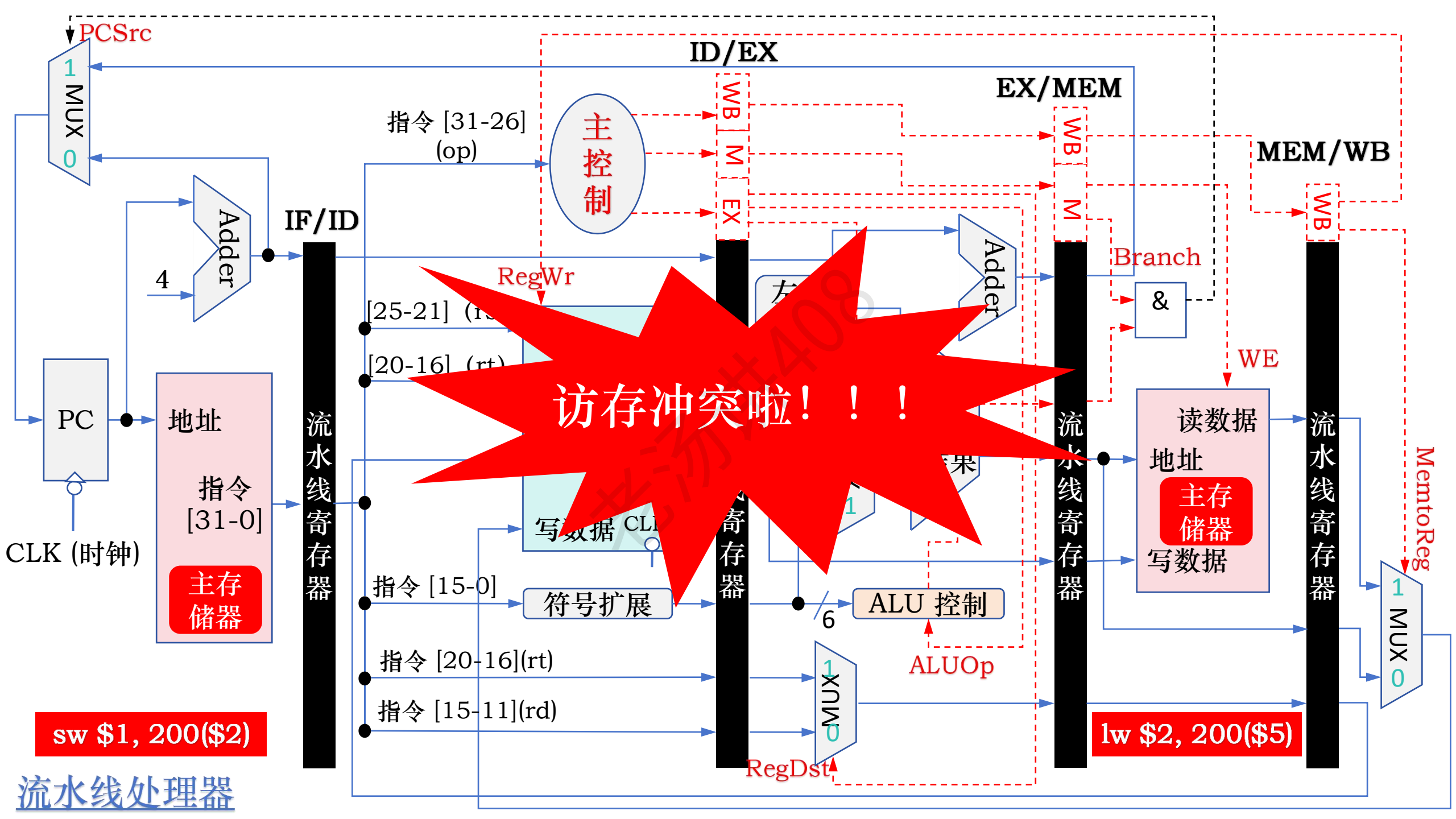
扫一扫上面的二维码图案，加我为朋友。

流水线处理器 (CPU) 的设计: 多时钟周期的流水线图

时钟周期: Clock Cycle, 简写 CC



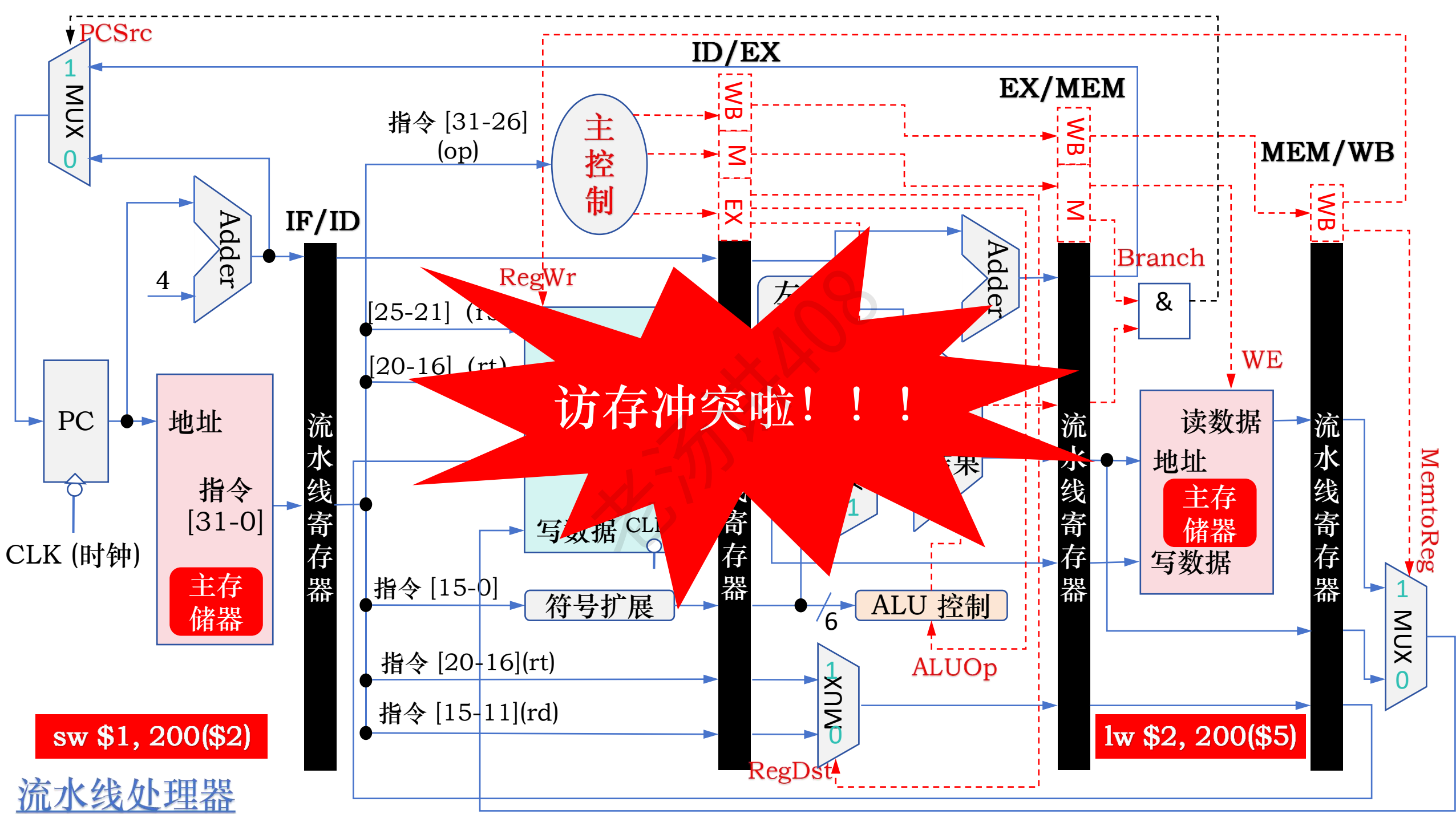




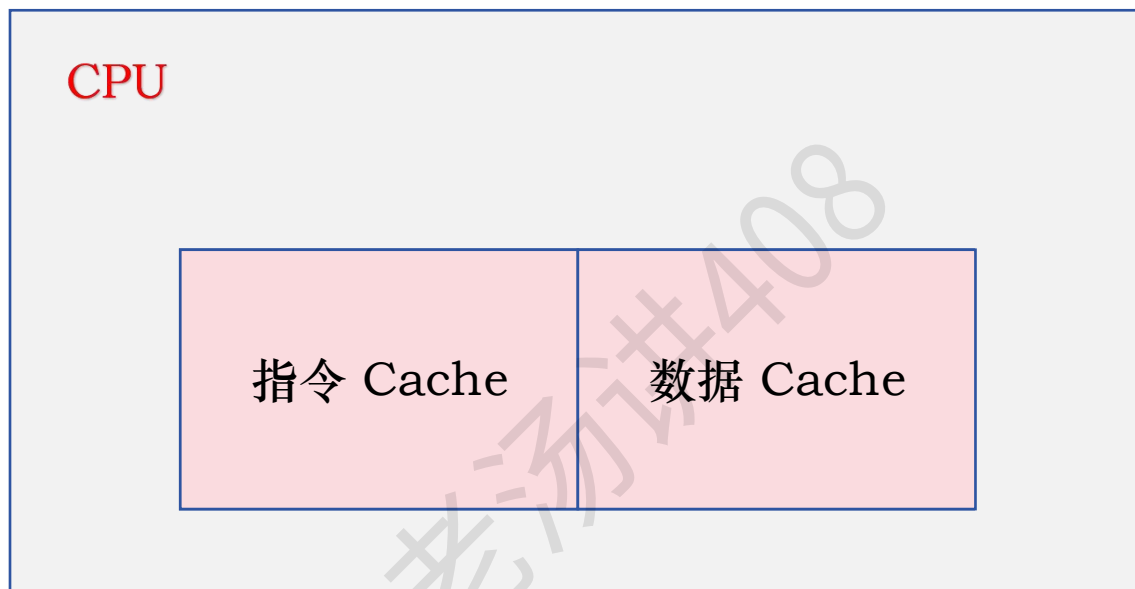
流水线处理器 (CPU) 的设计

在同一个时钟周期内，不同的指令需要用到同一个硬件，而且这个硬件还不支持共享

结构冒险



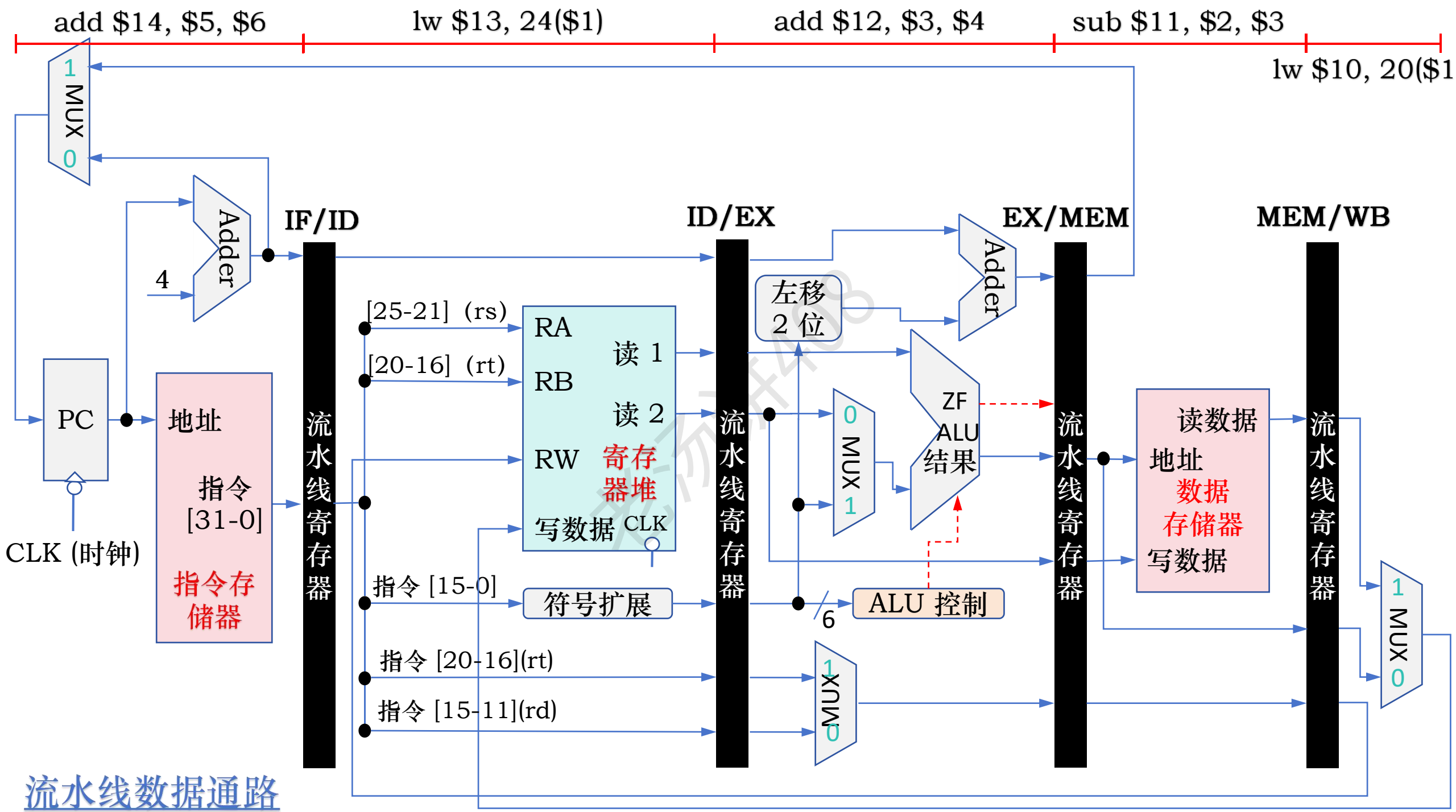
流水线处理器 (CPU) 的设计



设置多个独立的部件来避免硬件资源冲突

流水线处理器 (CPU) 的设计

一个部件每条指令只能使用一次，且只能在特定时钟周期使用



流水线处理器 (CPU) 的设计：流水线冒险

① 结构冒险

② 数据冒险

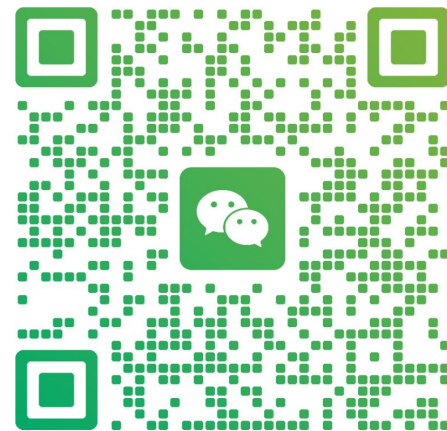
③ 控制冒险

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



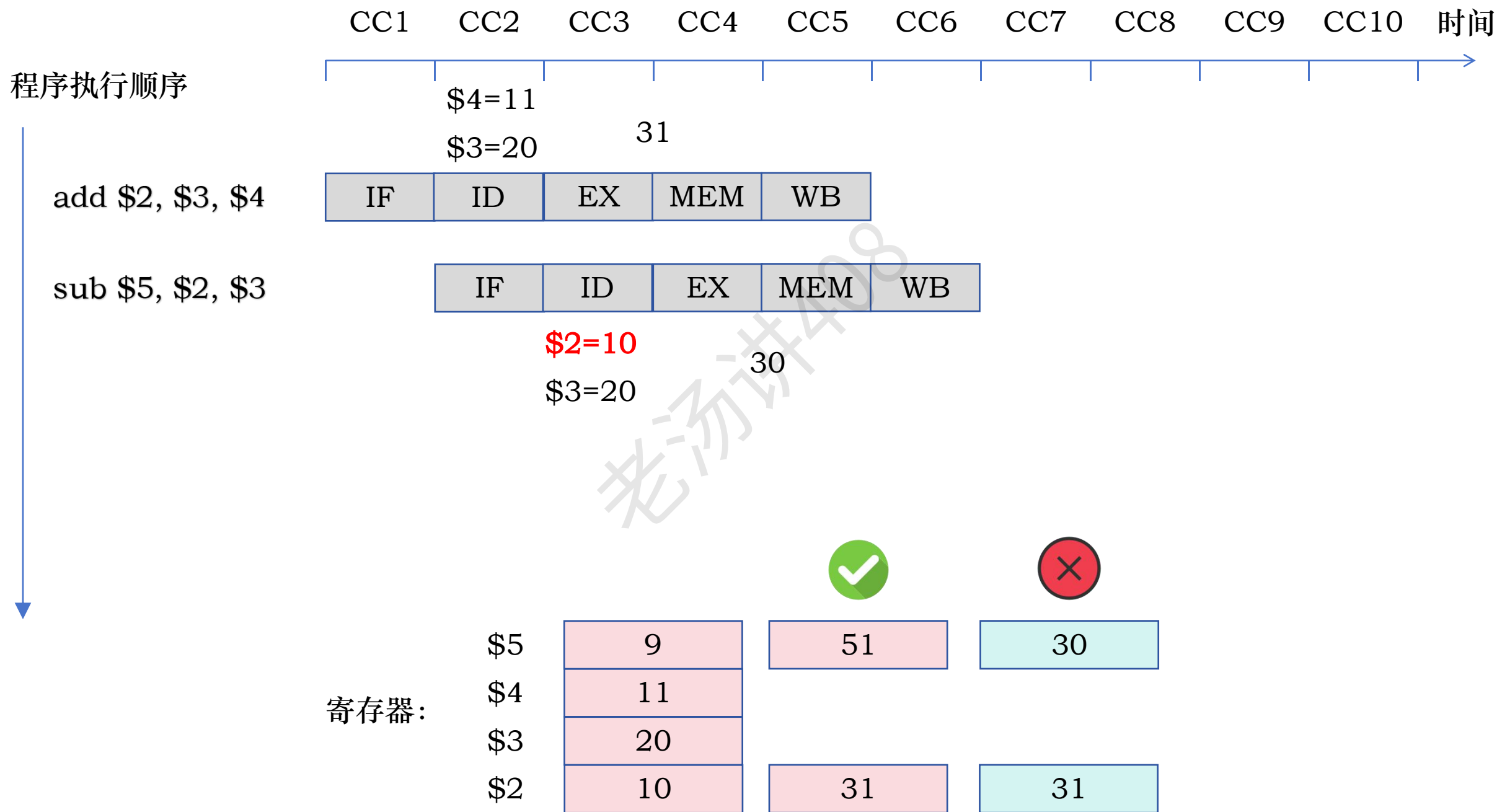
老汤

浙江 杭州

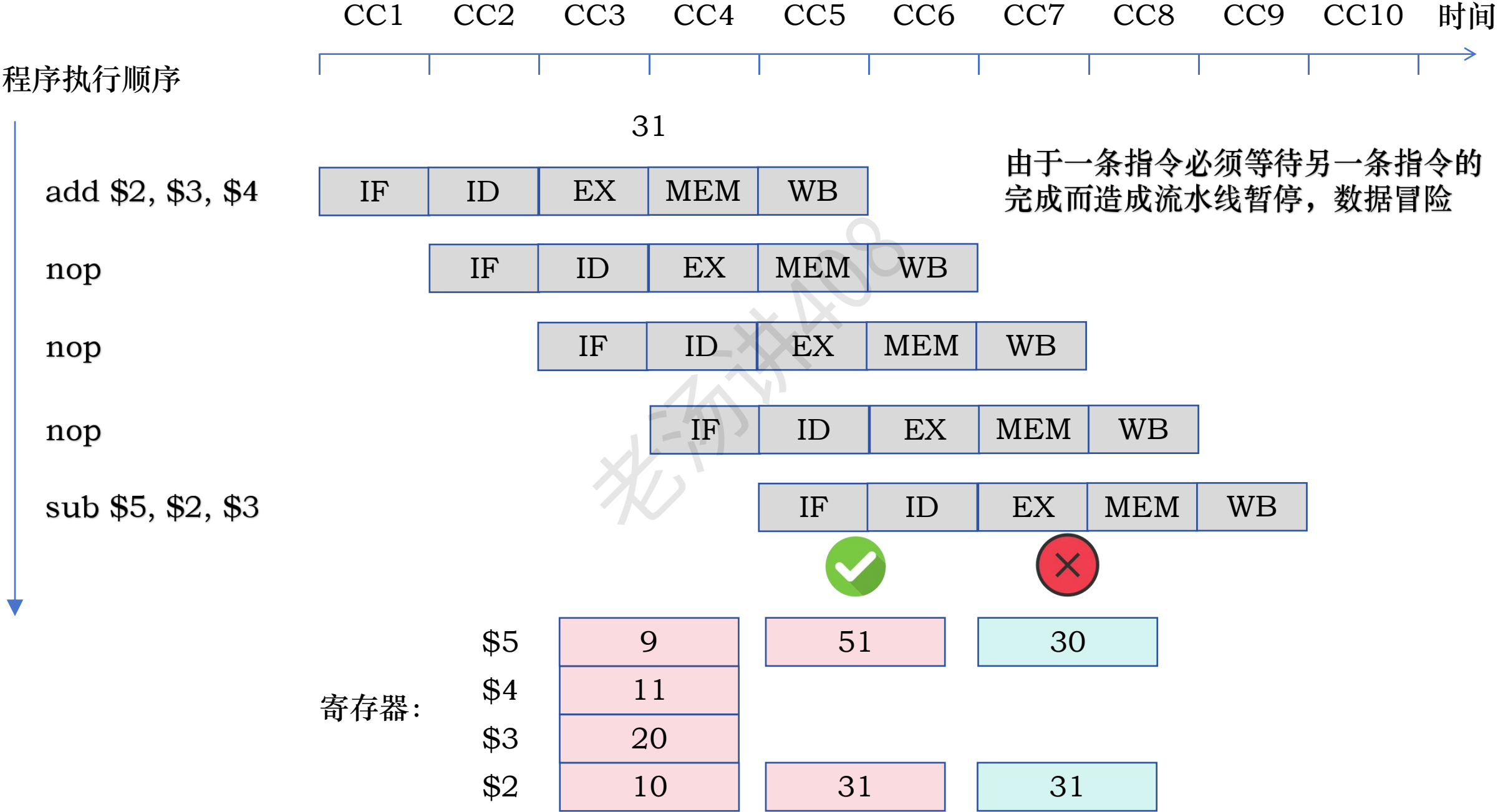


扫一扫上面的二维码图案，加我为朋友。

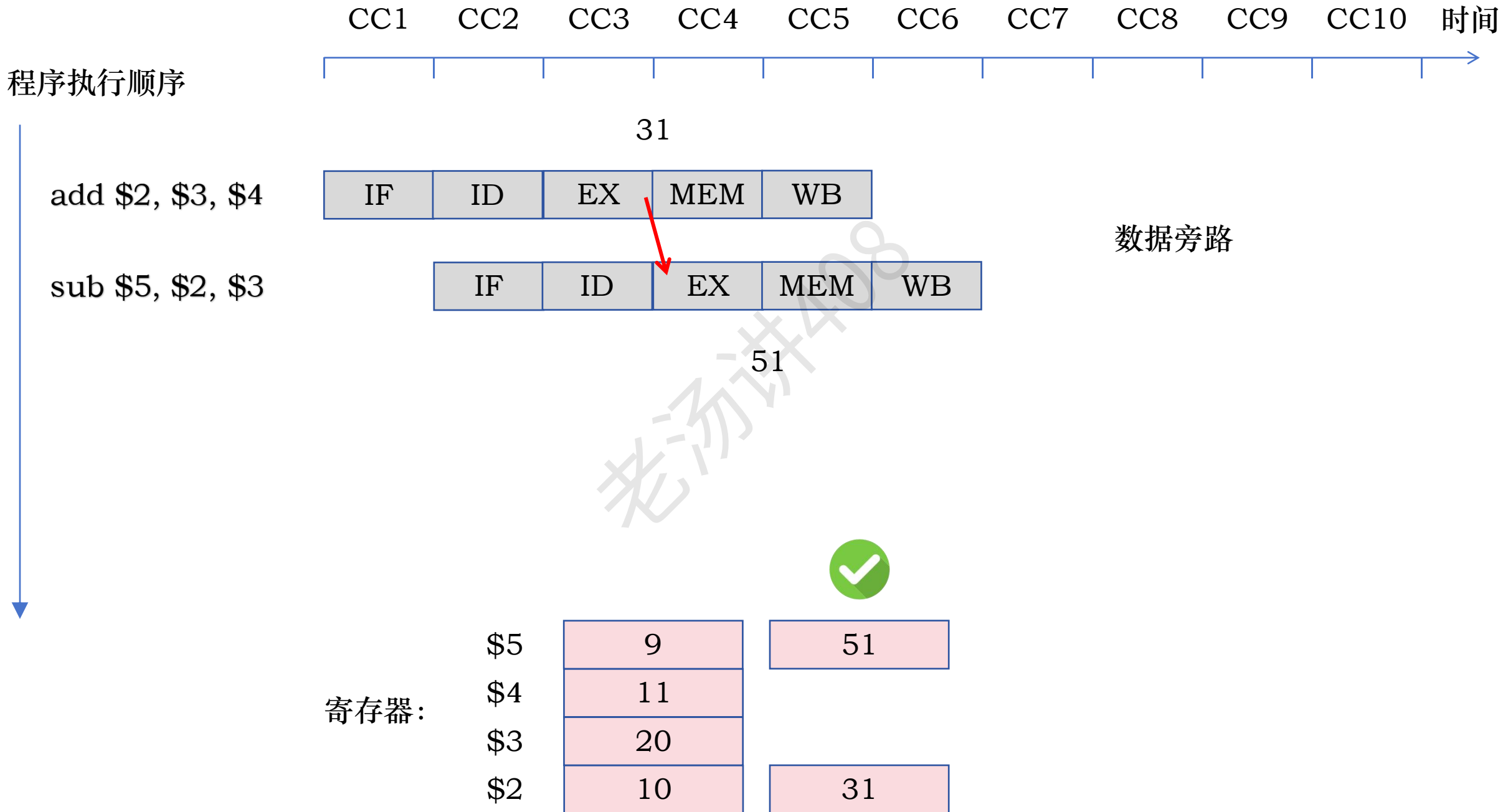
流水线处理器 (CPU) 的设计: 数据冒险

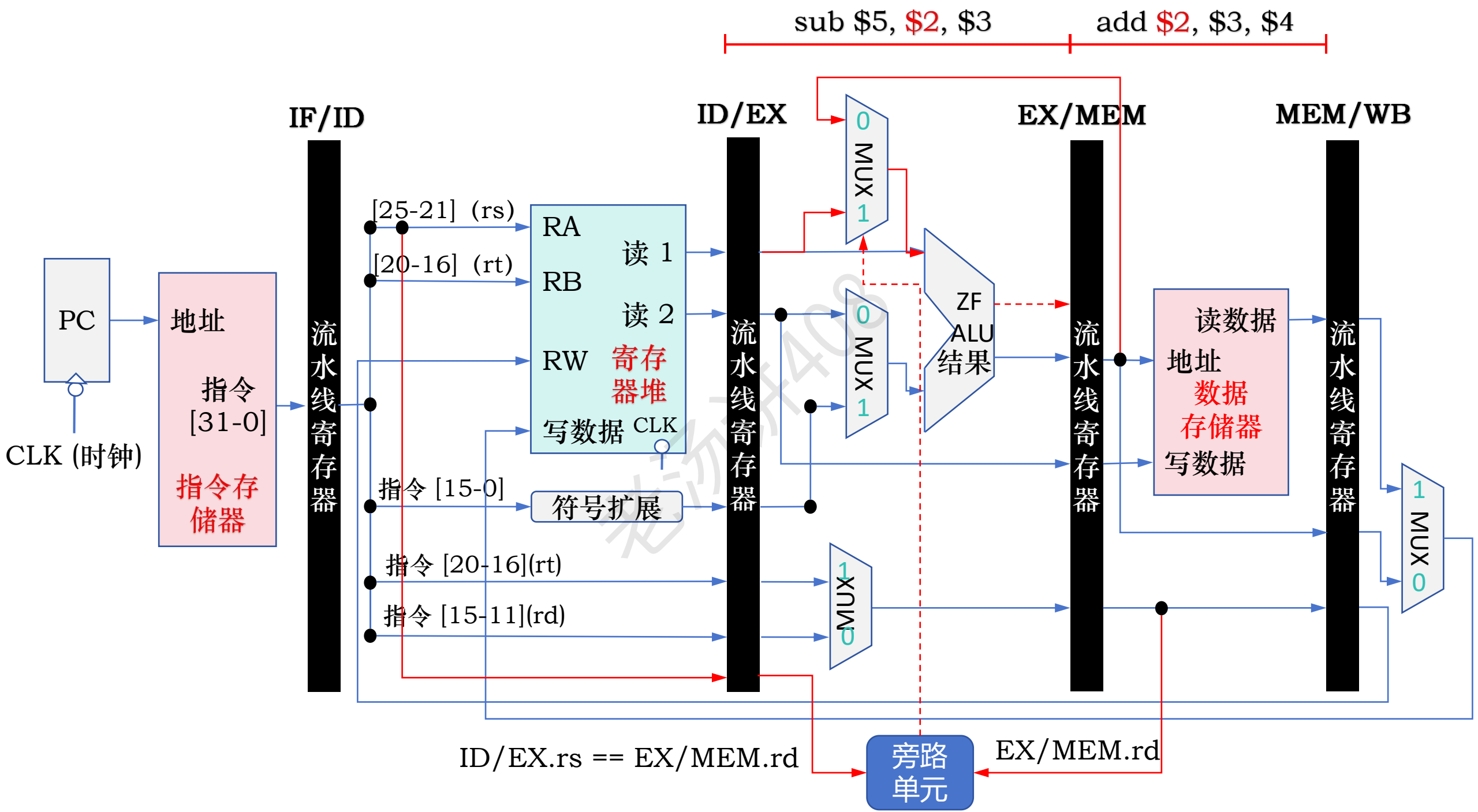


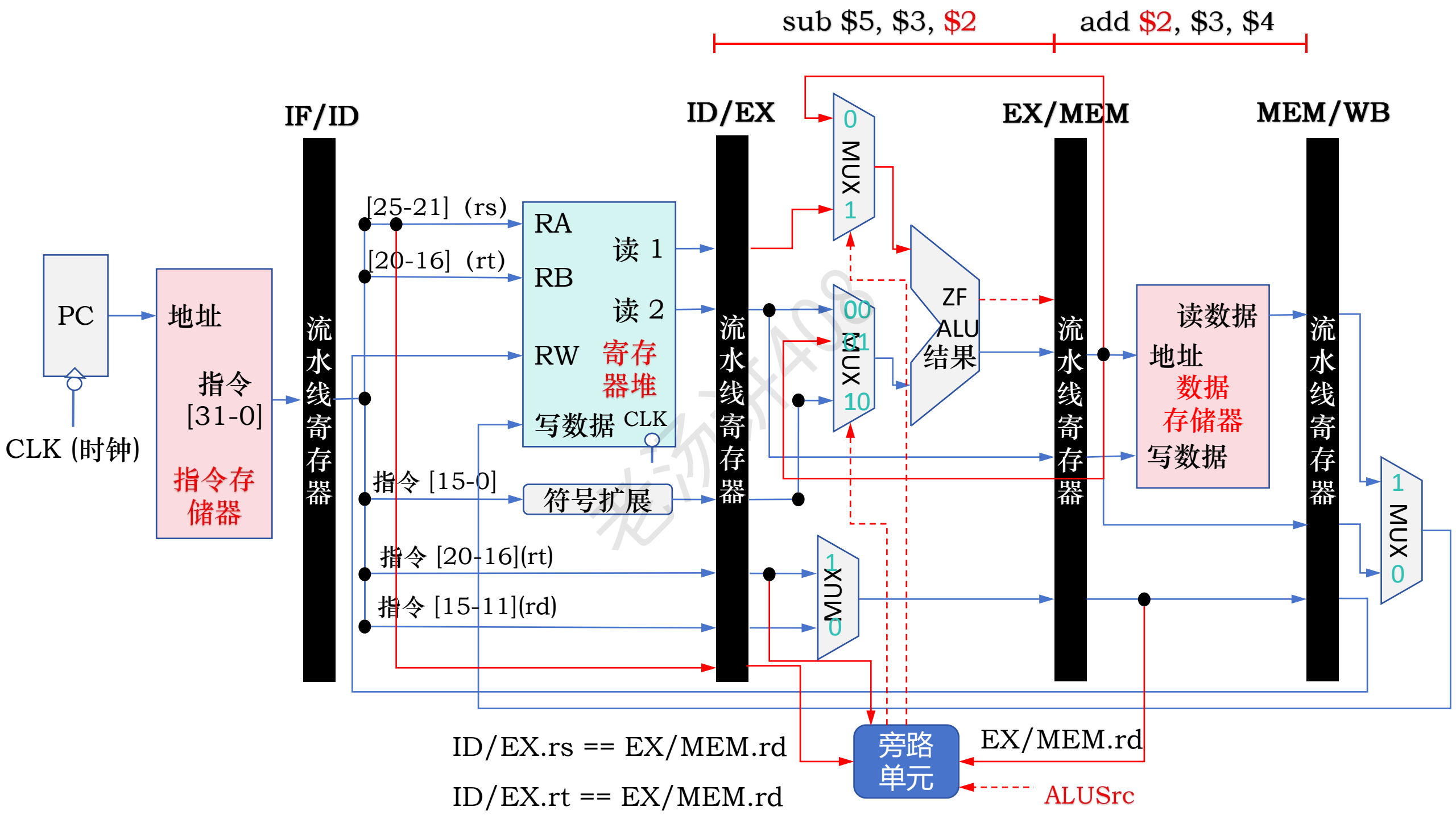
流水线处理器 (CPU) 的设计：数据冒险



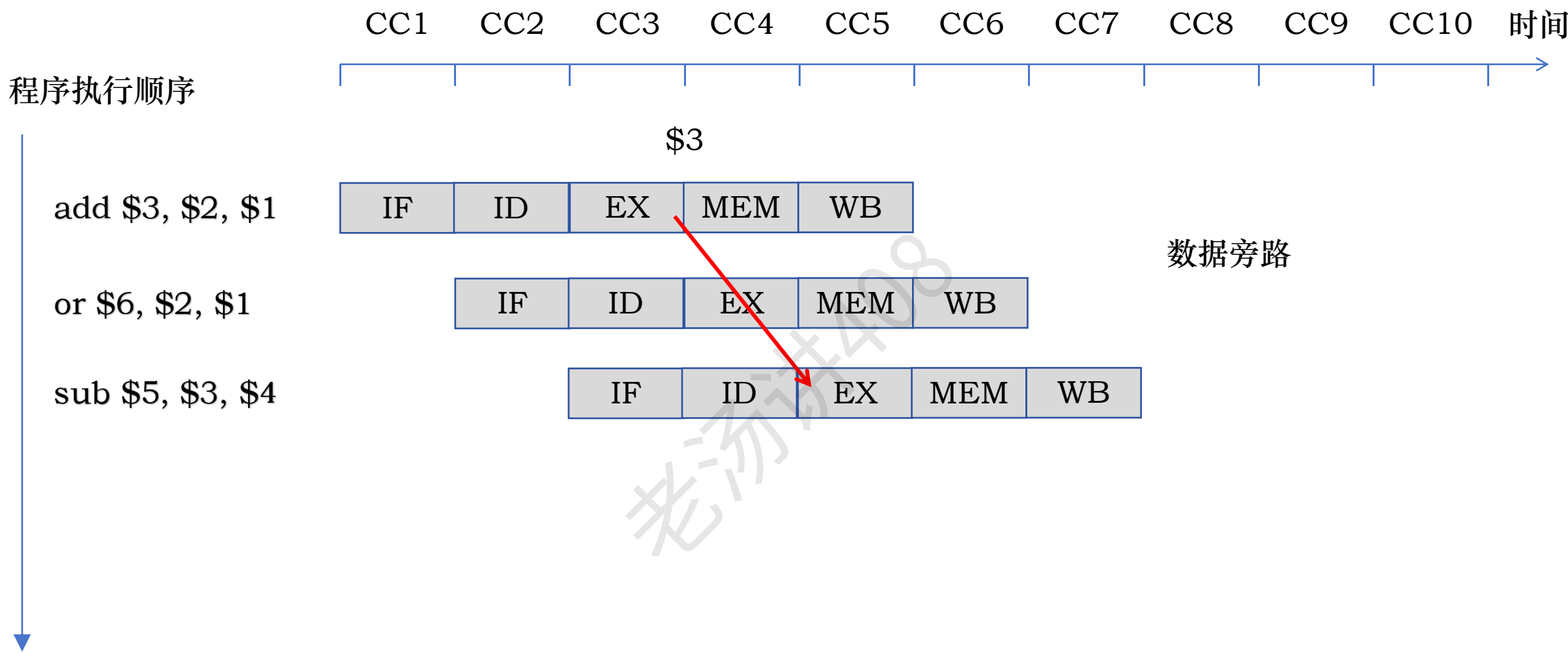
流水线处理器：数据冒险（数据转发技术）

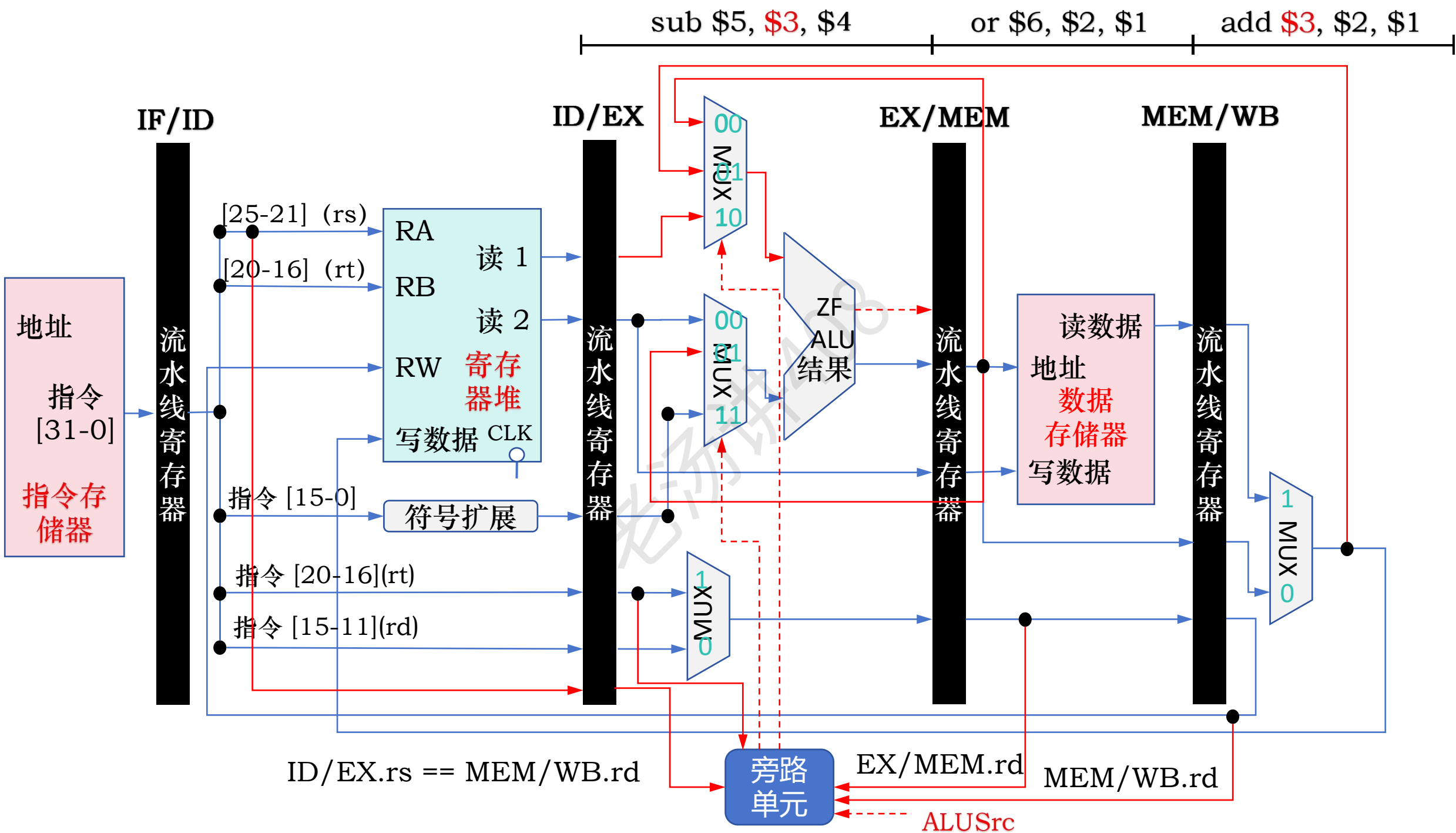


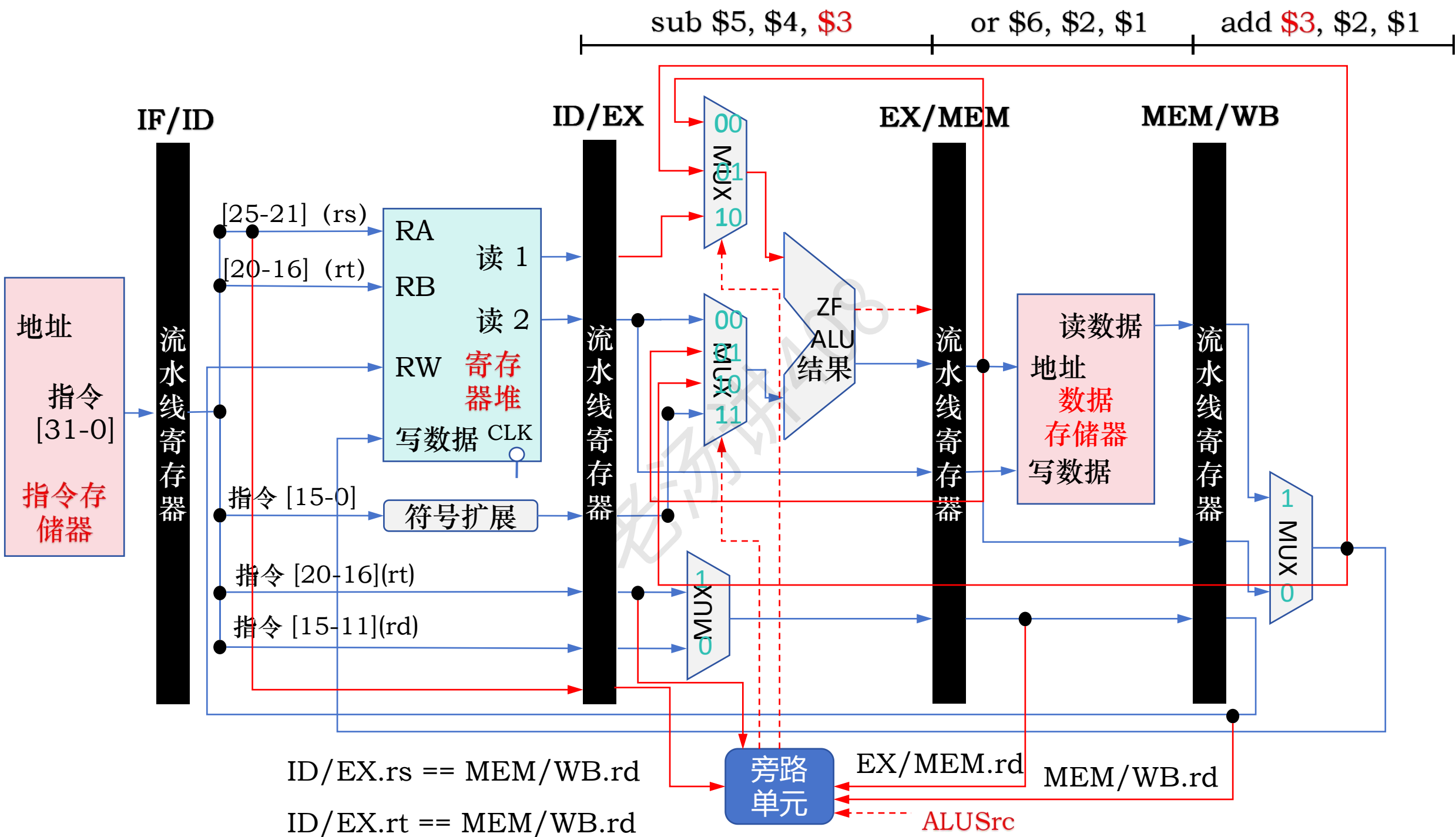




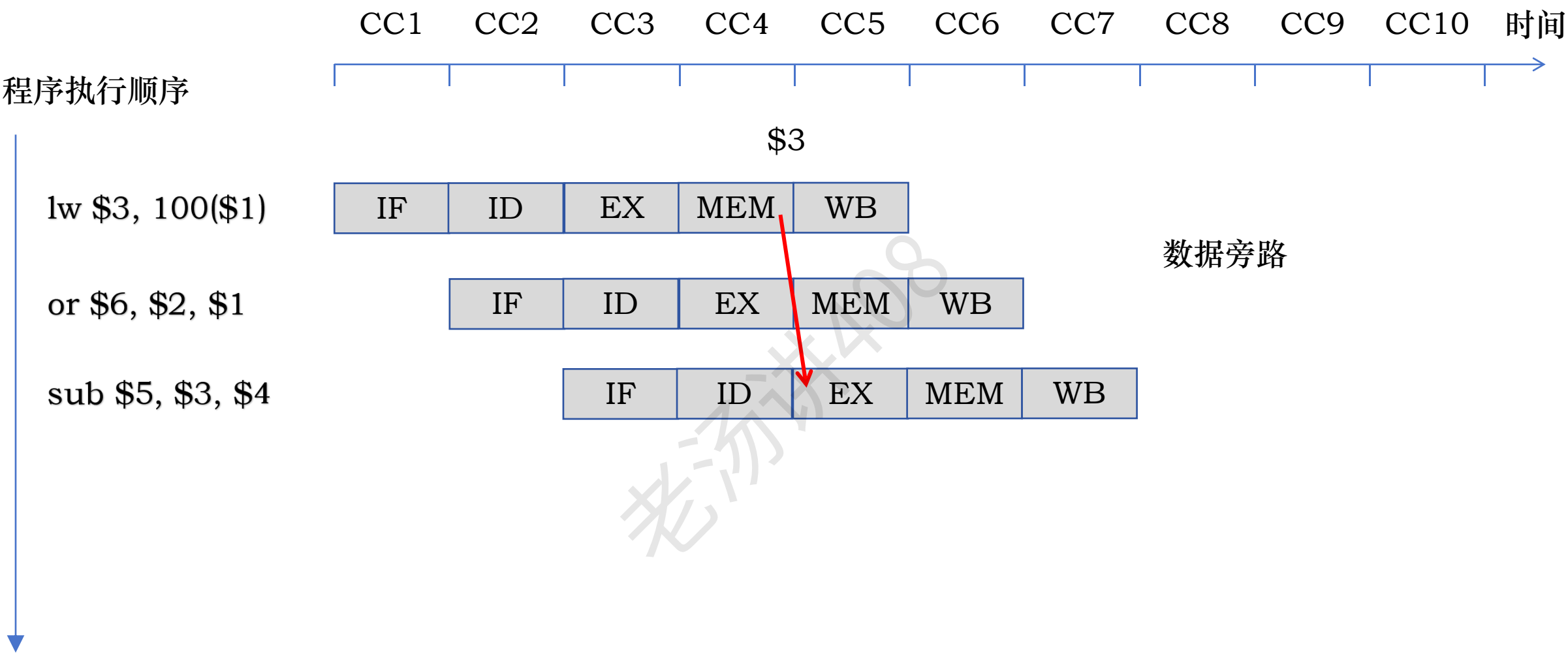
流水线处理器：数据冒险（数据转发技术）

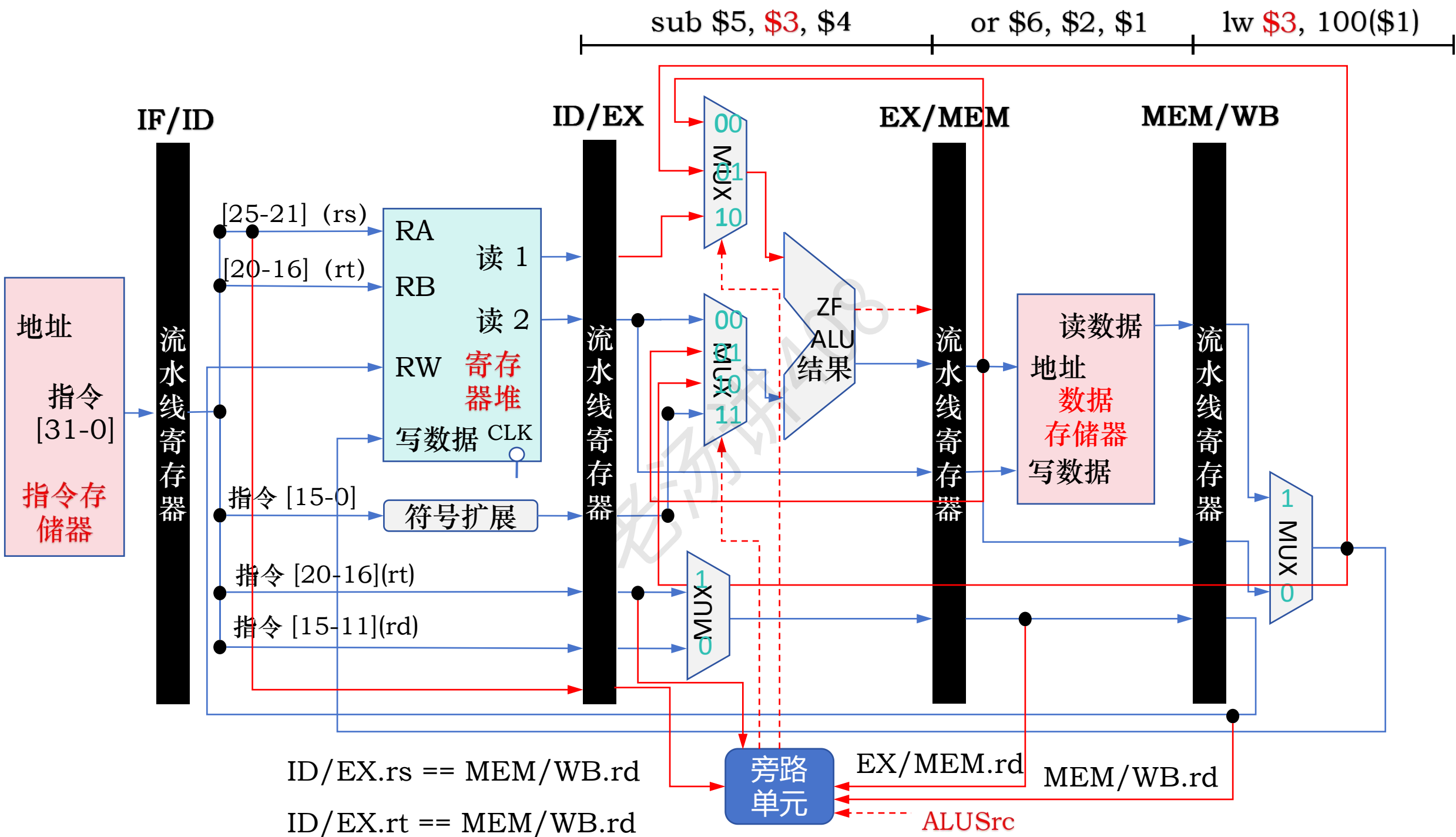






流水线处理器：数据冒险（数据转发技术）





流水线处理器：数据冒险（数据转发技术）

add \$2, \$3, \$4

sub \$5, \$2, \$3

add \$3, \$2, \$1

or \$6, \$2, \$1

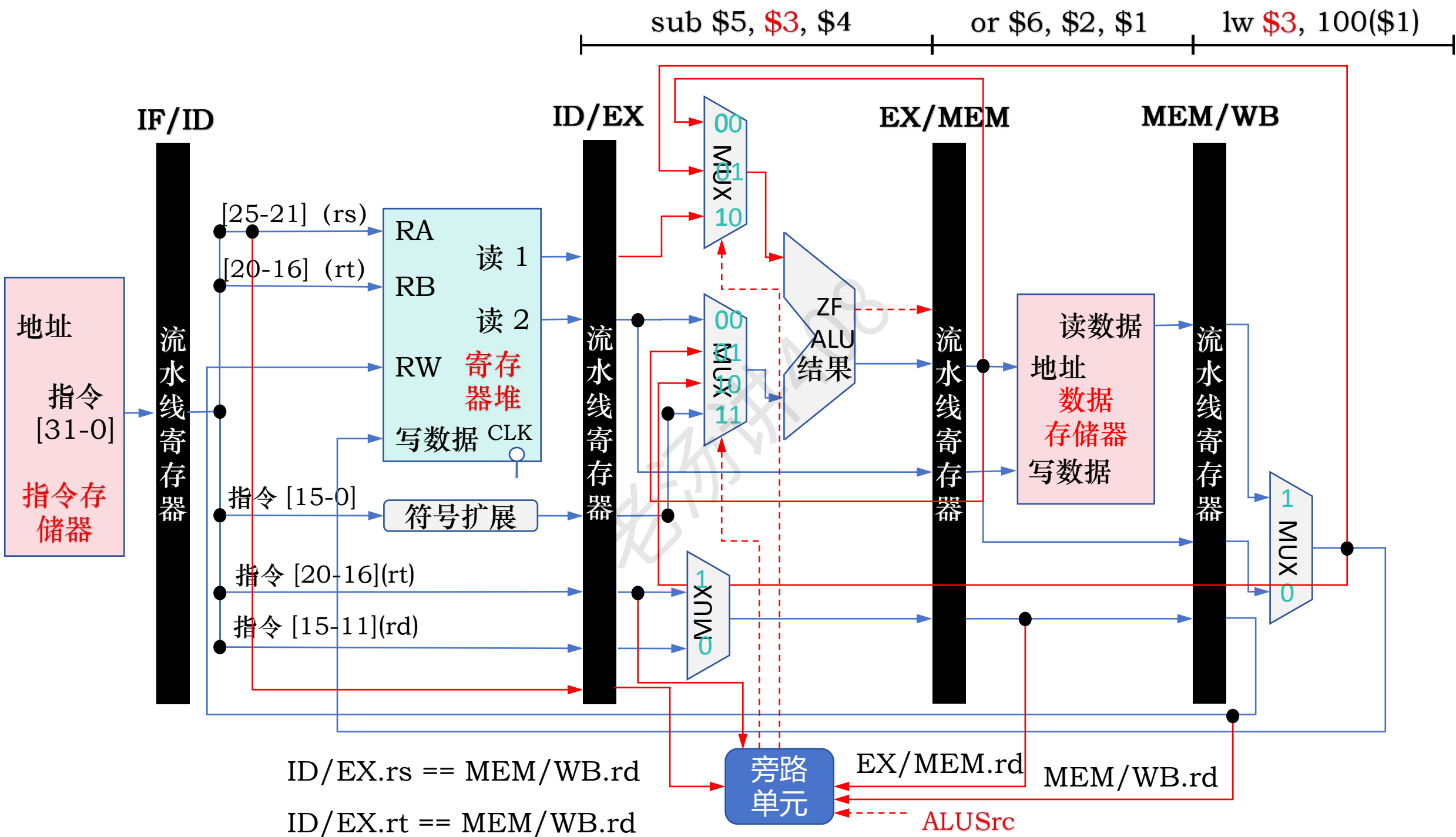
sub \$5, \$3, \$4

lw \$3, 100(\$1)

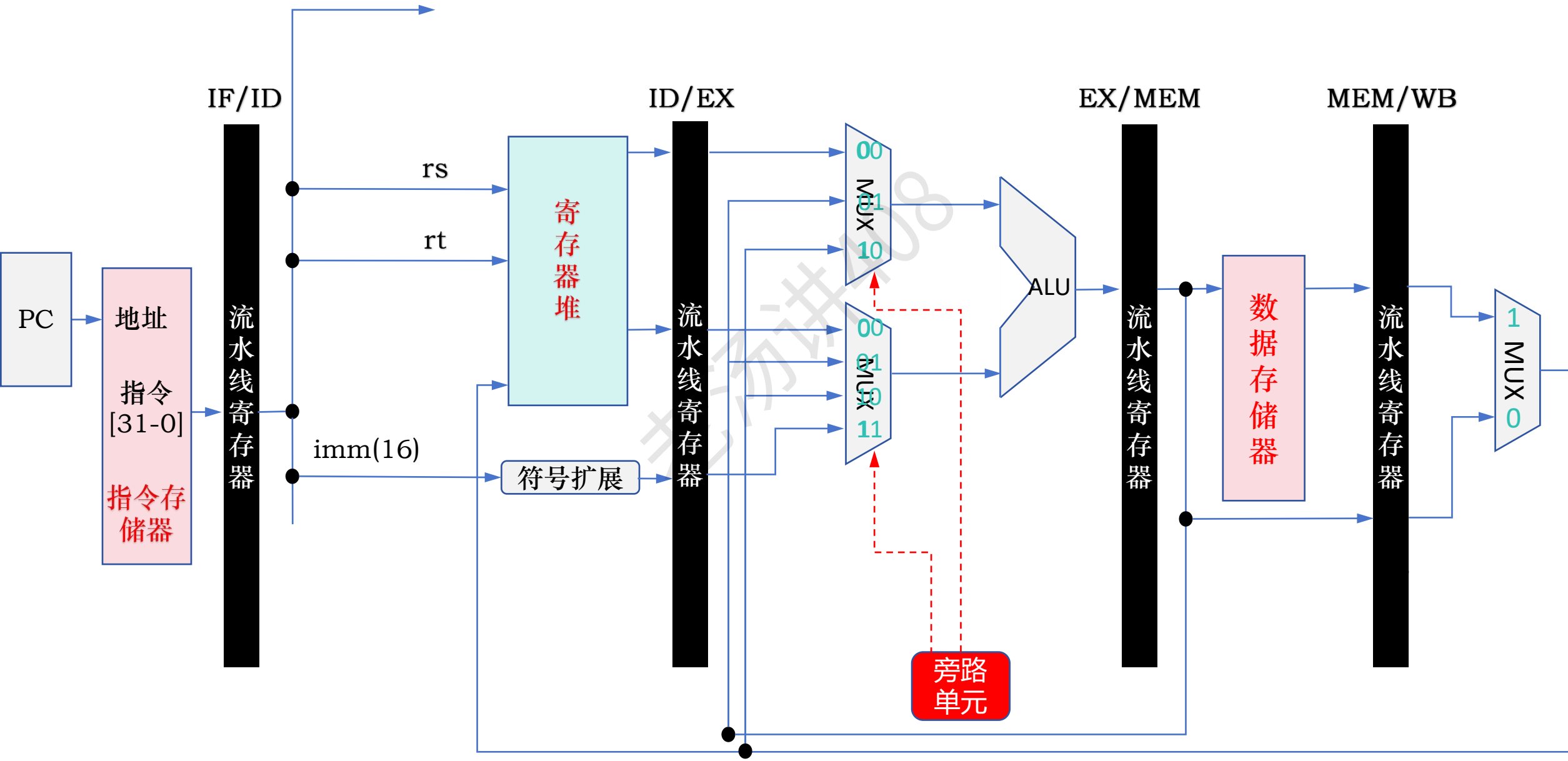
or \$6, \$2, \$1

sub \$5, \$3, \$4

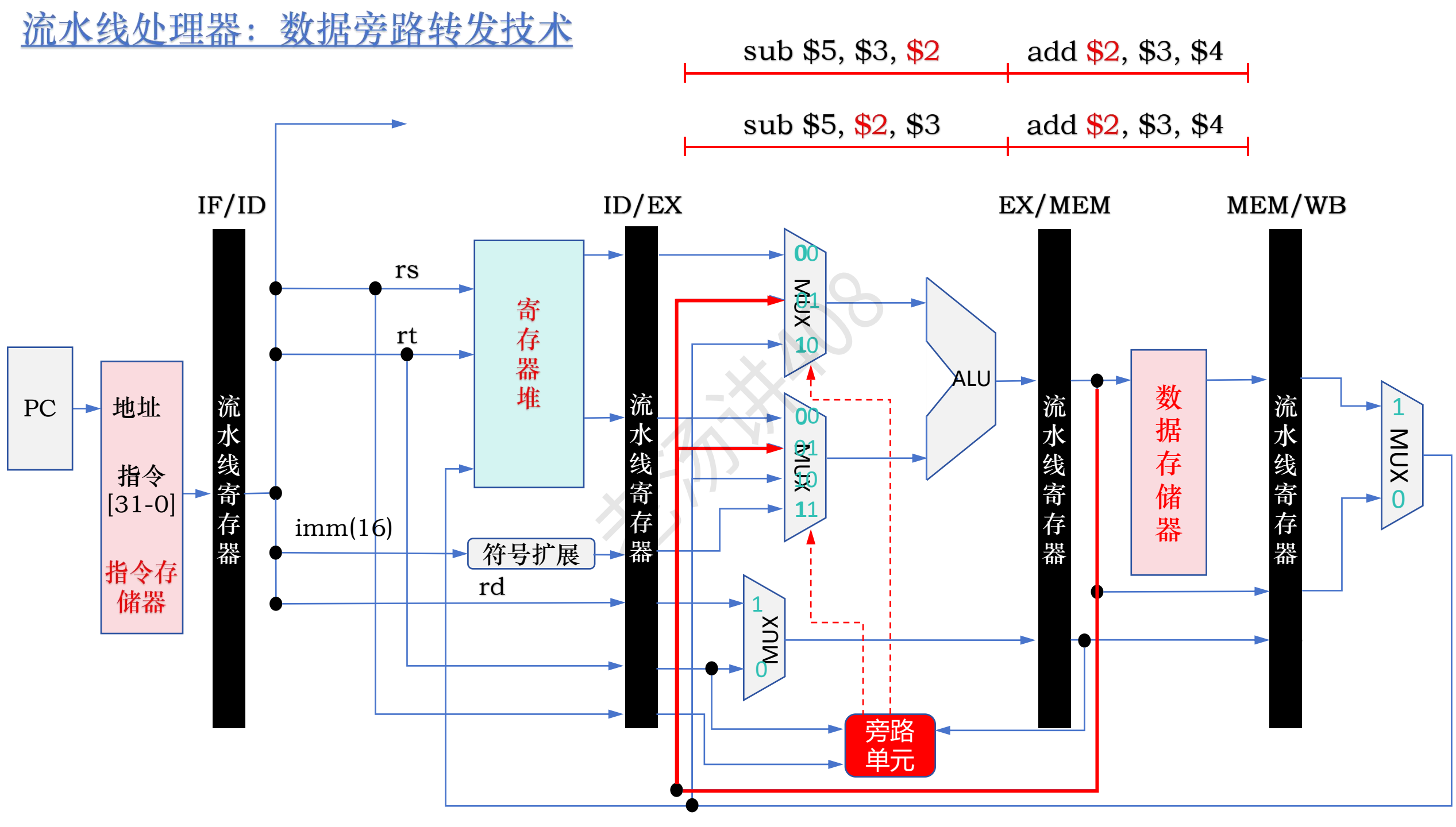
写后读（Read After Write、RAW）数据冒险



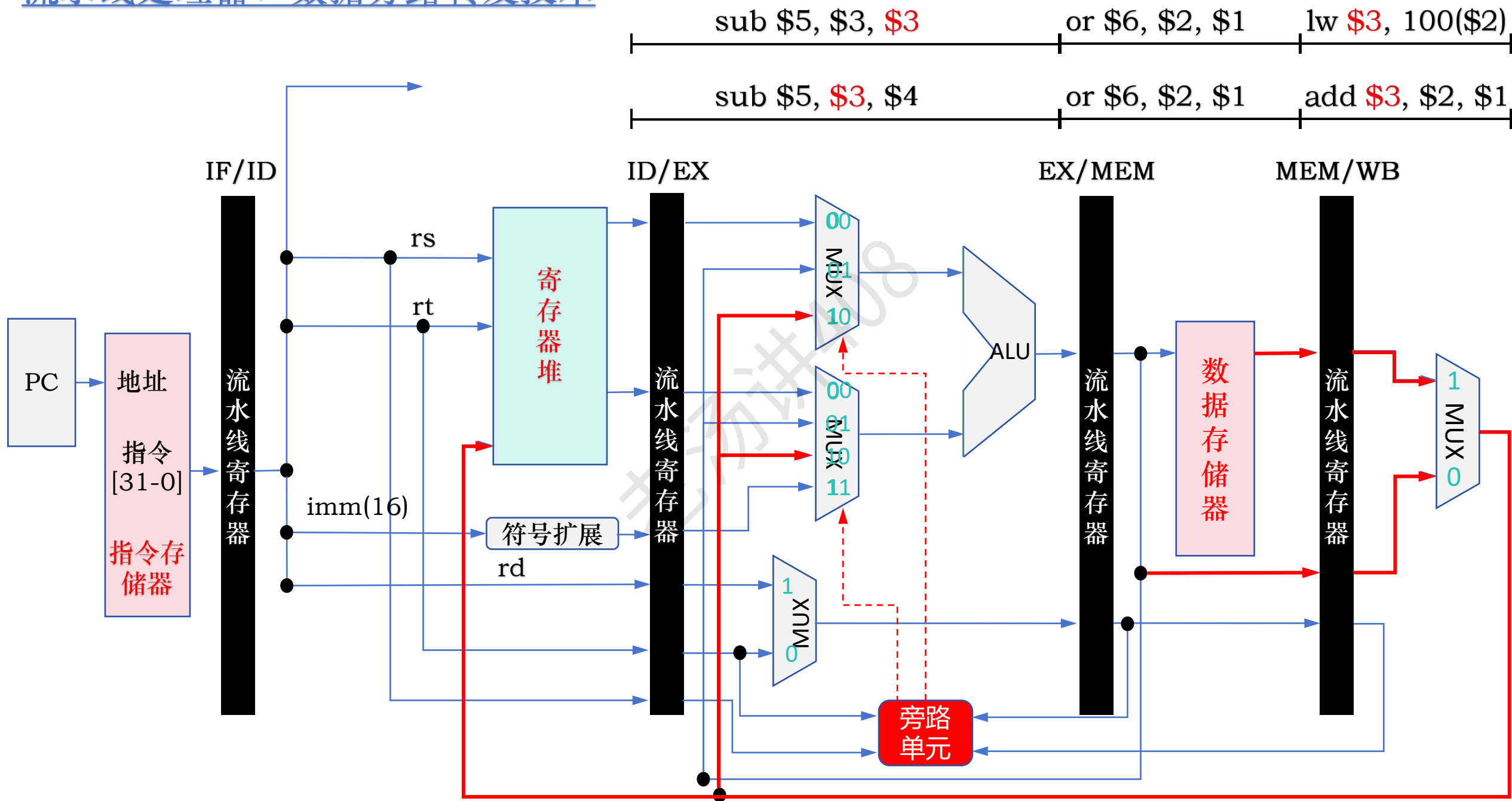
流水线处理器：数据旁路转发技术



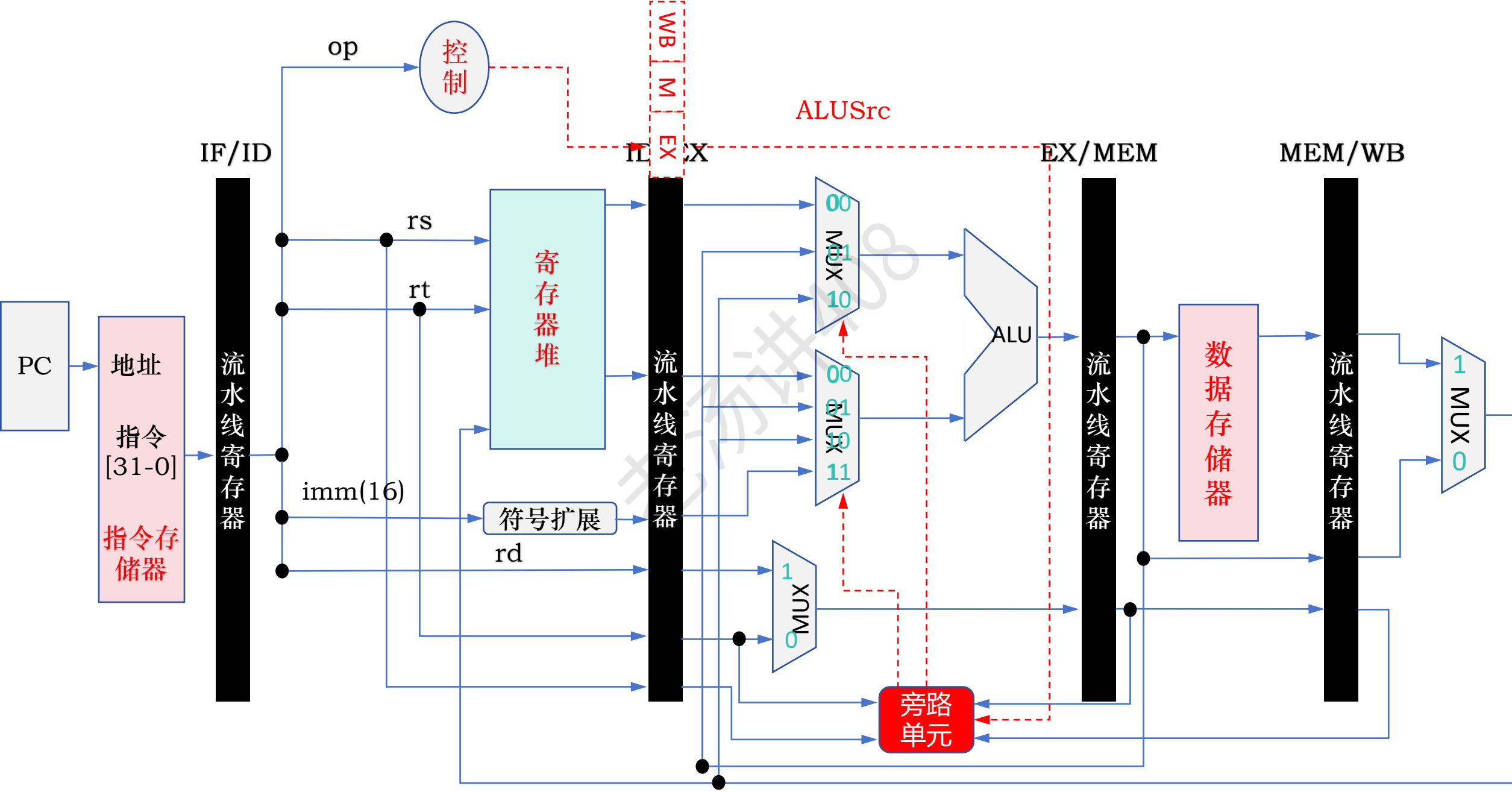
流水线处理器：数据旁路转发技术



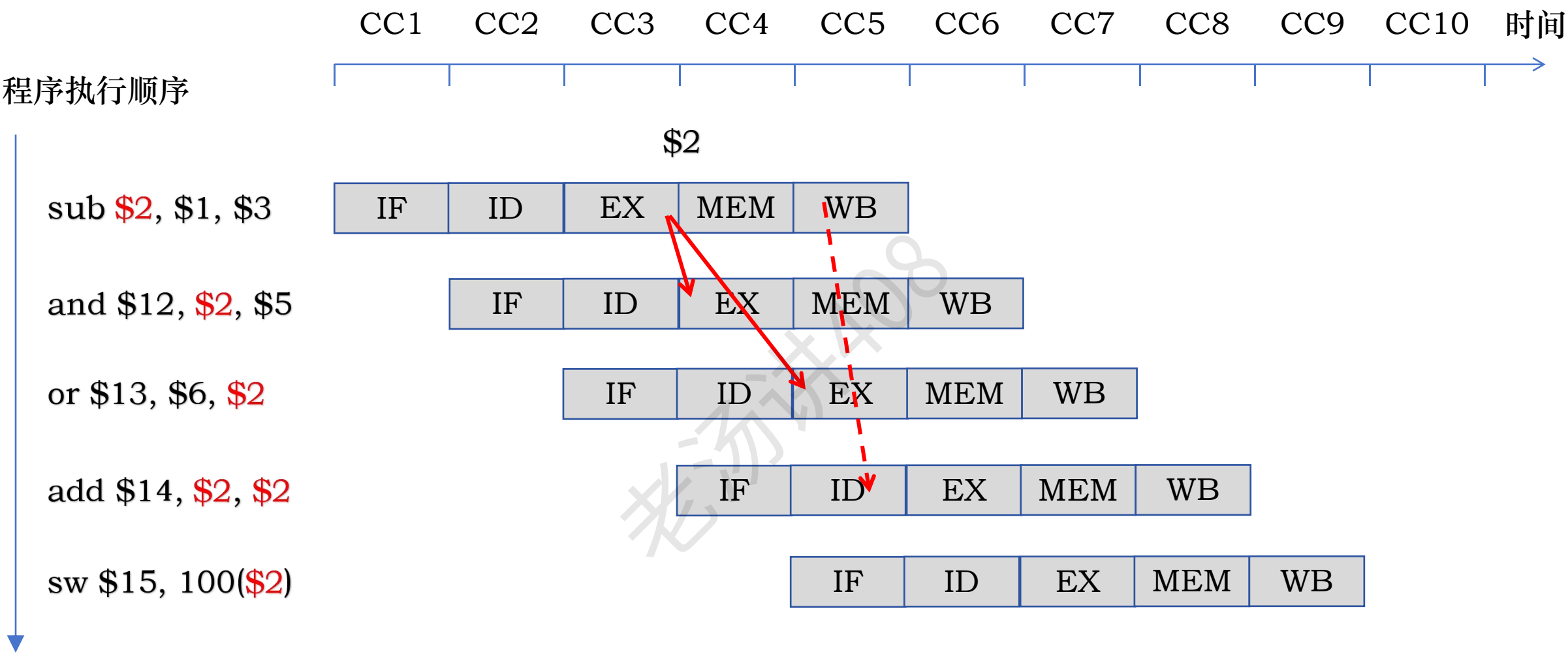
流水线处理器：数据旁路转发技术



流水线处理器：数据旁路转发技术

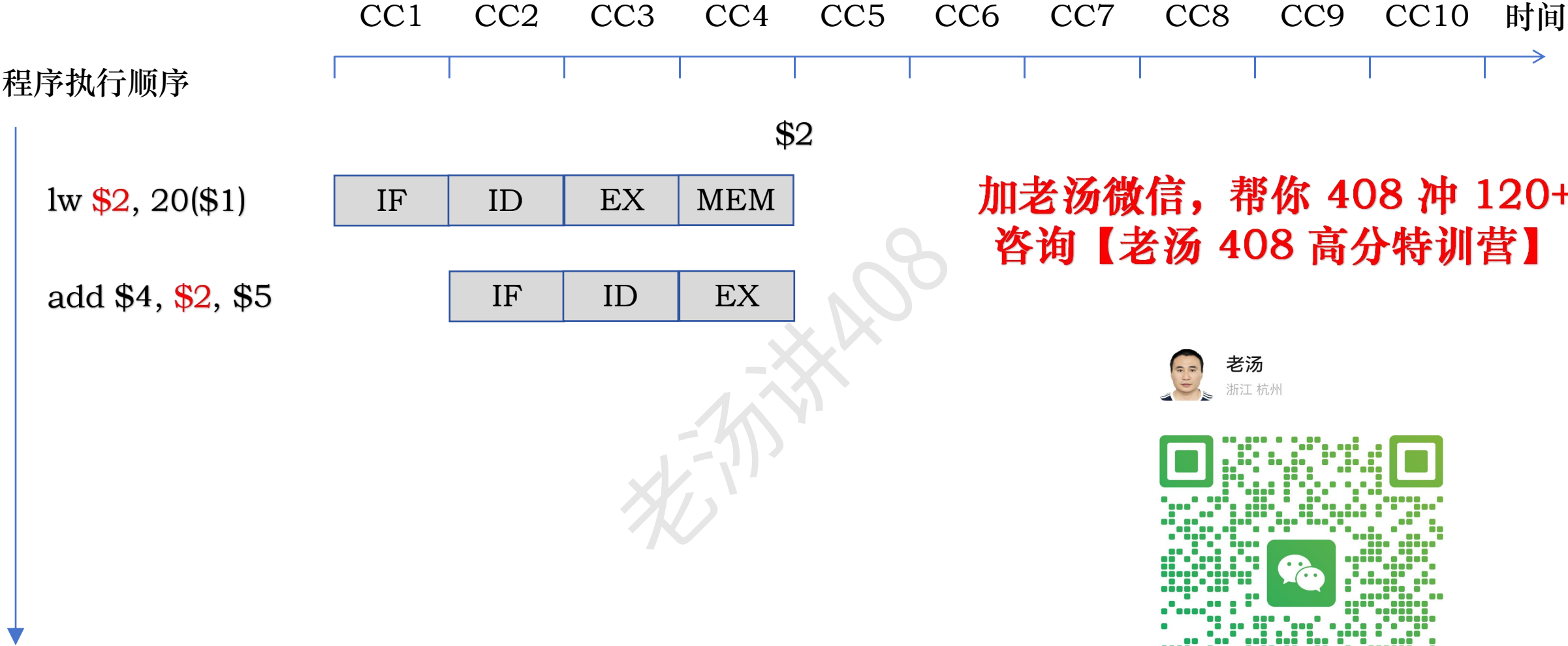


流水线处理器：数据冒险（数据转发技术）

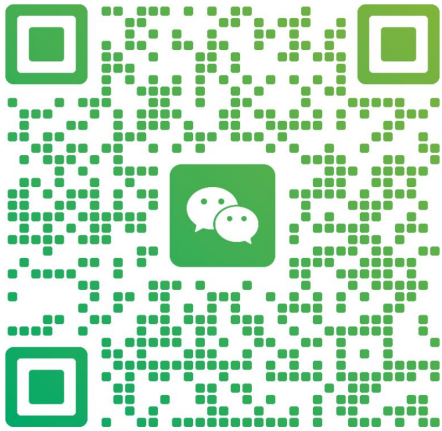


寄存器的写可以发生在前半个时钟周期内，而后半个时钟周期来读取寄存器

流水线处理器：数据冒险（数据转发技术）

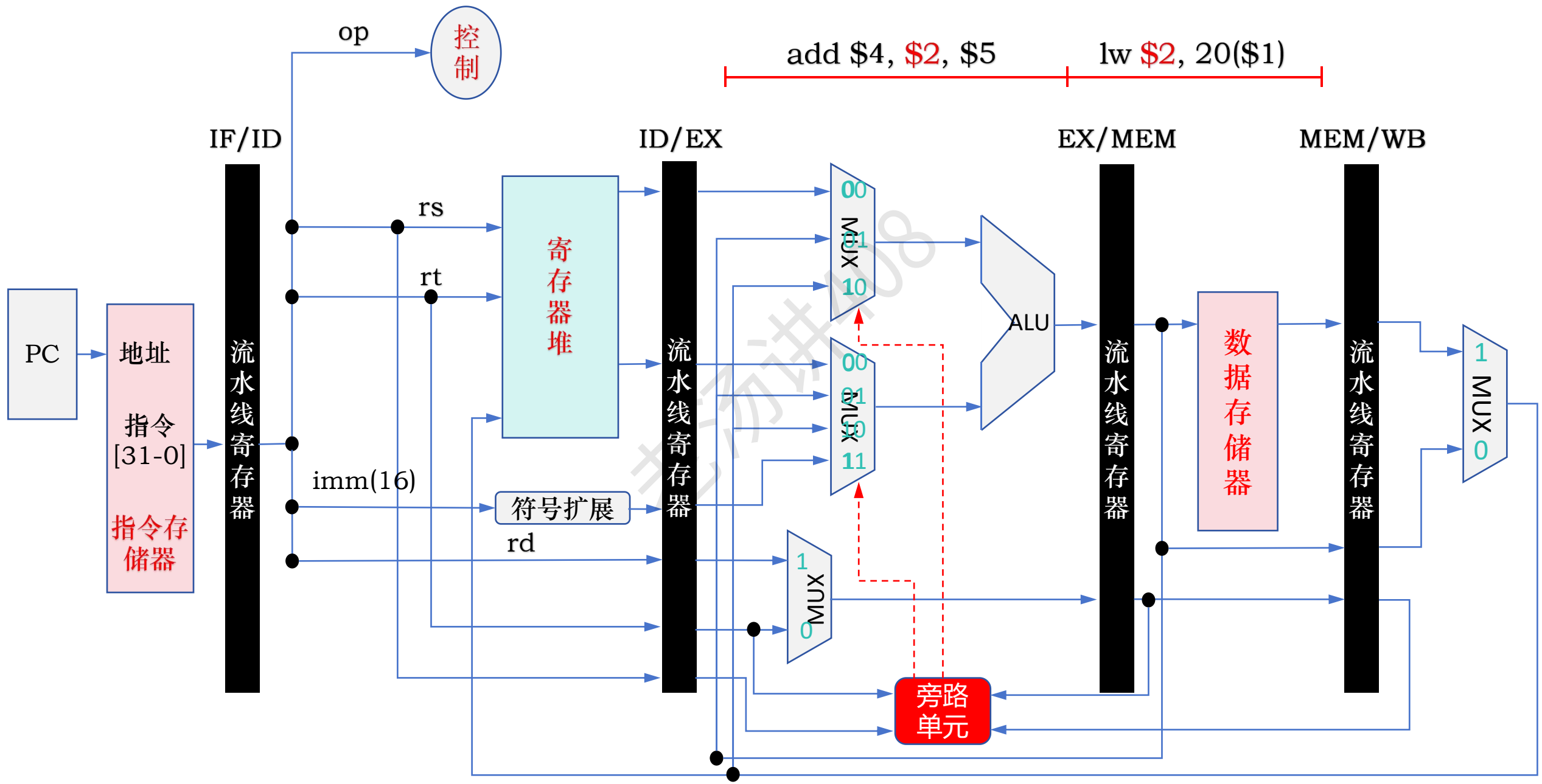


加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

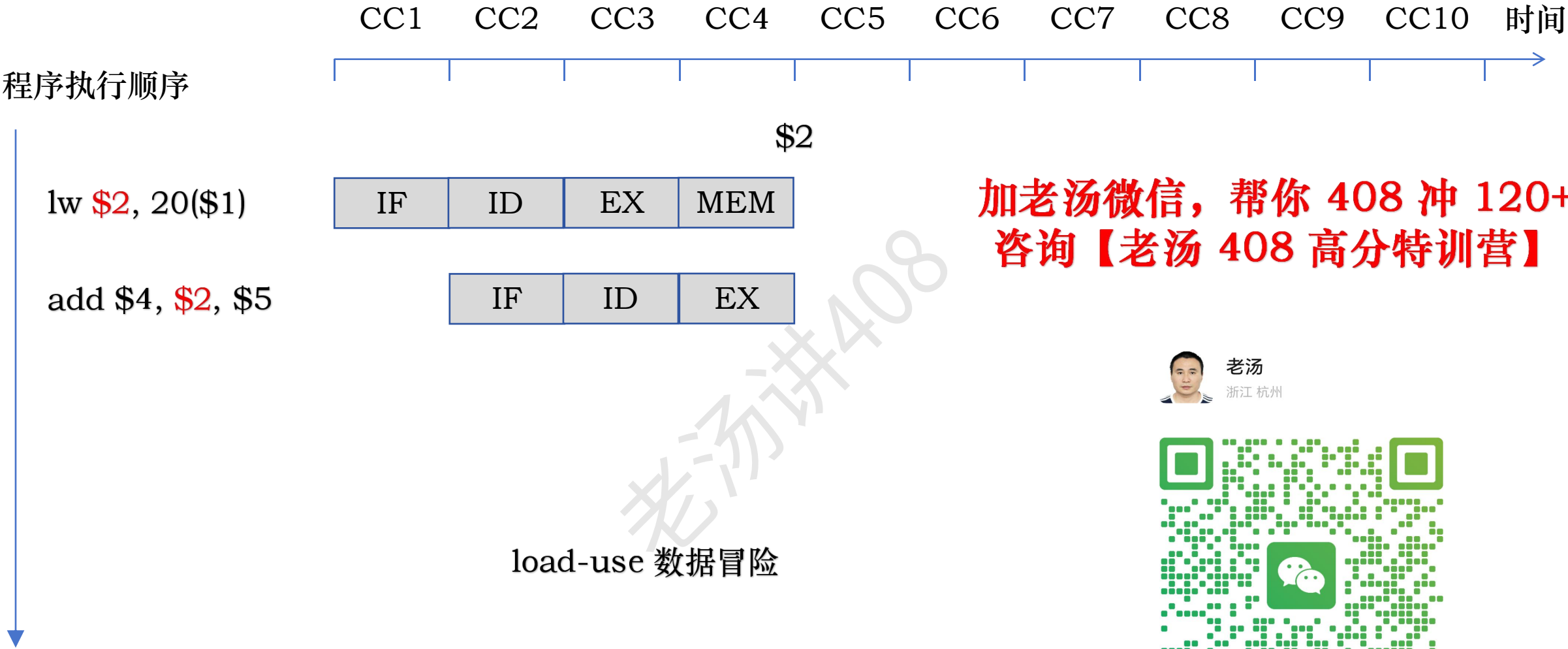


扫一扫上面的二维码图案，加我为朋友。

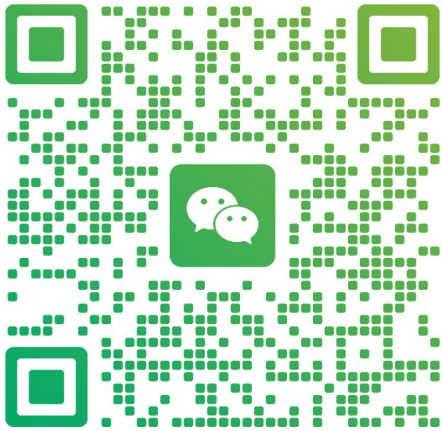
流水线处理器：数据旁路转发技术



流水线处理器：数据冒险（数据转发技术）



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

流水线处理器: load-use 数据冒险 (插入 nop 指令)

```
#include <stdio.h>
```

```
int add_3(int A[], int h) {  
    int g = h + A[3];  
    return g;  
}
```



```
lw      $2, 12($4)  
nop  
addu    $2, $2, $5
```

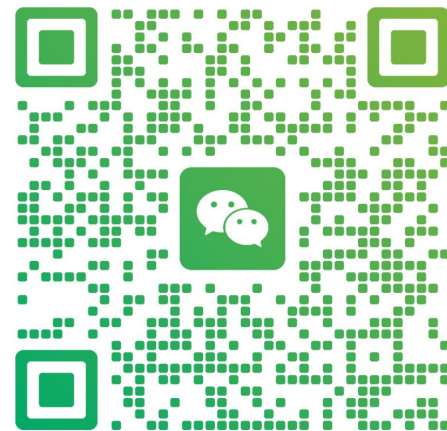
```
int main() {  
    int A[] = {1, 2, 3, 4, 5, 6};  
    int res = add_3(A, 10);  
    printf("res = %d\n", res);  
    return 0;  
}
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



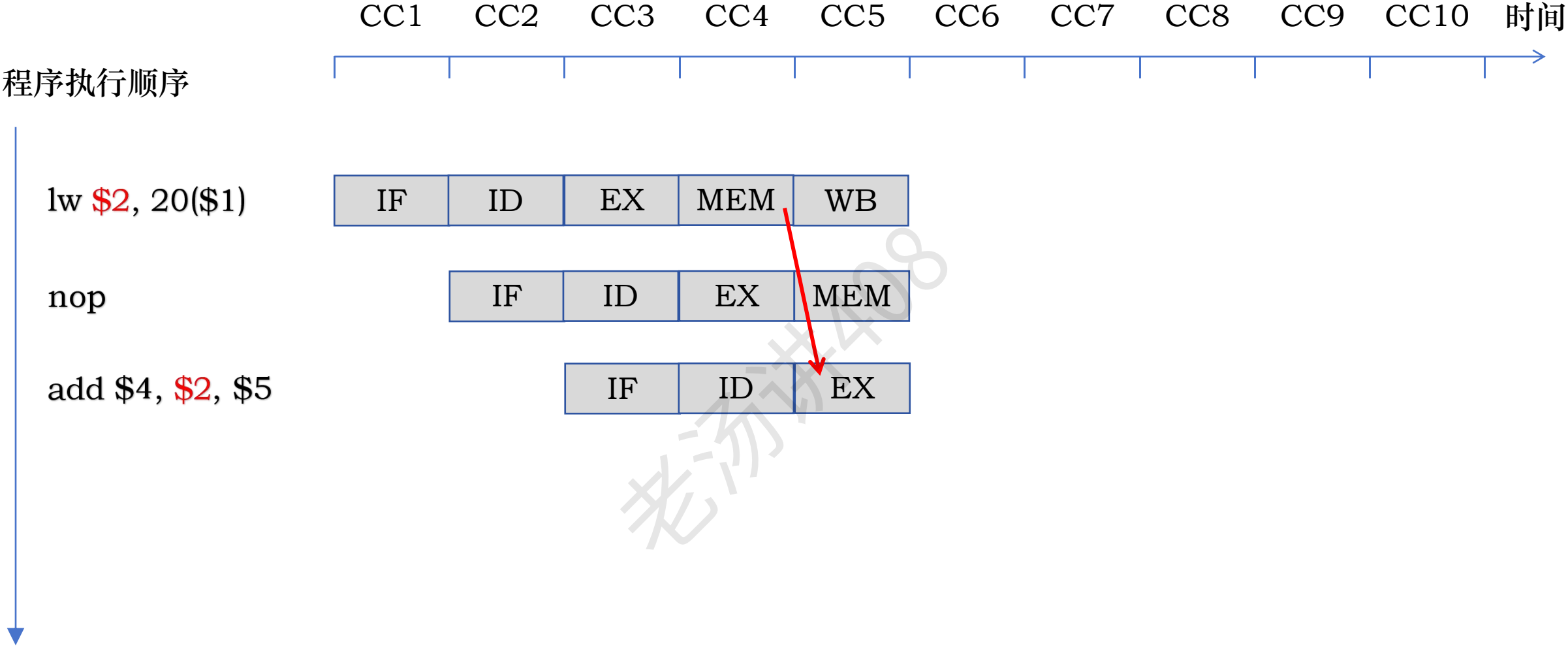
老汤

浙江 杭州

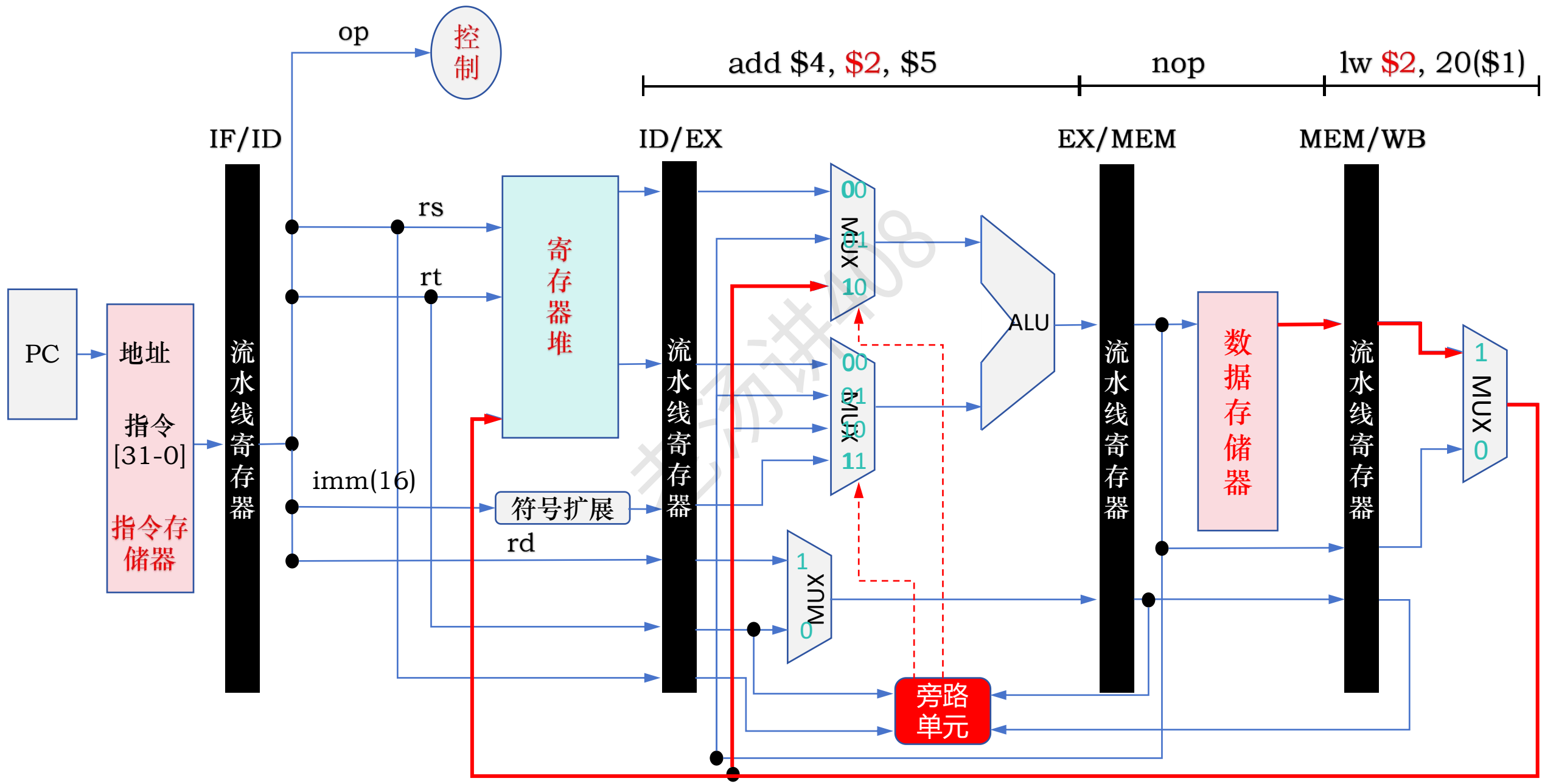


扫一扫上面的二维码图案，加我为朋友。

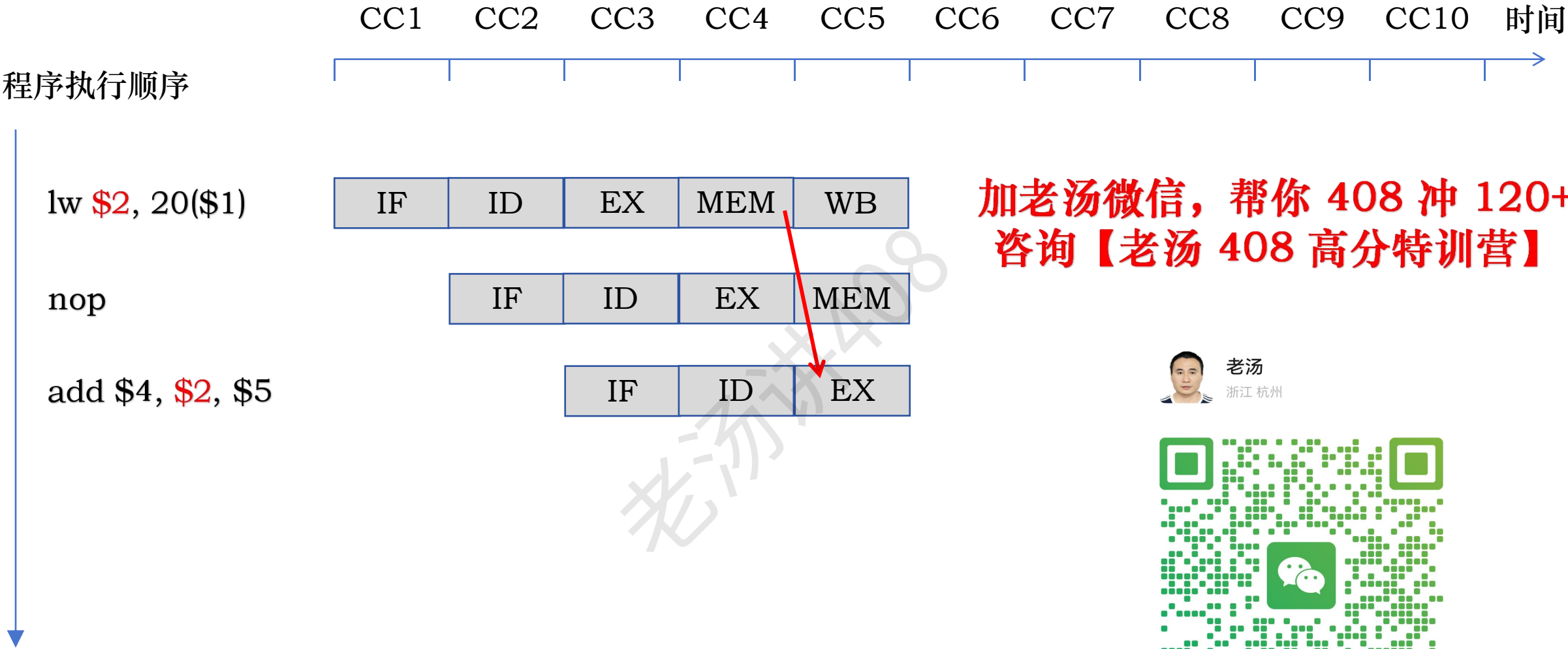
流水线处理器：load-use 数据冒险（插入 nop 指令）



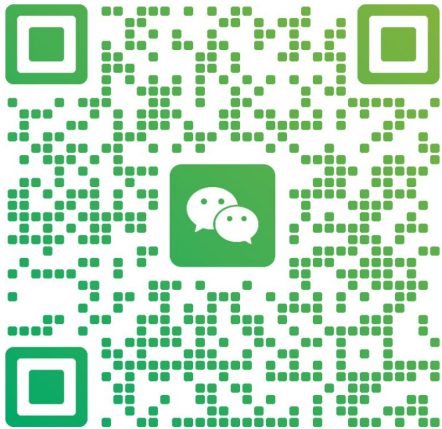
流水线处理器：数据旁路转发技术



流水线处理器: load-use 数据冒险 (插入 nop 指令)



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

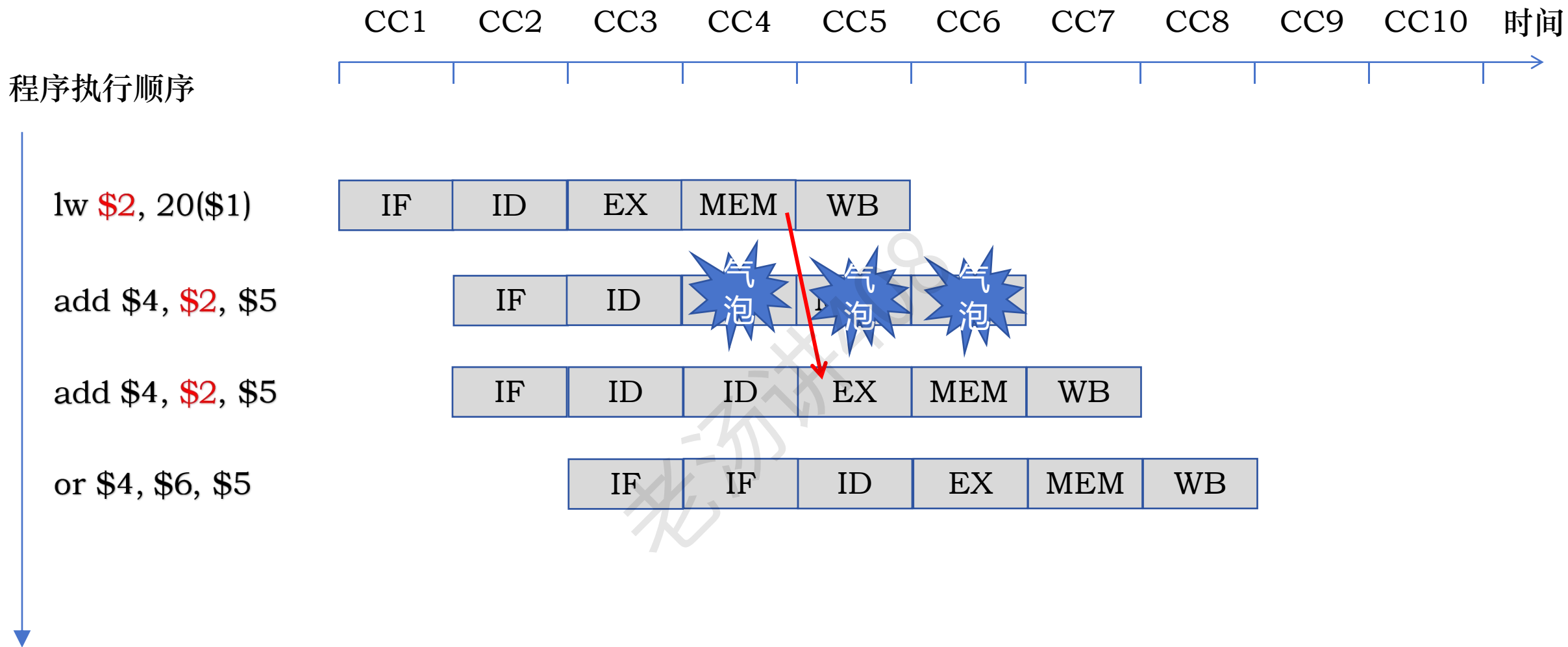


扫一扫上面的二维码图案，加我为朋友。

一条 nop 指令需要占 4 个字节的存储空间

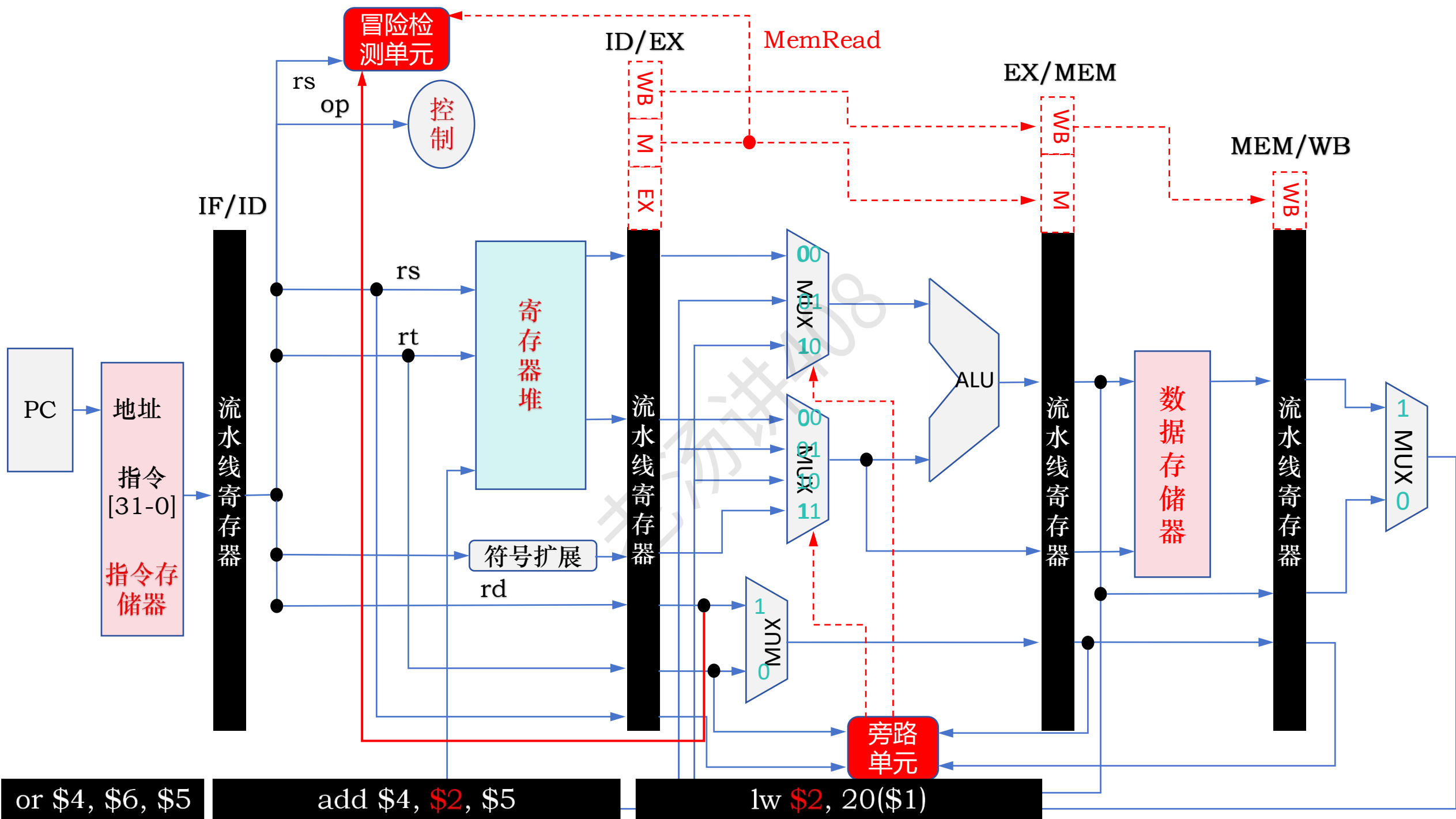
插入气泡

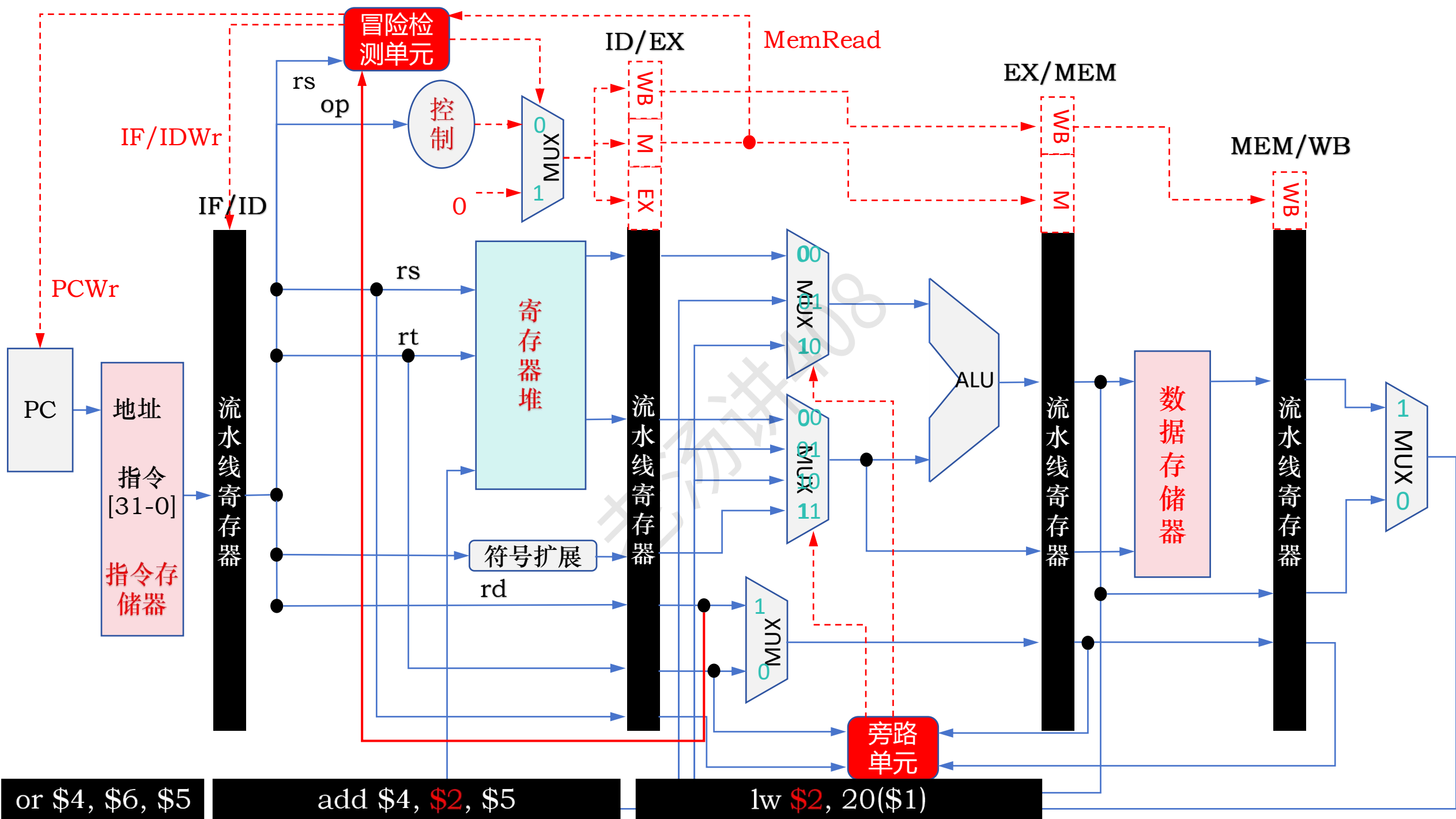
流水线处理器: load-use 数据冒险 (插入气泡)



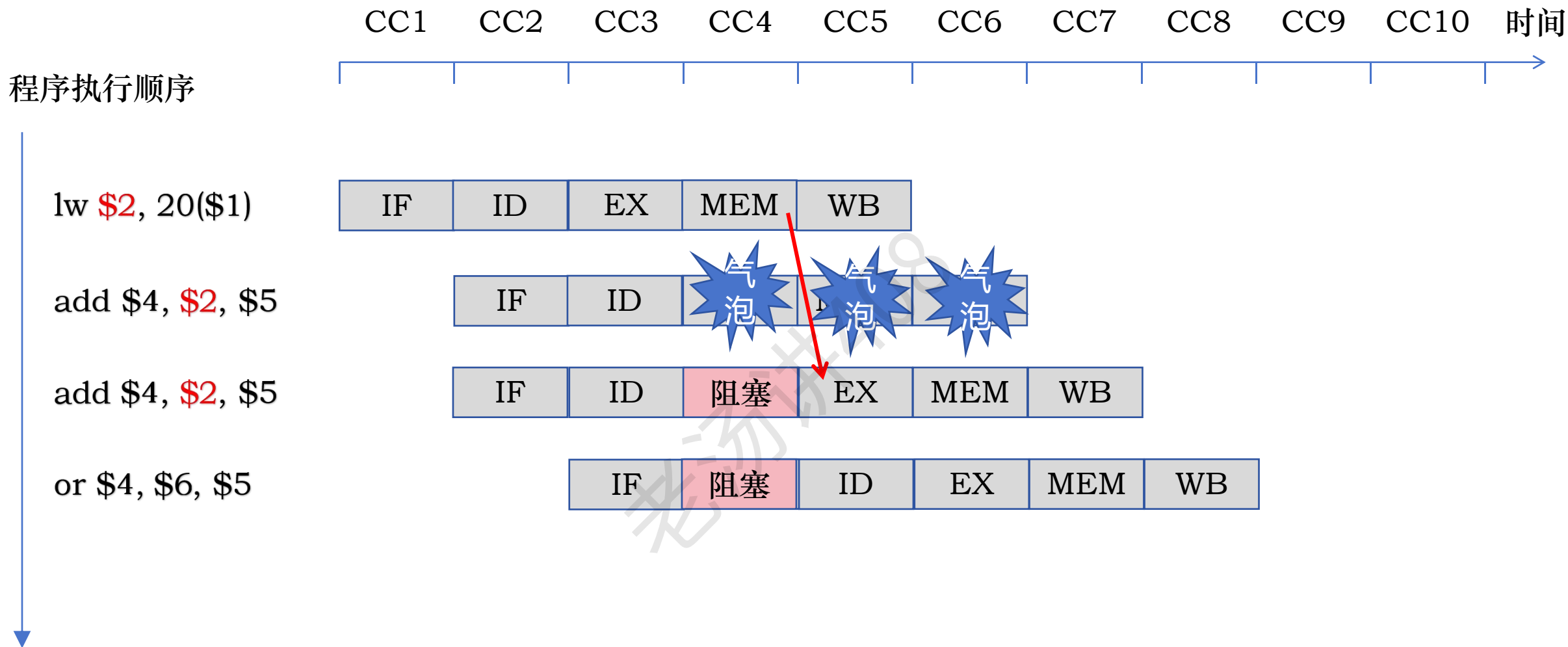
就是在硬件上采取措施, 使得发生数据冒险的指令延迟执行

通过硬件阻塞方式阻止后续指令的执行





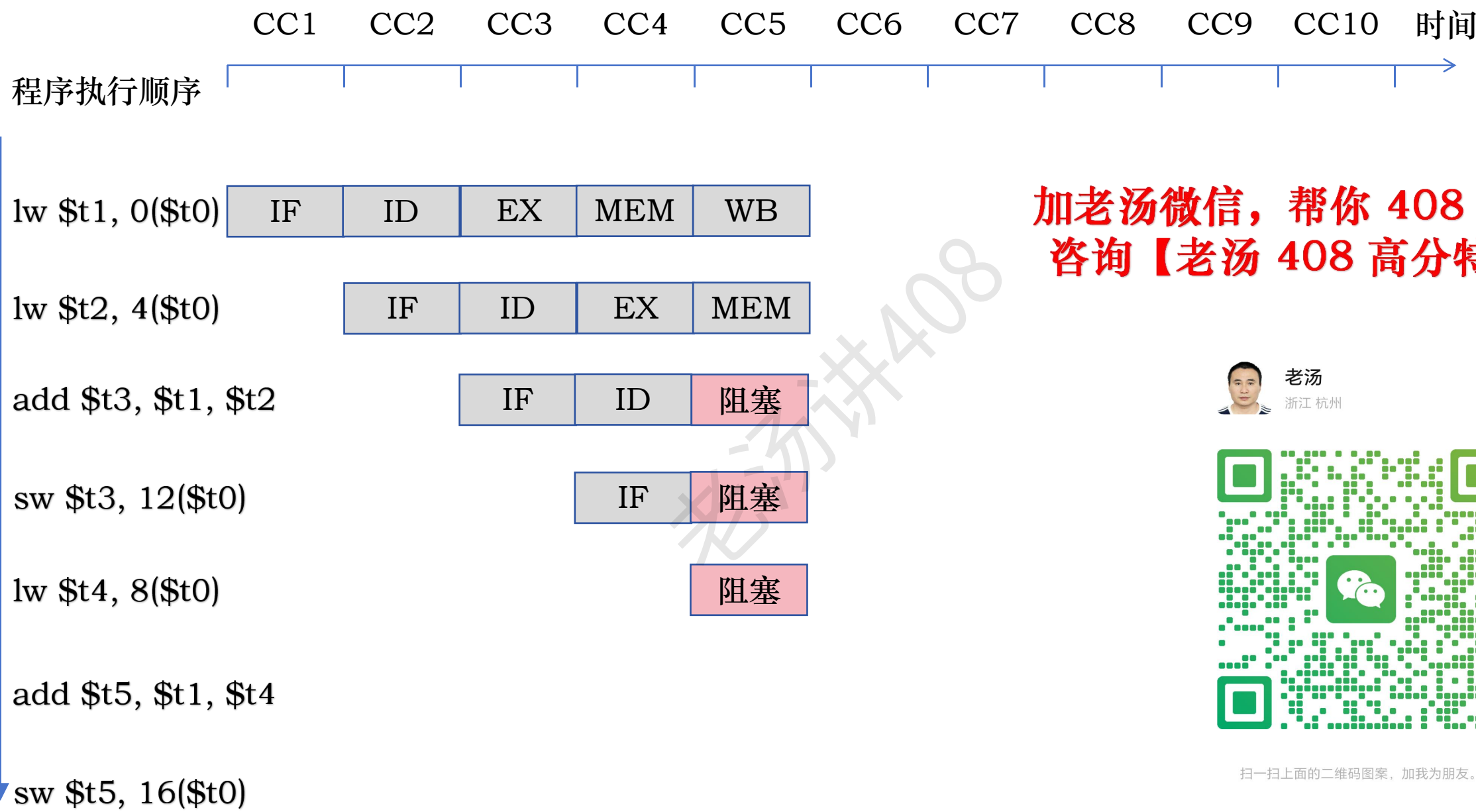
流水线处理器: load-use 数据冒险 (插入气泡)



就是在硬件上采取措施, 使得发生数据冒险的指令延迟执行

通过**硬件阻塞**方式阻止后续指令的执行

流水线处理器：数据冒险

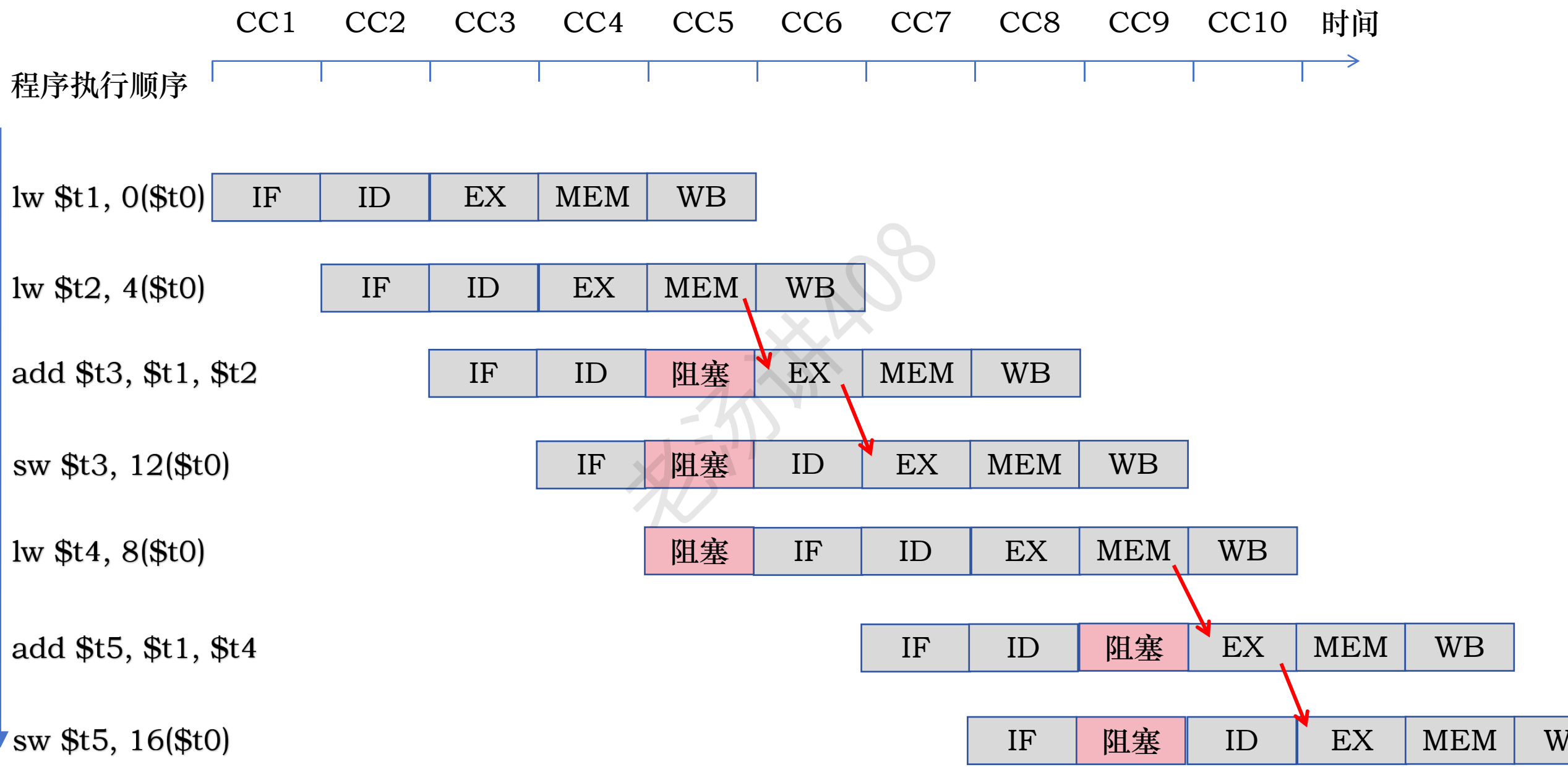


加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

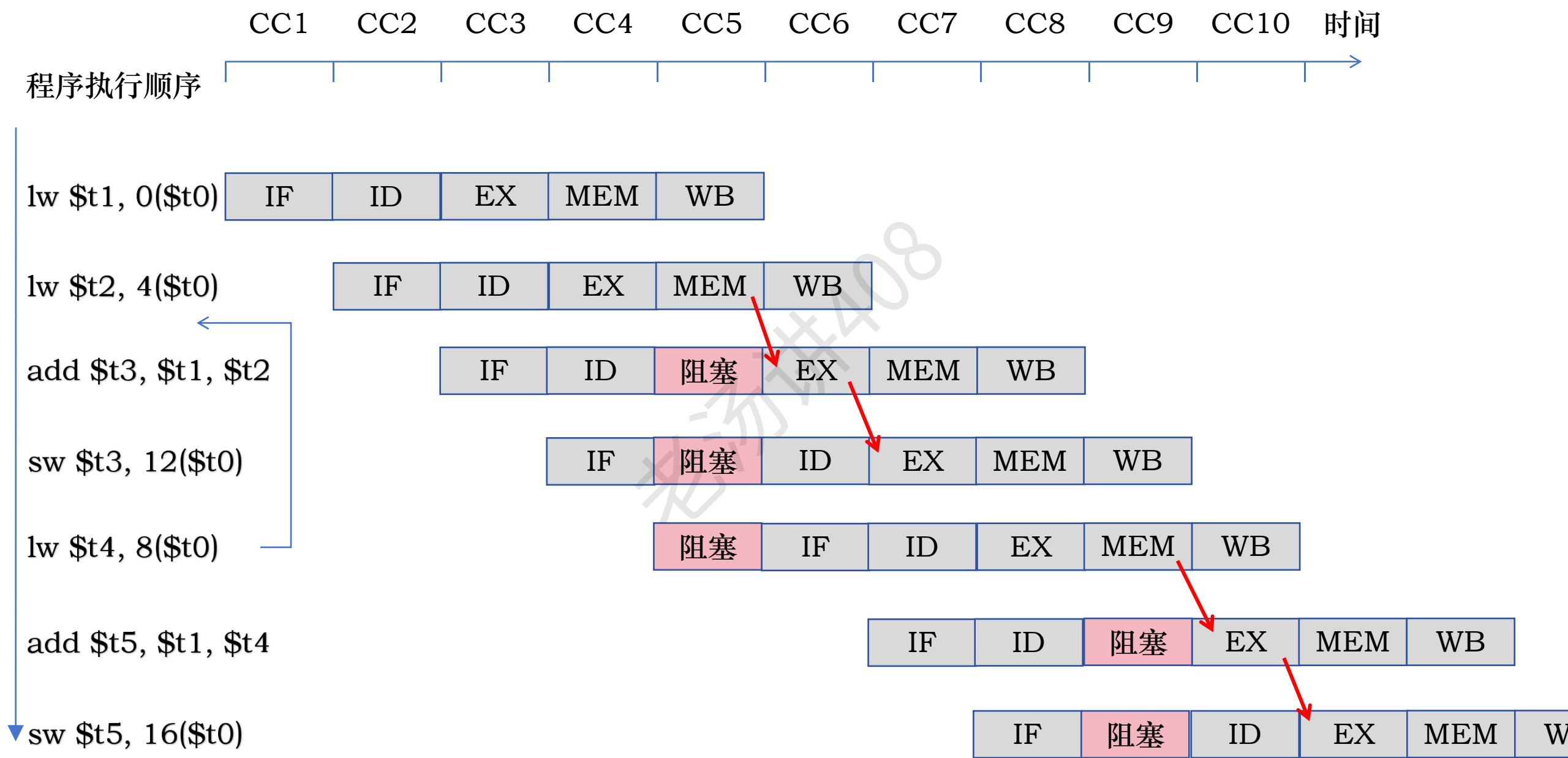


扫一扫上面的二维码图案，加我为朋友。

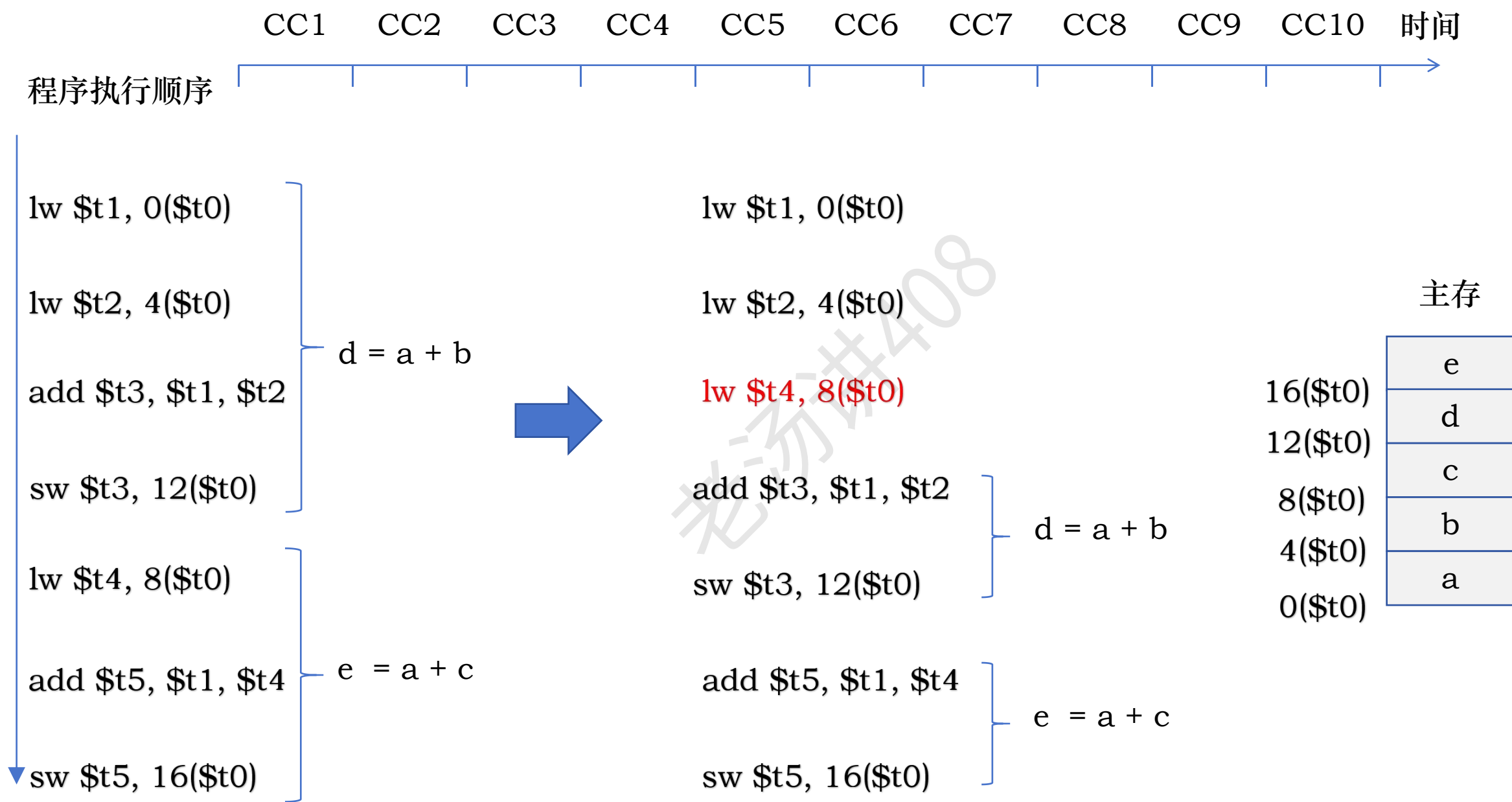
流水线处理器：数据冒险



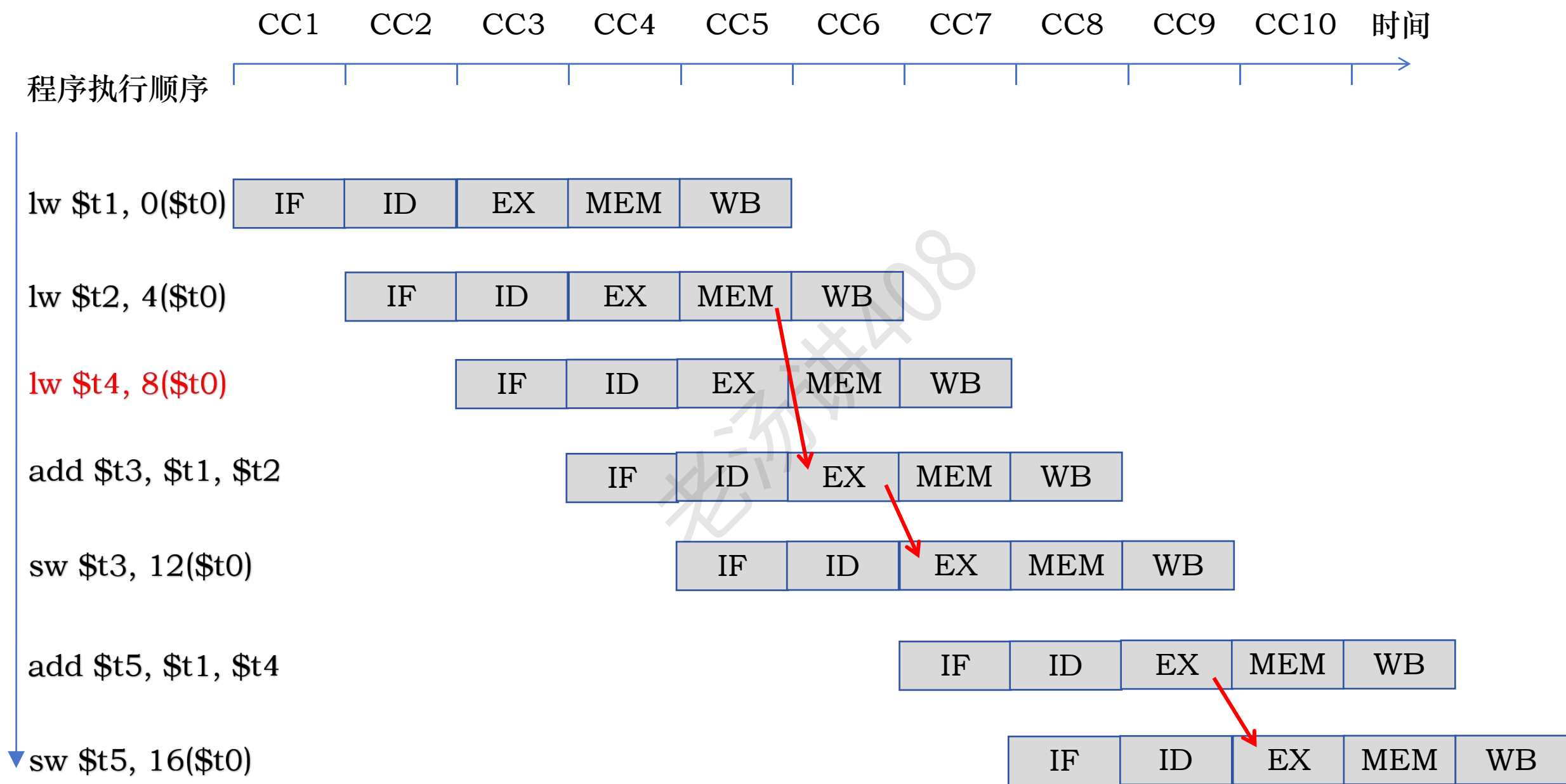
流水线处理器：数据冒险（编译器重排指令顺序）



流水线处理器：数据冒险（编译器重排指令顺序）

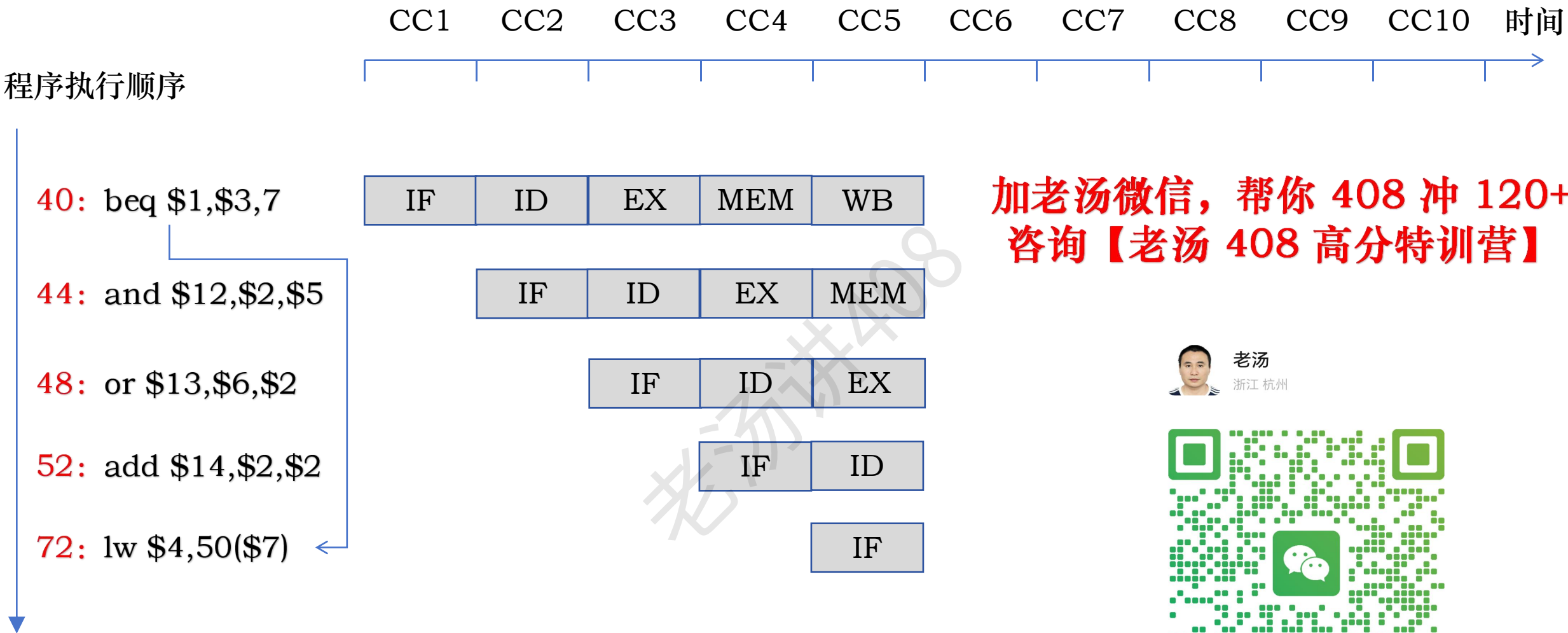


流水线处理器：数据冒险（编译器重排指令顺序）



流水线处理器：控制冒险

程序执行顺序

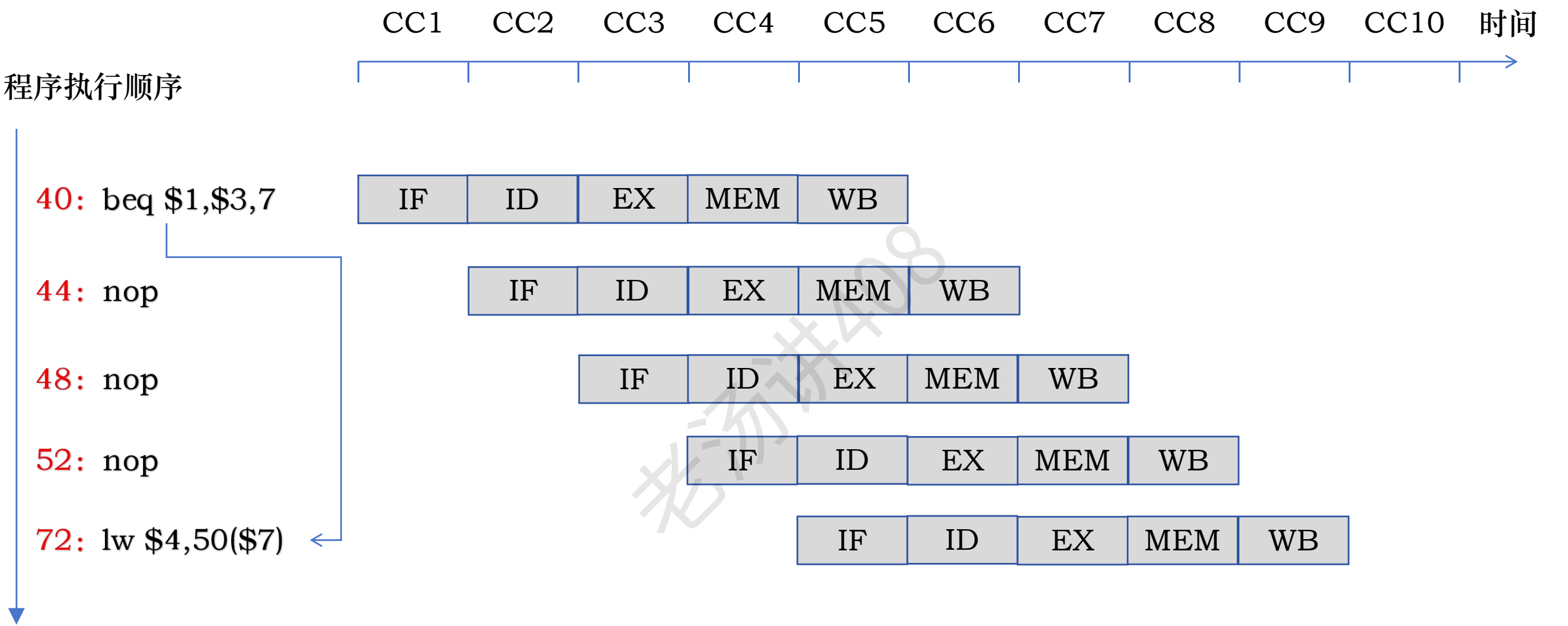


加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

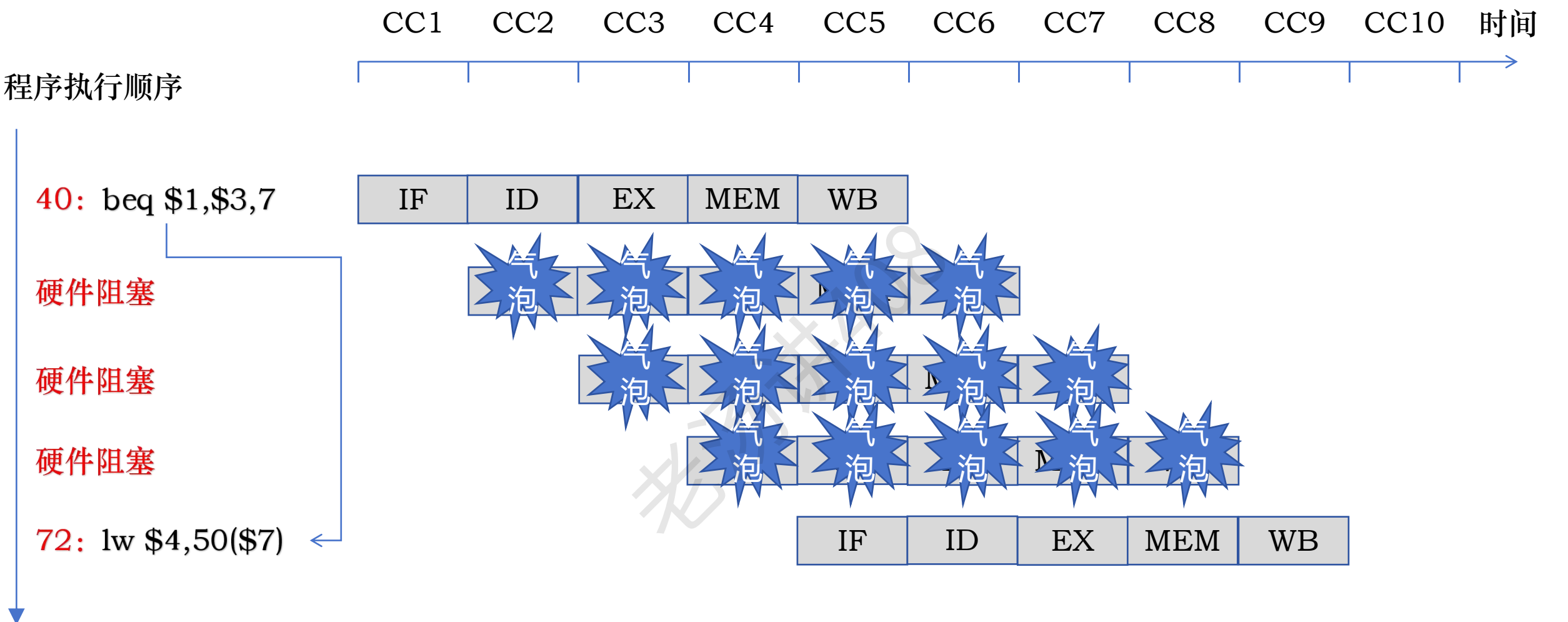
流水线处理器：控制冒险（加入 3 条 nop 指令）



由于流水线阻塞而带来的延迟执行时钟周期数称为延迟损失时间片 C

延迟损失时间片 $C = 3$

流水线处理器：控制冒险（插入气泡）



延迟损失时间片 $C = 3$

流水线处理器：控制冒险（分支预测）

① 静态预测

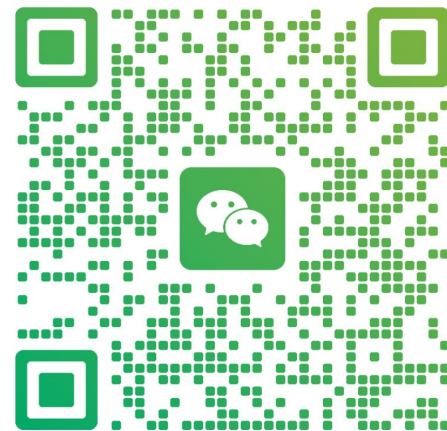
② 动态预测

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



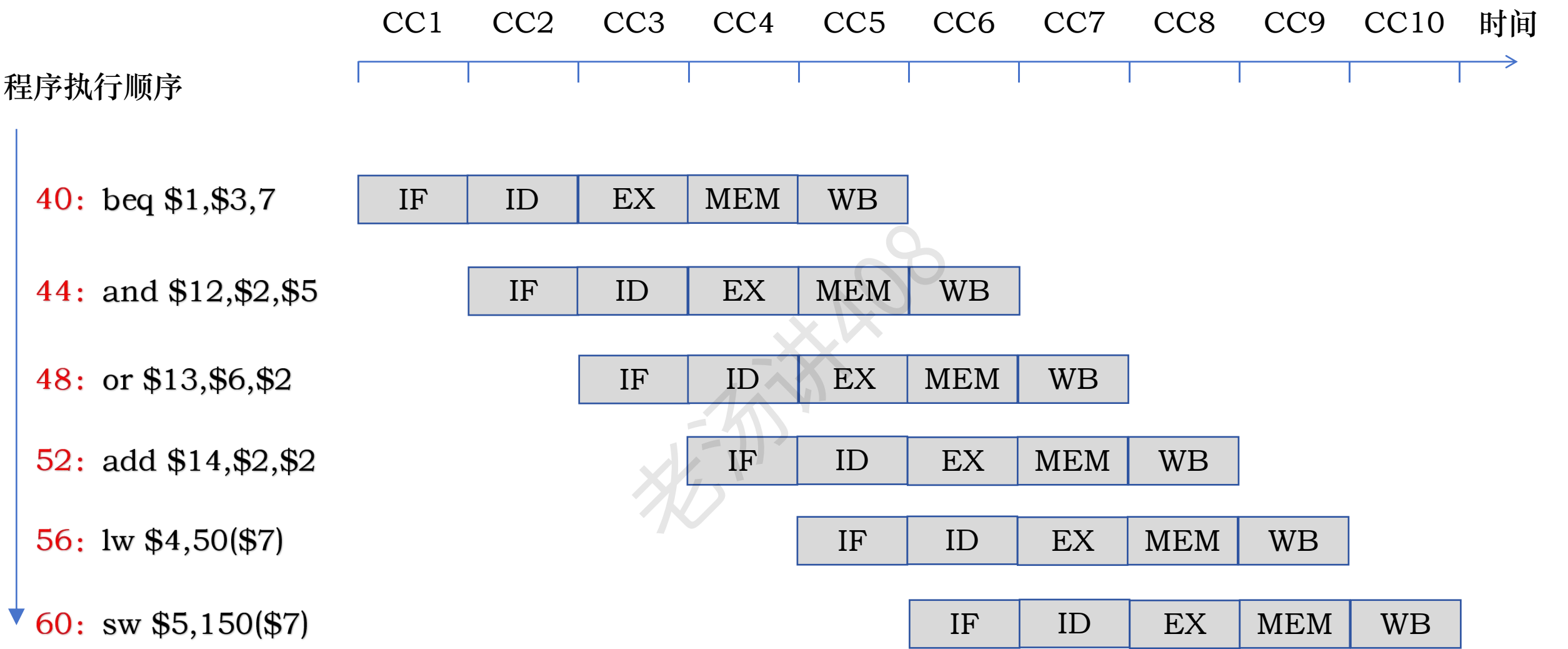
老汤

浙江 杭州

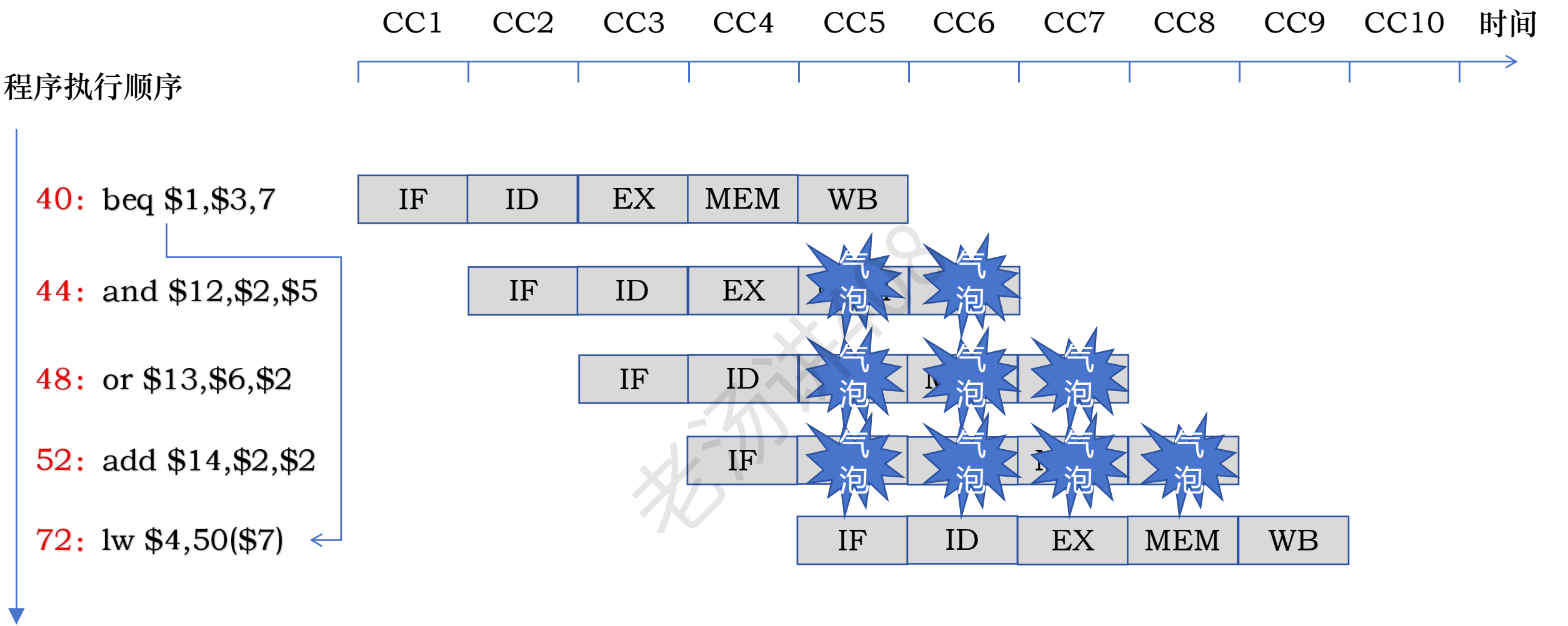


扫一扫上面的二维码图案，加我为朋友。

流水线处理器：控制冒险（分支预测 - 静态预测）



流水线处理器：控制冒险（分支预测 - 静态预测）



流水线处理器：控制冒险（分支预测 - 动态预测）

动态预测利用分支指令发生转移的**历史情况**来进行预测

```
int sum(int n) {  
    int i, j, sum = 0;  
    for (i = 0; i < n; i++)  
        for (j = 0; j < n; j++)  
            sum += 1;  
    return sum;  
}
```

\$6	1	i
\$5	1	
\$4	10	n
\$3	0	j
\$2	10	sum

sum:

move \$2,\$0

move \$6,\$0

b \$L2

nop

←

\$L3:

addiu \$2,\$2,1

addiu \$3,\$3,1

←

\$L4:

slt \$5,\$3,\$4

bne \$5,\$0,\$L3

nop

← j < n, 则 \$5=1

addiu \$6,\$6,1

\$L2:

slt \$3,\$6,\$4

beq \$3,\$0,\$L6

nop

← i < n, 则 \$3=1

move \$3,\$0

b \$L4

nop

设置 j = 0

\$L6:

jr \$31

流水线处理器：控制冒险（分支预测 - 动态预测）

动态预测利用分支指令发生转移的历史情况来进行预测

```
int sum(int n) {
    int i, j, sum = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            sum += 1;
    return sum;
}
```

\$6	1	i
\$5		
\$4	10	n
\$3	10	j
\$2	10	sum

0 表示预测下次不发生跳转

1 表示预测下次发生跳转

分支指令主存地址	预测值	目的地址
40	1	\$L3
56	0	\$L6

sum:

```
move    $2,$0
move    $6,$0
b        $L2
nop
```

\$L3:

```
addiu   $2,$2,1
addiu   $3,$3,1
```

\$L4:

```
40:     slt    $5,$3,$4
        bne   $5,$0,$L3
        nop
```

```
addiu   $6,$6,1
```

\$L2:

```
56:     slt    $3,$6,$4
        beq   $3,$0,$L6
        nop
```

```
move    $3,$0
b        $L4
nop
```

\$L6:

```
jr      $31
```

预测失败次数:

2

流水线处理器：控制冒险（分支预测 - 动态预测）

动态预测利用分支指令发生转移的历史情况来进行预测

```
int sum(int n) {  
    int i, j, sum = 0;  
    for (i = 0; i < n; i++)  
        for (j = 0; j < n; j++)  
            sum += 1;  
    return sum;  
}
```

\$6	10	i
\$5		
\$4	10	n
\$3	10	j
\$2	100	sum

0 表示预测下次不发生跳转

1 表示预测下次发生跳转

分支指令主存地址	预测值	目的地址
40	1	\$L3
56	0	\$L6

sum:

	move	\$2,\$0
	move	\$6,\$0
	b	<u>\$L2</u>
	nop	

\$L3:

	addiu	\$2,\$2,1
	addiu	\$3,\$3,1

\$L4:

40:	slt	\$5,\$3,\$4
	bne	\$5,\$0, <u>\$L3</u>
	nop	



预测失败次数:

20

	addiu	\$6,\$6,1
--	-------	-----------

\$L2:

56:	slt	\$3,\$6,\$4
	beq	\$3,\$0, <u>\$L6</u>
	nop	



预测失败次数:

1

	move	\$3,\$0
	b	<u>\$L4</u>
	nop	

\$L6:

	jr	\$31
--	----	------

流水线处理器：控制冒险（分支预测 - 动态预测）

动态预测利用分支指令发生转移的历史情况来进行预测

```
int sum(int n) {  
    int i, j, sum = 0;  
    for (i = 0; i < n; i++)  
        for (j = 0; j < n; j++)  
            sum += 1;  
    return sum;  
}
```

\$6	10	i
\$5		
\$4	10	n
\$3	10	j
\$2	100	sum

0 表示预测下次不发生跳转

1 表示预测下次发生跳转

分支指令主存地址	预测值	目的地址
40	0	\$L3
56	0	\$L6

分支历史记录表

sum:

	move	\$2,\$0
	move	\$6,\$0
	b	<u>\$L2</u>
	nop	

\$L3:

	addiu	\$2,\$2,1
	addiu	\$3,\$3,1

\$L4:

40:	slt	\$5,\$3,\$4
	bne	\$5,\$0, <u>\$L3</u>
	nop	

	addiu	\$6,\$6,1
--	-------	-----------

\$L2:

56:	slt	\$3,\$6,\$4
	beq	\$3,\$0, <u>\$L6</u>
	nop	

	move	\$3,\$0
	b	<u>\$L4</u>
	nop	

\$L6:

	jr	\$31
--	----	------

bne 预测正确率
= (110 - 20) / 110
≈ 81.8%

预测失败次数:

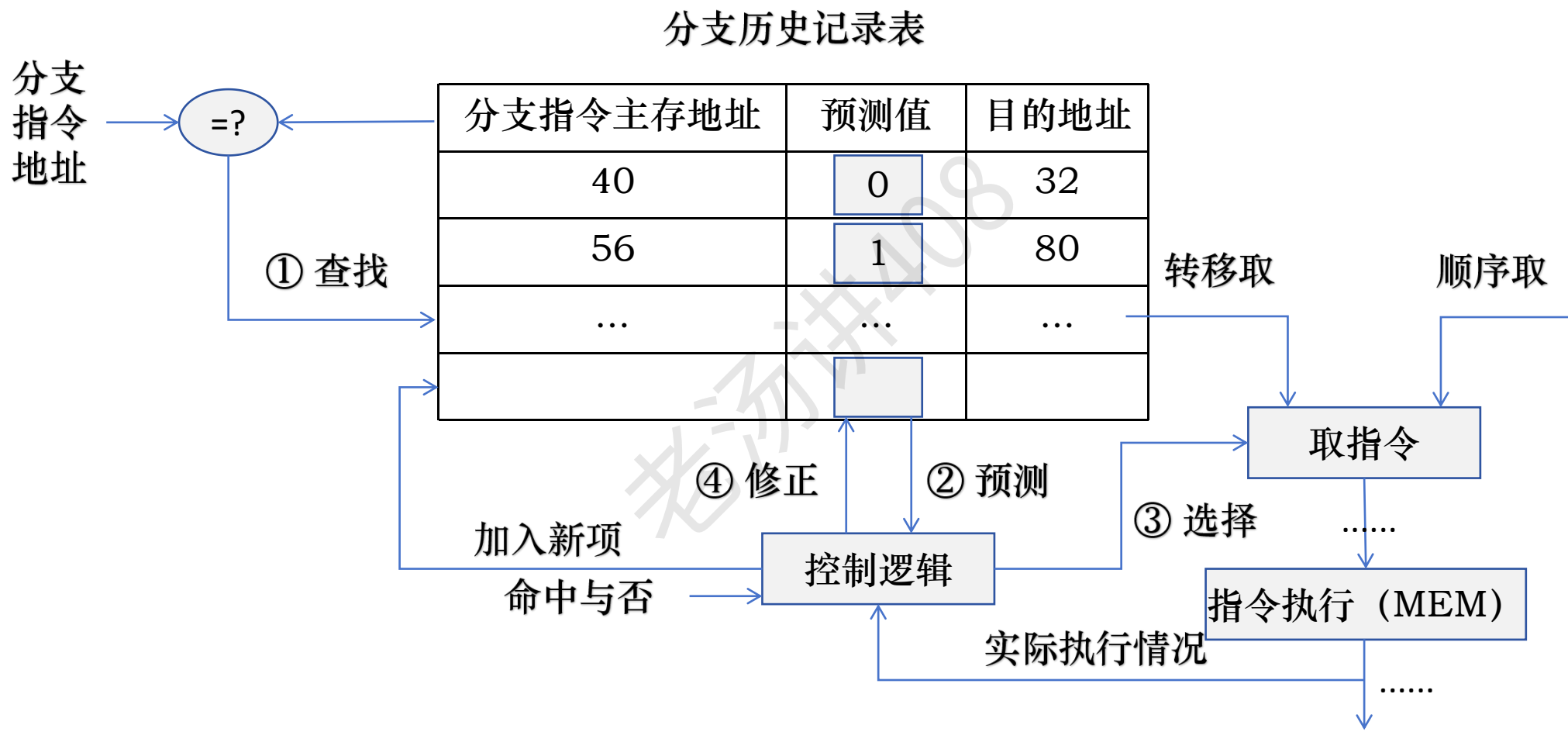
20

预测失败次数:

1

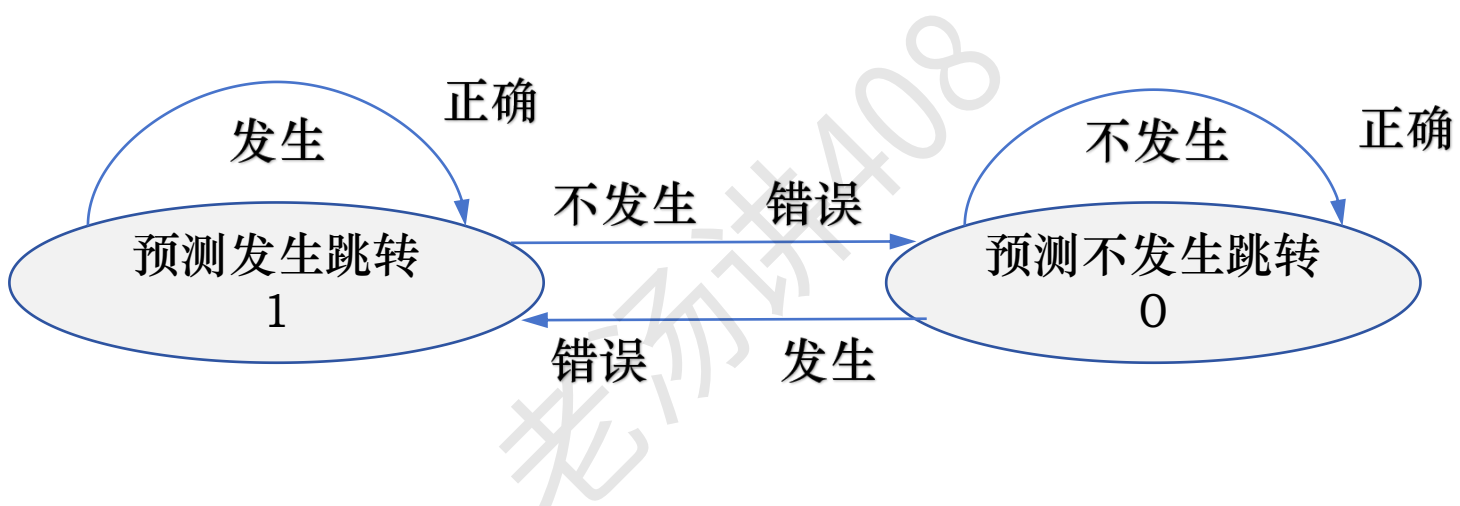
beq 预测正确率
= (11 - 1) / 11
≈ 90.9%

流水线处理器：控制冒险（分支预测 - 动态预测）

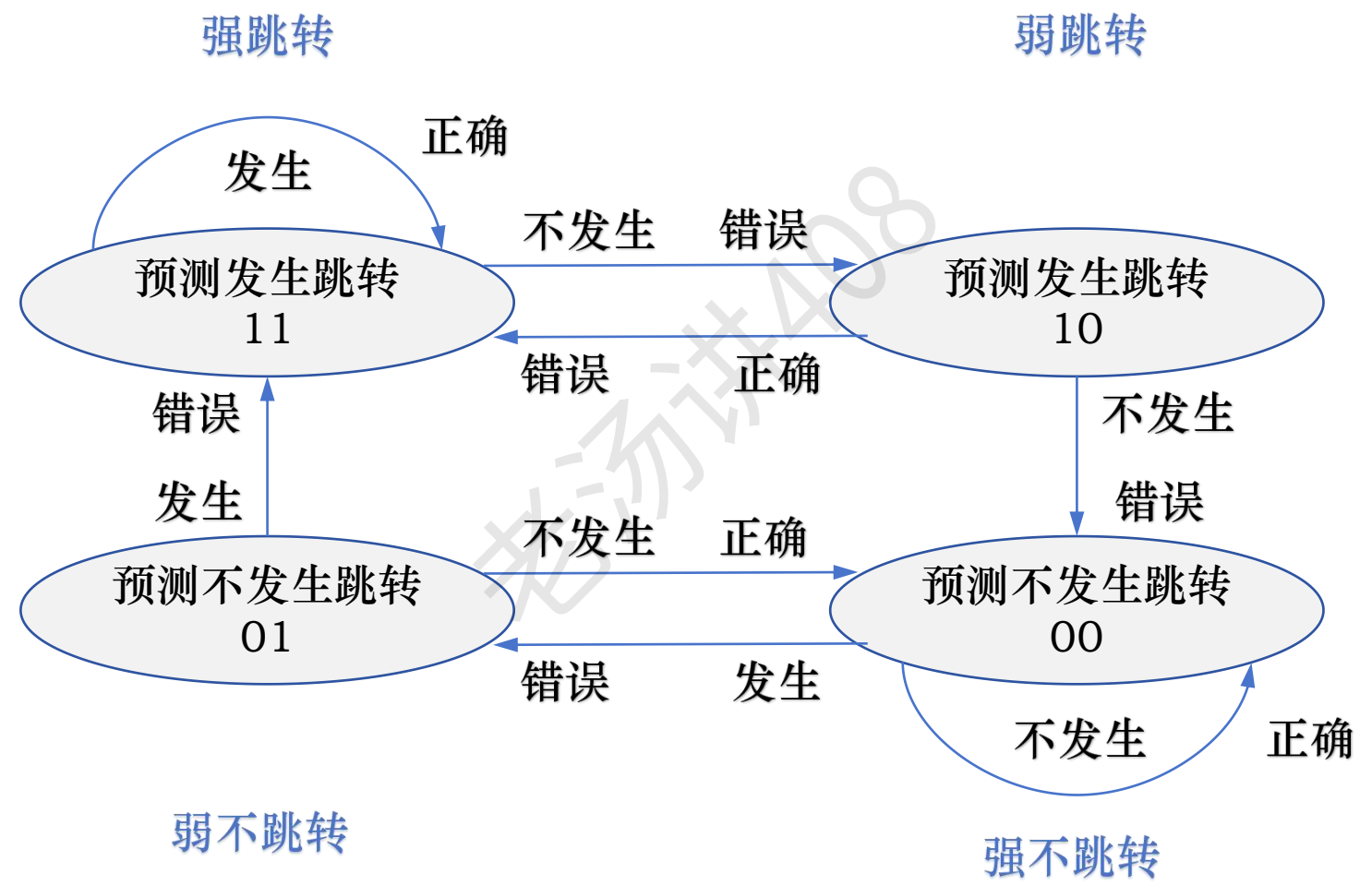


一位动态预测

流水线处理器：控制冒险（分支预测 - 一位动态预测）



流水线处理器：控制冒险（分支预测 - 两位动态预测）



流水线处理器：控制冒险（两位动态预测）

动态预测利用分支指令发生转移的历史情况来进行预测

```
int sum(int n) {  
    int i, j, sum = 0;  
    for (i = 0; i < n; i++)  
        for (j = 0; j < n; j++)  
            sum += 1;  
    return sum;  
}
```

\$6	0	i
\$5		
\$4	10	n
\$3	10	j
\$2	20	sum

00 强不跳转，01 弱不跳转

10 弱跳转，11 强跳转

分支指令主存地址	预测值	目的地址
40	10	\$L3
56	00	\$L6

sum:

```
move    $2,$0  
move    $6,$0  
b       $L2  
nop
```

bne 一位预测正确率
= (110 - 20) / 110
≈ 81.8%

\$L3:

```
addiu   $2,$2,1  
addiu   $3,$3,1
```

\$L4:

```
40: slt    $5,$3,$4  
    bne    $5,$0,$L3  
    nop
```

预测失败次数:

12

```
addiu   $6,$6,1
```

\$L2:

```
56: slt    $3,$6,$4  
    beq    $3,$0,$L6  
    nop
```

bne 两位预测正确率
= (110 - 12) / 110
≈ 89.1%

```
move    $3,$0  
b       $L4  
nop
```

\$L6:

```
jr      $31
```